

变物性药材烘干的守恒建模与全域判定

摘要

针对圆柱形药材烘干中温湿变化不同步、低含水率阶段扩散减慢及收缩域终点判定问题，建立理想化有效参数一维径向传热传质模型。以干基含水率为水分变量，以径向全域最大含水率低于 0.15 为完成条件，区分表面变干、平均脱水与全域达标。

依据全部附件记录重建环境与半径，四小时后环境固定为 50 摄氏度和 0.05 kg/kg。固定域采用守恒有限体积与变物性耦合方程；收缩域采用同调随体坐标，从干质量守恒推导几何因子及干基水分方程。利用表面加密网格、解析稀疏雅可比和隐式 BDF 积分统一求解四问，并由未舍入状态定位终点。

问题一 1800 秒时中心、表面温度分别为 33.5753、36.7856 摄氏度，含水率为 2.5500、1.5102。问题二三小时温差仅 0.1169 摄氏度，而中心、表面含水率仍为 1.7662、1.0081，说明温度接近均匀不能替代水分达标。问题三、四的越阈时间分别为 57.4728、51.0908 小时，满足严格不等式的整秒终点分别为 206902、183927 秒；问题四终点半径为 1.2000 厘米。按题定时空间隔输出全过程，收缩后域外位置留空。

问题四较问题三缩短 11.1043%，但二者物性同时改变。保持问题四物性而固定半径的对照时间为 129.8449 小时，同物性下收缩使时间减少 60.6524%，揭示了两种比较的不同含义。最终 640 区间通过网格与时间容差复验；非均匀移动域 Bessel 基准的最大采样误差为 8.11×10^{-7} ，冻结几何因子的错误算子误差为 5.87×10^{-2} ，被测试拒绝。结论限于声明模型与工况，整秒输出不代表亚秒级误差保证。

关键词：药材烘干；热湿耦合；守恒有限体积；移动边界；全域阈值

1 问题重述与总体分析

1.1 研究对象、变量和四问的递进关系

烘干过程的直接目标是降低药材内部水分，而不是仅使外表面变干或达到环境温度。题设药材初始为长 25 厘米、半径 2 厘米的圆柱，初始温度均匀为 28 摄氏度，初始水分为每千克干物质含 2.55 千克水。后一个量是干基含水率，以下记为 C ，并始终保留其质量基准。数值 $C = 2.55$ 不意味着湿物料中水的质量分数大于一；对应的湿基质量分数为 $C/(1 + C)$ 。题设物性中的这一比值与状态变量本身不能混用，终止阈值 0.15 也作用于干基变量。

问题一要求利用给定环境时间序列及常热物性，计算前 1800 秒内部温度和水分的径向分布，并按指定时空分辨率输出。它构成整个模型的基础情景：几何不变，热传导不受水分反馈影响，而水分扩散系数随含水率变化。问题二引入温湿相关物性，温度通过扩散系数影响脱水，水分又通过有效热容量和导热率影响升温。因而必须求解两组状态的联立初边值问题，而不能先独立求温度、再以固定温度计算全部干燥过程。

问题三沿用问题二的物性与固定半径，把时间范围延伸到所有位置满足含水率要求。此时任务的重点由给定时刻的场分布转向首次满足全域条件的的时间。问题四进一步给出随时间变化的实测半径及另一组物性。其难点不仅是扩散距离缩短，还包括计算域移动、干基变量的输运解释，以及固定物理位置是否仍在药材内部。本文分别输出名义结果和同物性几何对照，避免将多因素差异解释成单因素效应。

表1概括四问的递进关系。四问共用径向守恒离散与边界符号，分别改变物性、输出范围及几何。问题二、三共享同一个从初态出发的完整解，问题四则同时改变物性与几何；这一结构决定了哪些结果可以直接比较、哪些差异需要另设对照。

表 1: 四问的模型变化、求解目标与结果对应

问题	物性与几何	核心求解目标	正文数值表
一	常热物性, $D(C)$, 固定半径	前 1800 秒温度、水分径向场	5、6
二	温湿相关物性, 固定半径	全过程耦合场及前三小时摘录	7、8
三	完全沿用问题二	全域含水率首次越阈及整秒终点	9
四	另一组物性, 给定 $R(t)$	移动域越阈、实际表面及域内输出	10

本文的处理重点是三处一致性：用共享面通量保证离散水分收支；用干基变量与随体坐标保证收缩运动的解释一致；用最大值事件及同物性对照保证终点判定和几何归因一致。它们是经典方法针对本题的组合与细化，不以算法名称的新颖性替代数值证据。题目未提供内部观测，因此不进行参数回归；环境、半径是输入，内部场与终点是模型输出。

1.2 空间非均匀性与模型选择

圆柱侧面与环境发生热量和水分交换，中心没有跨轴线的净通量。升温时，热量从表面向内传递；脱水时，内部水分向表面迁移。这两种过程的特征时间未必相同。因此，截面平均温度接近环境不能推出中心已干燥，表面达到含水阈值也不能代替全截面判定。本文保留径向坐标，以全域最大含水率作为终点变量，同时保存体积权均值与表面值解释空间差异。

本文研究轴向与周向均匀、径向允许非均匀的一维理想场。轴向和周向不设独立状态，题给长度保持不变。下文“全域”均指所定义的径向场，不将有限圆柱的端部效应另设为待求机制，也不以长径比推断未计算的三维误差。

已有圆柱干燥研究比较过有限外部传质阻力与表面直接给定状态的边界形式 [1]。题目给定热、质传递系数，因此本文保留 Robin 边界，不把表面温度强制等于空气温度，也不把表面含水率强制等于环境记录。变物性耦合干燥可用隐式有限体积方法求解 [2]；本文采用该类方法的守恒通量思想，但所有材料函数均取自题目，不迁移文献中其他食品的经验参数。

2 数据盘点、单位与边界重建

2.1 全量输入检查

附件 1 含 241 条环境记录，时间从 0 到 14400 秒，等间隔 60 秒。温度从 28 摄氏度升至接近 50 摄氏度，水分记录从 0.01963 升至约 0.05。附件 2 含 145 条半径记录，时间从 0 到 259200 秒，等间隔 1800 秒，半径单位为厘米。两份数据均无缺失值、非有限数值或重复时间，时间严格递增；原始半径非增，具有 53 个不同数值和 92 次相邻重复。这些重复反映测量分辨率及后期平台，不是需要删除的重复记录。

表2给出数据的作用和处理约定。计算中使用全部 241 条环境记录和全部 145 条半径记录，不为得到平滑曲线删除局部波动。热湿状态计算采用国际单位制，半径进入微分方程前除以 100，时间保持秒；展示时才转换为厘米或小时。这里的单位转换尤其重要，因为扩散特征时间与半径平方成正比，误将厘米当米会使几何尺度发生严重偏差。

表 2: 原始附件与输出模板的数据合同

对象	记录数	时间范围/h	角色与处理
附件 1	241	0–4	温度与有效平衡含水率边界；全量线性插值
附件 2	145	0–72	实测半径；厘米转米，保留全部平台记录
结果模板 1、2	两工作表	见正文	温度、水分分别输出；每秒和每 0.1 厘米
结果模板 3、4	单工作表	至严格终点	含水率每 60 秒输出，并补入最终整秒

图1展示全部环境输入。早期环境显著变化，不能用最终 50 摄氏度替代整个过程；

后期小幅波动也不构成内部状态的实验验证。图中的有效平衡含水率表示本文约定的边界数值 $C_e = W_a$ ，其中 W_a 为附件水分记录。该映射是有效传质模型的定义，不表示空气与固体的质量基准相同，也不声称由本题数据识别出了真实吸附平衡关系。

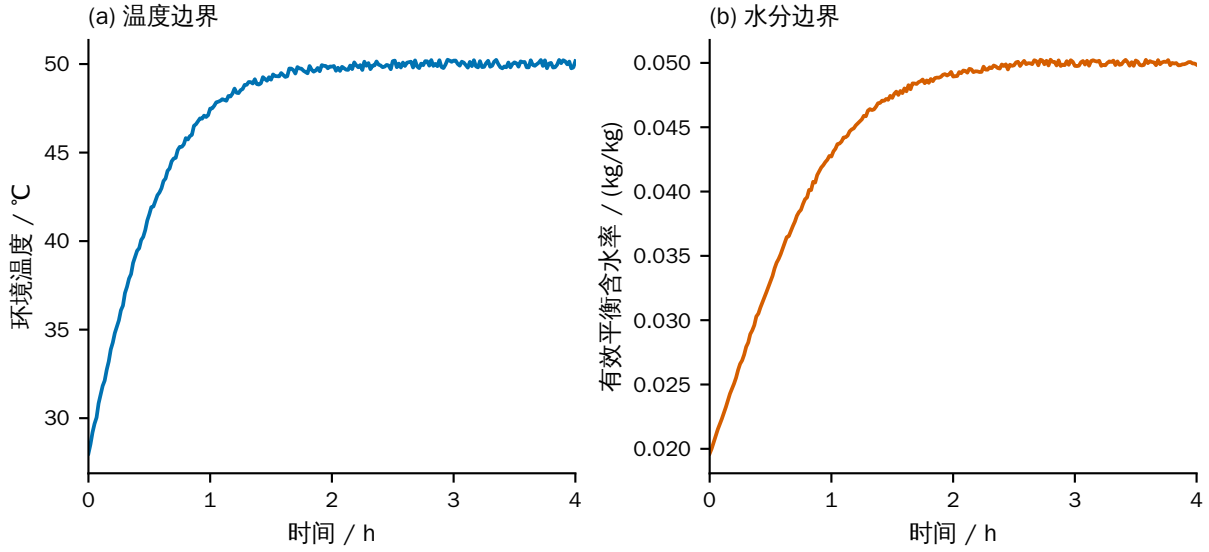


图 1: 环境边界输入：全部 241 条记录分别重建温度与有效边界 C_e ，两种量采用独立纵轴面板。

2.2 分段插值与四小时后的名义环境

记环境温度为 $\theta_\infty(t)$ ，有效平衡含水率为 $C_e(t)$ 。对任意相邻实测时刻 t_j, t_{j+1} ，采用式(1)重建二者。线性插值保留原始端点，避免高阶多项式在边界波动处产生未经数据支持的过冲，也无需选择平滑强度或拟合阶数。插值函数连续但导数可以在记录时刻改变，这是对离散采样信息的明确使用，而不是宣称真实环境只有分段线性变化。

$$b(t) = b_j + \frac{t - t_j}{t_{j+1} - t_j}(b_{j+1} - b_j), \quad t_j \leq t \leq t_{j+1}, \quad b \in \{\theta_\infty, C_e\}. \quad (1)$$

问题三、四的终点远晚于环境记录末端，必须声明延拓方法。名义情景在 $t > 14400$ 秒取 $\theta_\infty = 50$ 摄氏度、 $C_e = 0.05$ ；在 $t = 14400$ 秒仍使用实测末值。求解器在四小时处分段重启，因此名义延拓与末条记录间的微小跳变不会被无意混成一条跨越边界的拟合曲线。该做法是未观测时间段的工况假设，不是补造测量值。

用最后一小时，即 $t \geq 10800$ 秒的 61 条记录检查稳定水平，温度均值为 49.998934 摄氏度、样本标准差为 0.154353 摄氏度；水分记录均值为 0.04998754、样本标准差为 0.00015013。两均值与名义水平接近，说明这一延拓与已观测末段相容。但相邻记录具有时间结构，不能把 61 条数据简单视为独立重复实验，进而构造整段未来环境的置信区间。本文仅用末值延拓、末小时均值延拓及人为指定的小扰动检验结论的情景依赖性。

2.3 半径重建与域内输出

图2给出附件 2 全部半径数据。半径从 2 厘米快速下降，6 小时为 1.374 厘米，12 小时为 1.248 厘米，24 小时为 1.204 厘米，之后变化减缓；记录最终值为 1.198 厘米。本文直接对这些记录作分段线性插值，不另拟合指数收缩律。经验拟合虽可减少参数数量，却会改变题给的几何轨迹，并使终点误差同时包含曲线拟合误差；本题没有这一必要。

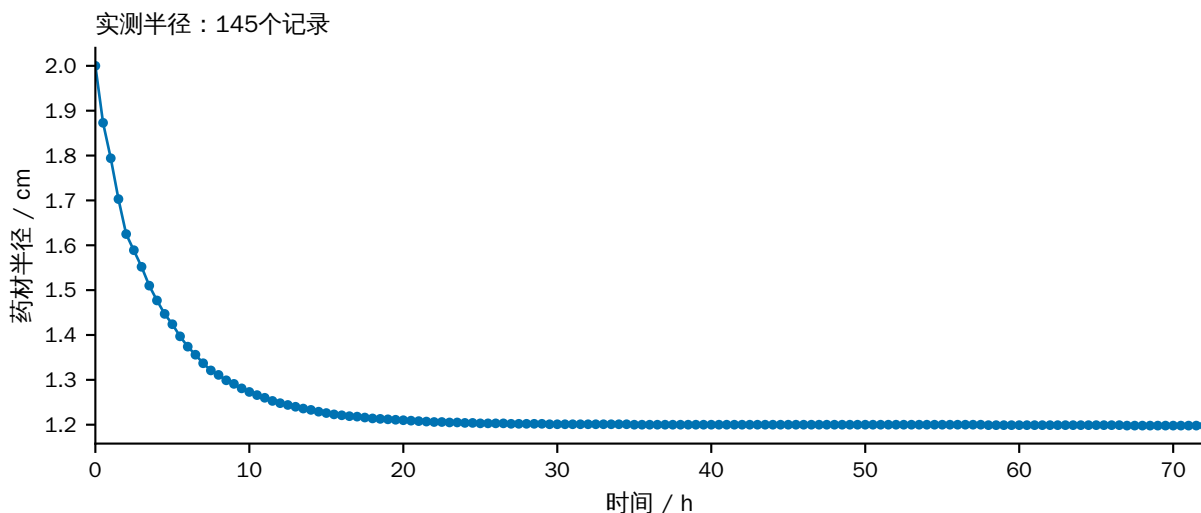


图 2: 几何输入：附件 2 全部 145 条半径记录及分段线性重建，保留平台段，不向 72 小时外延拓。

收缩后，同一个物理半径位置可能先在药材内，随后离开计算域。输出位置 r 只有在 $r \leq R(t)$ 时才有温度或含水率意义。域外值不等于零，也不是随机缺测，而是该时刻没有药材。问题四工作簿因此保留 0 至 1.9 厘米的固定位置列，额外设置随 $R(t)$ 变化的真实表面列。域外单元格为空；论文表格用破折号表示相同含义。早期 1.5 厘米处仍可能有值，只有题目表 6 从 6 小时开始的各行，该位置才全部在药材外。

问题一工作簿输出 1 至 1800 秒。问题二的温度、水分浓度两工作表均从 1 秒输出至问题三严格整秒终点 206902 秒，时间间隔 1 秒，半径位置为 0、0.1、...、2.0 厘米；每表有 206902 个时间行。题目表 3、表 4 在论文中仍只列 0.5 至 3.0 小时。问题二全过程逐秒文件与问题三逐分钟文件是同一初边值解的两种输出分辨率，不相互替代。问题三、四每 60 秒输出一次，并补入不重复的严格整秒终点，保证最后一行与终止判据一致。

3 模型假设、物性与热湿耦合方程

3.1 状态、符号及必要闭合

记 $\theta(r, t)$ 为摄氏温度， $C(r, t)$ 为干基含水率， $R(t)$ 为外半径， ρ_h 为仅参与有效热容量的经验参数， c_p 、 k 、 D 分别为题设比热、导热率和水分扩散系数。另用 ρ_d 表示质量

守恒中的单位当前体积干物质质量，二者不混用。绝对温度仅在 Arrhenius 指数中使用，定义为 $T_K = \theta + 273.15$ 。表3集中列出单位，避免把传热系数 h 与时间小时混同，或把以米每秒计的传质系数直接当成质量通量。

表 3: 主要符号与量纲

符号	含义	单位
θ, θ_∞	药材温度、环境温度	摄氏度
C, C_e	干基含水率、有效平衡含水率	kg/kg
$r, R, x = r/R$	物理半径、外半径、归一化半径	m、m、1
ρ_h, c_p	热容量经验参数、比热	kg/m ³ 、J/(kg K)
ρ_d	单位当前体积干物质质量	kg/m ³
k, D	导热率、水分扩散系数	W/(m K)、m ² /s
h, h_m	表面对流换热、有效传质系数	W/(m ² K)、m/s
$A = \rho_h c_p$	有效体积热容量	J/(m ³ K)
t_*, t_{int}	越阈事件、严格整秒终点	s
$\bar{C}, C_{\text{max}}$	径向体积权均值、全域最大值	kg/kg

本文建立理想化、有效参数的一维径向传热传质模型，目标是在题给数据和约定条件下描述变化规律，而非还原特定药材的完整真实物理过程。表4集中给出模型闭合；后续推导、检验和对照均保持这些定义，不另行改变模型族。

表 4: 模型闭合与各项约定承担的作用

对象	明确约定	在模型中的作用
空间与几何	轴向、周向均匀，径向可非均匀；长度不变	只求径向场；“全域”限于该场
外部水分	$C_e = W_a$ 为有效边界定义	将附件记录用于 Robin 传质驱动力，不识别真实吸附平衡
能量	忽略蒸发潜热，无内部热源	采用 $A(C)\theta_t$ 的有效热容量闭合
两类密度	ρ_h 只用于 $A = \rho_h c_p$ ； ρ_d 用于干质量守恒	不附加 $\rho_h = (1 + C)\rho_d$ 关系
未观测工况	$t > 4$ 小时取 50 摄氏度、0.05 kg/kg	定义长期边界；以情景扰动评估其影响
问题四运动	同调径向收缩， $x = r/R(t)$ 为材料坐标	外半径轨迹确定内部速度 $v = r\dot{R}/R$
干固体	初始密度空间均匀、干质量不流失	得到 $\rho_d(t)R(t)^2 = \rho_{d0}R_0^2$

其中同调性并非由外轮廓数据反演所得，而是使移动域方程闭合的运动定义。忽略潜热也不意味着题给经验参数已经吸收了该机制。热、质传递系数分别取题给 $h = 25$ W/(m² K) 和 $h_m = 8 \times 10^{-7}$ m/s，四问均保持不变。

3.2 固定半径下的控制方程

对于问题一至三, $0 \leq r \leq R_0$ 、 $R_0 = 0.02$ 米。采用式(2)给出局部热湿传递, 题设变系数保留在散度内部。若直接写成 k 乘拉普拉斯算子, 会遗漏导热率梯度的作用; 若把 $1/(\rho_h c_p)$ 也放进散度, 则改变了原来的热容量模型。两种误写在常系数时可能看得出来, 在问题二至四中会影响温湿反馈。

$$\begin{aligned} A(C) \frac{\partial \theta}{\partial t} &= \frac{1}{r} \frac{\partial}{\partial r} \left(r k(C) \frac{\partial \theta}{\partial r} \right), \\ \frac{\partial C}{\partial t} &= \frac{1}{r} \frac{\partial}{\partial r} \left(r D(\theta, C) \frac{\partial C}{\partial r} \right). \end{aligned} \quad (2)$$

初始条件见式(3)。温度初始与空气温度相同, 初始热通量为零; 含水率初始显著高于有效平衡值, 表面在开始时立即向外传质。均匀初值的零径向梯度与初始 Robin 水分通量存在角点不相容, 这是扩散初边值问题常见的短时边界层, 不应通过修改初始表面含水率来消除。隐式积分从均匀初态发展该边界层, 早期网格和步长检查因而不可省略。

$$\theta(r, 0) = 28, \quad C(r, 0) = 2.55, \quad 0 \leq r \leq R_0. \quad (3)$$

中心满足轴对称条件, 表面采用有限交换阻力的 Robin 条件, 见式(4)。符号按外法向为正定义: 脱水时 $C_s > C_e$, 向外水分通量为正; 升温时 $\theta_s < \theta_\infty$, 向外热通量为负。这一符号约定使两类边界共享同一通量实现, 减少在热量进入、质量流出之间分别判断符号的风险。

$$\begin{aligned} \theta_r(0, t) &= 0, \quad C_r(0, t) = 0, \\ -k_s \theta_r(R_0, t) &= h(\theta_s - \theta_\infty), \quad h = 25, \\ -D_s C_r(R_0, t) &= h_m(C_s - C_e), \quad h_m = 8 \times 10^{-7}. \end{aligned} \quad (4)$$

式(2)中的 $A(C)\theta_t$ 不能替换为 $\partial_t[A(C)\theta]$, 后者多出 $\theta A'(C)C_t$, 会改变表4定义的能量方程。本文因此检查通量离散与该有效方程的一致性, 并对于基水分单独建立守恒关系, 不把它扩展为未定义的多相总焓验证。

3.3 三组题设物性

问题一采用式(5)。虽然热方程为常系数线性扩散, 水分方程仍是非线性的: 随着表面干燥, D 会减小, 局部扩散阻力逐渐增加。因此问题一也不能使用一个常数扩散系数替代整个水分过程。

$$\rho_h = 820, \quad c_p = 2600, \quad k = 0.36, \quad D = 7 \times 10^{-9} \exp(-0.89/C). \quad (5)$$

问题二、三共享式(6), 其中指数温度必须使用开尔文。水分降低使有效热容量和导热率变化, 温度升高则通过指数项促进水分扩散。这种双向作用要求在每一次隐式迭代

中依据当前状态更新物性，不能仅在整秒输出时更新。问题二从 $t = 0$ 即使用本组物性，不是在问题一半小时末状态上切换。

$$\begin{aligned} \rho_h &= 650 + 128C, & c_p &= 1450 + \frac{2736C}{1+C}, \\ k &= 0.21 + \frac{0.38C}{1+C}, & D &= 0.0024 \exp\left(-\frac{0.45}{C} - \frac{3850}{T_K}\right). \end{aligned} \quad (6)$$

问题四使用式(7)，并同时引入实测半径。它不是仅把问题三的 R_0 换成 $R(t)$ ，因为所有有效物性函数也发生变化。后文设置保持式(7)而固定半径的反事实，以区分几何与物性的作用。

$$\begin{aligned} \rho_h &= 760 + 90C, & c_p &= 1850 + \frac{2150C}{1+C}, \\ k &= 0.12 + \frac{0.20C}{1+C}, & D &= 0.00042 \exp\left(-\frac{0.30}{C} - \frac{3850}{T_K}\right). \end{aligned} \quad (7)$$

图3只展示式(6)在 28 和 50 摄氏度下的函数值，不是新增材料测量。高水分区扩散系数变化较缓，接近有效边界低含水率时则跨越多个数量级。这说明干燥后期可能由低水分表层形成的高阻力控制，而非保持初始扩散速度。与柱状食品热湿耦合研究中的温度依赖处理相一致 [3]，本文保留 Arrhenius 结构；文献在此仅说明方法背景，不作为题给物性或一维假设的实验证据。

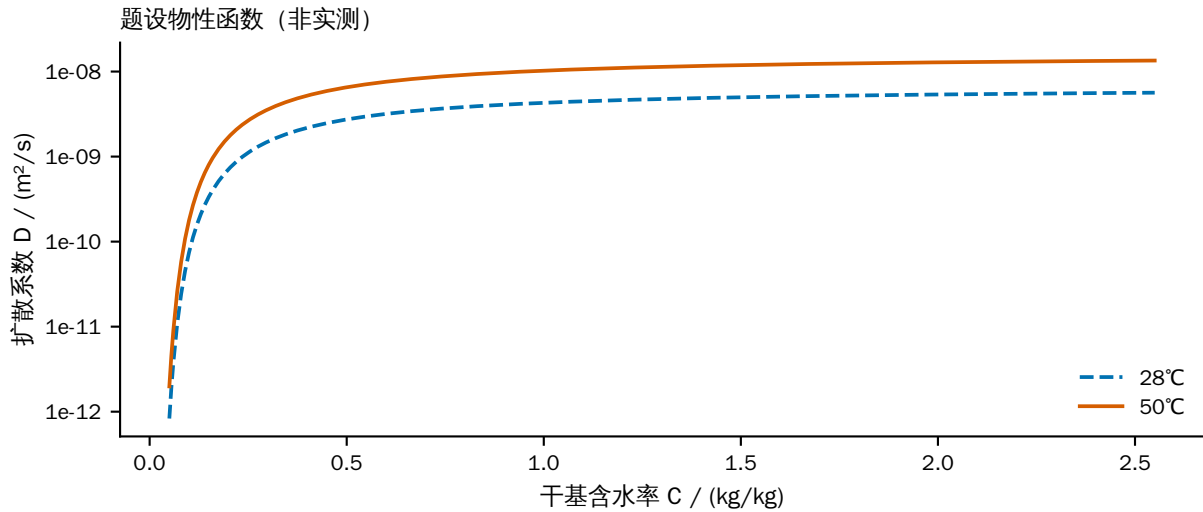


图 3: 问题二、三的题设扩散函数：28 与 50 摄氏度下，低含水率区的 D 迅速下降；纵轴为对数刻度，非实验数据。

4 守恒有限体积与隐式求解

4.1 表面加密网格及控制体

令 $x = r/R$ ，固定域时 $R = R_0$ 。使用包含中心和表面的 $N + 1$ 个节点，按式(8)向表面加密。这里 N 是区间数，不是节点数；近表面的较密节点用于分辨后期含水率陡降区域。网格分配只说明分辨率放在哪里，精度是否足够仍由多网格比较确定，本文不据此宣称相对均匀网格的等精度计算效率优势。

$$x_i = 1 - \left(1 - \frac{i}{N}\right)^2, \quad i = 0, \dots, N; \quad x_{i+1/2} = \frac{x_i + x_{i+1}}{2}, \quad w_i = \frac{x_{i+1/2}^2 - x_{i-1/2}^2}{2}. \quad (8)$$

两端面分别取 $x_{-1/2} = 0$ 和 $x_{N+1/2} = 1$ 。 w_i 来自柱坐标体积权重 $\int x dx$ ，而不是简单的节点间距；因此靠外的环形控制体与中心控制体具有不同权重。全部权重之和为 $1/2$ ，径向平均可以直接计算为 $2 \sum_i w_i C_i$ 。若把这些节点当成等体积样本求算术平均，会偏置截面平均含水率。

对一般变量 u 、扩散系数 a ，在相邻节点之间使用调和平均 $a_{i+1/2}$ ，构造式(9)的向内通量记号 G 。两个半段串联的扩散阻力对应调和平均，尤其有利于保留低扩散系数表层的阻滞作用。这里的 G 包含径向面权重，但尚未除以控制体和 R^2 ；它不是以千克每秒计的实际出水速率。

$$\begin{aligned} a_{i+1/2} &= \frac{2a_i a_{i+1}}{a_i + a_{i+1}}, \\ G_{i+1/2} &= x_{i+1/2} a_{i+1/2} \frac{u_{i+1} - u_i}{x_{i+1} - x_i}, \\ G_{-1/2} &= 0, \quad G_{N+1/2} = R\beta(u_\infty - u_N). \end{aligned} \quad (9)$$

温度取 $(u, a, \beta, u_\infty) = (\theta, k, h, \theta_\infty)$ ，水分取 (C, D, h_m, C_e) 。于是半离散方程统一为式(10)，其中温度方程的 $q_i = A_i$ ，水分方程的 $q_i = 1$ 。相邻控制体使用同一个面通量且符号相反，内部通量在全域求和时精确相消，这是归一化水分平衡的基础。

$$\frac{du_i}{dt} = \frac{G_{i+1/2} - G_{i-1/2}}{R^2 w_i q_i}. \quad (10)$$

中心的 $1/r$ 表观奇点不通过人为加一个小半径回避，而由中心控制体积分自然消除。由 $x_{1/2} = x_1/2$ 可得中心极限离散为式(11)，系数为 4 而不是平板几何的 2。这个差异若写错，会影响中心升温与最终中心脱水时间，却未必在表面曲线上明显暴露。

$$\frac{du_0}{dt} = \frac{4a_{1/2}(u_1 - u_0)}{R^2 x_1^2 q_0}. \quad (11)$$

4.2 刚性积分与解析雅可比

加密网格使最小空间尺度变小，而低含水率又使不同位置的扩散时间相差很大，半离散系统具有刚性。本文使用 BDF 隐式积分，并提供解析稀疏雅可比。状态向量同时包含全部温度、水分以及用于平衡检查的累计归一化损失；各节点只与相邻节点直接耦合，雅可比具有稀疏块结构。热湿交叉块不能忽略： C 改变热容量与导热率， θ 改变水分扩散系数。

以调和平均 $H(a, b) = 2ab/(a + b)$ 为例，其导数及问题二、三的 D 导数见式(12)。温度方程还必须包含热容量分母求导产生的修正 $-\dot{\theta}_i A'_i / A_i$ 。若只对导热通量求导，得到的不是完整雅可比，可能降低长时稳态附近的求解稳定性。解析形式同时避免靠数值差分估计极小导数时产生不必要的运行警告。

$$\begin{aligned} \frac{\partial H}{\partial a} &= \frac{2b^2}{(a+b)^2}, & \frac{\partial H}{\partial b} &= \frac{2a^2}{(a+b)^2}, \\ \frac{\partial D}{\partial \theta} &= D \frac{3850}{T_K^2}, & \frac{\partial D}{\partial C} &= D \frac{0.45}{C^2}. \end{aligned} \quad (12)$$

名义相对容差取 10^{-8} ，温度绝对容差 10^{-8} 摄氏度，含水率及累计损失绝对容差 10^{-10} 。前四小时最大时间步为 15 秒，之后为 600 秒；步长由隐式误差估计自动控制，最大值不是实际固定步长。四小时处分段处理环境延拓。半径曲线虽然在采样点改变斜率，但在随体方程中仅以连续的 $R(t)$ 进入，求解器不需要用不稳定的数值差分估计半径导数。

输出时间与内部积分步不同。程序利用每一积分段的连续输出，在题目指定时刻采样，再用保形分段三次插值映射到指定物理半径。对于单调的径向含水率剖面，保形插值不产生超出端点的新极值。输出四位小数只发生在 Excel 和论文展示阶段，误差比较、阈值判定及均值积分始终使用未舍入浮点状态。读取跨越分段边界或顺序不规则的请求时刻时，程序逐段分配并检查每个时刻均被覆盖，避免遗漏或读取未初始化状态。

5 问题一：常热物性下的短时温湿分布

5.1 指定位置与时刻的计算结果

按照题目要求，表5和表6列出 100、300、600、900、1200、1500、1800 秒，半径 0、0.5、1.0、1.5、2.0 厘米处的温度与干基含水率。表中四位小数来自最终 $N = 640$ 计算，不由较粗网格结果四舍五入后再插值得到。完整逐秒结果同时保存在结果工作簿 1 的温度和水分两个工作表中，每张表均具有 21 个径向位置。

1800 秒时中心温度为 33.5753 摄氏度，表面为 36.7856 摄氏度，两者相差 3.2102 摄氏度；中心水分仍为 2.5500，表面已降至 1.5102。中心显示初始值不意味着数值状态完全没有变化，未舍入值为 2.549992388。热扩散已影响整个截面，水分显著下降则主要集

表 5: 问题一指定时刻的温度（对应题目表 1，单位：摄氏度）

时间/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8488
600	28.4533	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1126	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7856

表 6: 问题一指定时刻的干基含水率（对应题目表 2，单位：kg/kg）

时间/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	2.5500	2.5500	2.5500	2.5500	2.2470
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

中在近表面区域。由此可见，在给定的半小时观测窗口内，把全截面视为均匀水分会掩盖内部尚未脱水的事实。

图4选取 100、600 和 1800 秒剖面解释这一差异，选点对应表中真实计算时刻。温度剖面在中心附近平坦，符合对称边界；外层温度较高，反映外部加热。水分剖面方向相反，表面最低，向中心逐渐恢复到初值。图中每条曲线均来自同一个数值场，不通过独立拟合各个位置构造，因而与微分方程的通量关系一致。

5.2 传递尺度及边界阻力解释

问题一热扩散率为 $k/(\rho_h c_p) = 1.6886 \times 10^{-7}$ 平方米每秒，初始含水率下的水分扩散系数为 4.9377×10^{-9} 平方米每秒。两者相差约 34 倍，说明在相同几何尺度下热量比水分更快影响内部，但这一比较只是初态量级，水分扩散系数会继续下降。以半径为长度尺度定义式(13)，热 Biot 数为 1.3889，并不处于内温度梯度可忽略的小量范围。

$$\alpha = \frac{k}{\rho_h c_p}, \quad \text{Bi}_T = \frac{hR}{k}, \quad \text{Bi}_C = \frac{h_m R}{D}. \quad (13)$$

初始水分 Biot 数约为 3.2404，当局部 $C = 0.15$ 时，题设 D 降至 1.8547×10^{-11} 平方米每秒，按同一半径计算的水分 Biot 数约为 862.65。这不是说问题一在 1800 秒已达

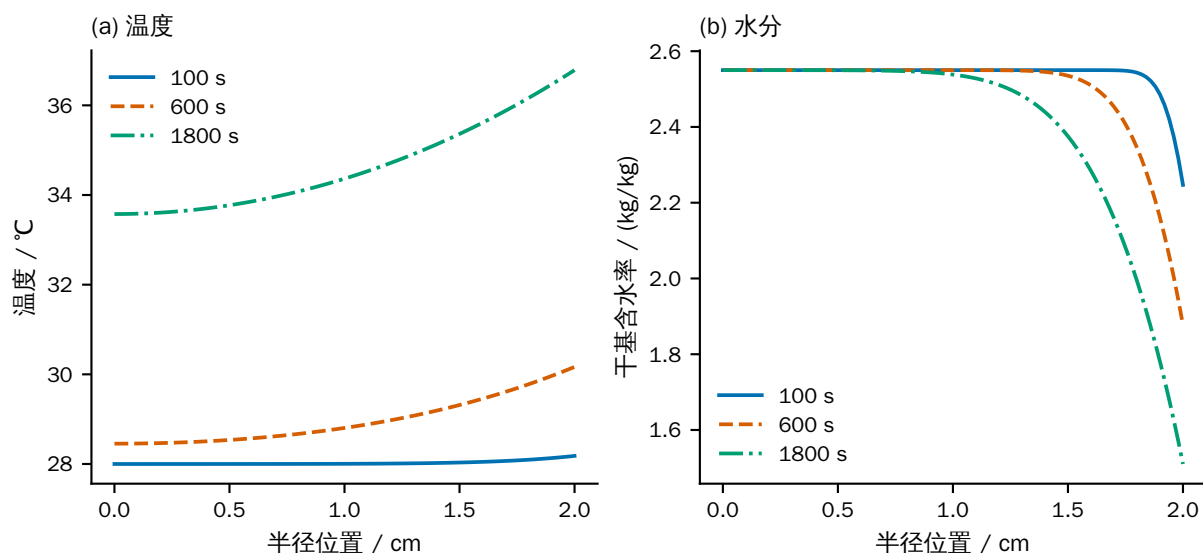


图 4: 问题一数值剖面: 100、600 及 1800 秒时, 外层温度较高而含水率较低; 同一时刻在两面板中采用相同线型与颜色。

到 0.15, 而是用题设函数说明低水分阶段可能出现很强的内部阻力。该量随位置和时间变化, 不能以一个全程恒定的 Biot 数替代非线性模型。

表面含水率在短时间内迅速下降, 但 Robin 条件始终保留外部阻力。直接把表面水分强制设为 C_e 将使开始时表面几乎瞬时降到约 0.02, 与当前有限 h_m 模型中的 1.5 至 2.2 量级明显不同。类似地, 强制表面温度等于空气会夸大早期进入药材的热流。题设提供的传递系数不仅是一个修正参数, 而是决定表面状态如何响应环境的必要边界信息。

问题一温度方程不依赖水分, 因此可以作为热传导模块的基准; 水分方程虽然与温度解耦, 却仍需逐节点更新 $D(C)$ 。这一结构说明“热湿模型”不一定在所有情景都双向耦合。本文保留同一数值接口, 允许问题一的交叉雅可比块自然为零, 而不人为制造题目未给出的温度依赖。

6 问题二：变物性热湿耦合过程

6.1 前三小时的结果与反馈

问题二的六个指定时刻结果见表7和表8。与问题一半小时结果相比, 此处中心、表面温度分别为 32.1892 和 35.4130 摄氏度, 均较低; 表面含水率却为 1.6486, 高于问题一的 1.5102。两模型同时改变了热容量、导热率和扩散函数, 所以不能把差值单独解释为某一个物性参数的贡献。需要强调的是, 这一比较并没有重新估计传热或传质系数。

到三小时, 中心和表面温度分别为 49.8495 和 49.9664 摄氏度, 径向温差仅约 0.1169 摄氏度; 含水率分别为 1.7662 和 1.0081, 仍相差约 0.7581。在本模型中, 温度接近均匀而水分仍有明显梯度, 因此温度接近环境不能替代全域含水率判据。

图5将前三小时的温度和含水率画在同一时间、半径坐标上, 各自使用与变量含义

表 7: 问题二指定时刻的温度 (对应题目表 3, 单位: 摄氏度)

时间/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5000	32.1892	32.3821	32.9659	33.9608	35.4130
1	40.3816	40.5536	41.0605	41.8782	42.9976
1.5000	45.8468	45.9348	46.1930	46.6051	47.1401
2	48.4502	48.4881	48.5984	48.7736	49.0033
2.5000	49.4670	49.4792	49.5136	49.5653	49.6609
3	49.8495	49.8553	49.8746	49.9101	49.9664

表 8: 问题二指定时刻的干基含水率 (对应题目表 4, 单位: kg/kg)

时间/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5000	2.5499	2.5489	2.5256	2.3257	1.6486
1	2.5257	2.4948	2.3578	2.0230	1.4711
1.5000	2.3861	2.3256	2.1344	1.8020	1.3476
2	2.1709	2.1084	1.9236	1.6259	1.2311
2.5000	1.9566	1.9006	1.7360	1.4720	1.1166
3	1.7662	1.7165	1.5702	1.3333	1.0081

相符的色标。温度场由外向内逐渐升高，水分场由表面向内逐渐降低；两者不能共用一个数值色标。保留全时段数据还可看出，升温接近完成的阶段并非水分下降终止，而是干燥逐渐进入由内部扩散和表层阻力共同控制的阶段。

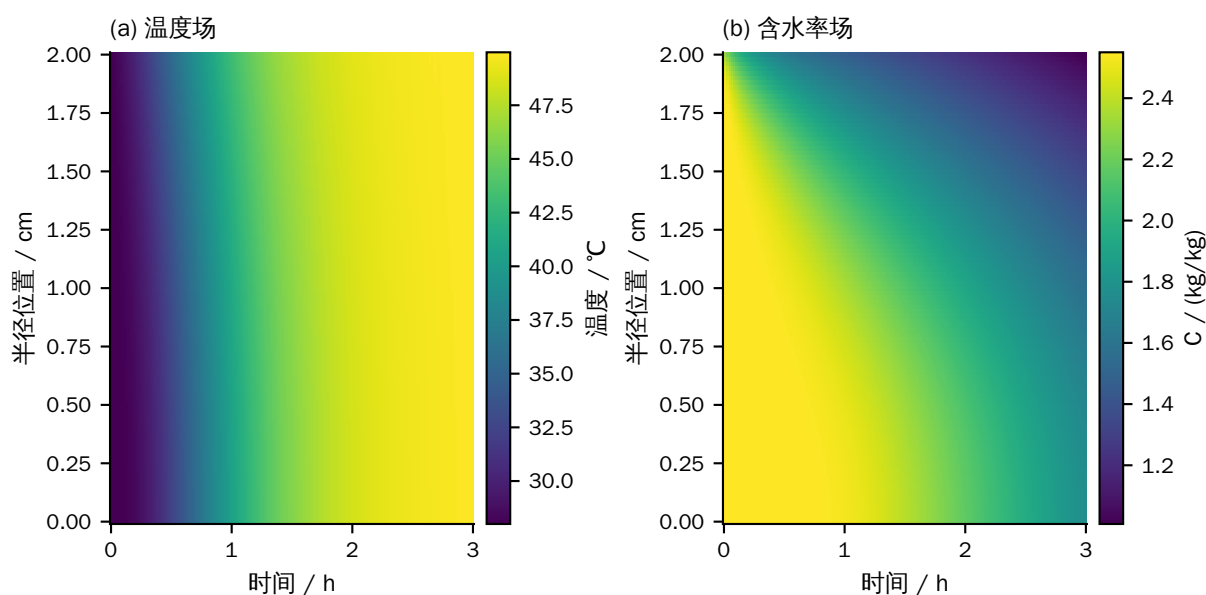


图 5: 问题二前三小时数值场: 按 60 秒轨迹及 101 个径向重建点展示, 温度与水分分别使用带单位色标; 水分梯度持续晚于升温过程。

6.2 扩散函数沿计算轨迹的变化

问题二的 D 同时随 θ 增加而增大、随 C 降低而减小。二者方向相反，不能仅凭“温度越高扩散越快”判断实际轨迹中的 D 必然单调上升。图6把已经求得的温湿场代回题设函数，展示前三小时的局部扩散系数。这是一张模型过程图，色标采用对数形式，描述计算状态的有效输运能力，不是拟合得到的参数分布。

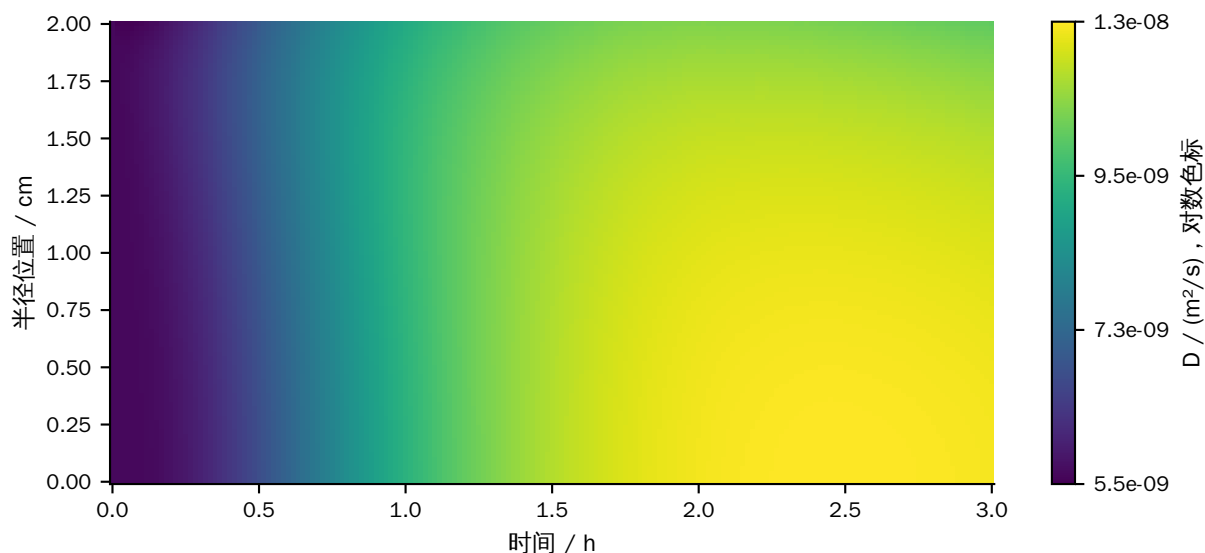


图 6: 问题二前三小时的扩散系数轨迹: 将同一温湿解代入题设 $D(\theta, C)$, 使用对数色标, 不是参数拟合或新增测量。

例如，在 50 摄氏度、 $C = 2.55$ 时，题设扩散系数约为 1.3471×10^{-8} 平方米每秒；同温度下 $C = 0.15$ 时为 8.0012×10^{-10} ，到 $C = 0.05$ 时进一步降至 1.9833×10^{-12} 。这些函数值说明，后期接近有效边界含水率的表层即使已升温，也可能成为低扩散区。三小时图像主要展示升温阶段，不能拿其中较高的 D 作为五十多小时全程的平均扩散系数。

热物性的变化同样不是单向地“加速”或“减速”。水分减少时体积热容量下降，有利于同等净热流引起更快温度变化；导热率也下降，又可能限制内部热通量。实际热扩散率是二者之比，表面传递阻力还通过 Biot 数参与。因此本文不对单个系数作脱离方程的优劣排序，而用联立解给出温度场和水分场，并在问题三继续该初边值问题到全域阈值。

问题二与问题三在整个烘干过程中是同一初边值解的不同输出。程序直接从问题三使用同一物性、初值和边界的积分中采样问题二的全部 206902 个逐秒时刻，防止分别设置参数时发生不一致；精度检查也遍历全过程共同逐秒请求点。这样的结果复用不减少问题二要求的时间分辨率，更不意味着用逐分钟工作簿插值代替逐秒求解器连续输出。

7 问题三：固定半径的全域达标时间

7.1 事件判据与三种含水指标

“药材内部各处均低于 0.15”是一个最大值条件。本文同时计算式(14)中的径向全域最大、体积权均值和表面值。在本次已检查的采样时刻，内部节点没有出实现质上高于中心的含水率峰值；算法仍以所有节点的最大值定位事件，而不是事先假定中心必定最慢。保形径向重建不引入超出节点的新峰值。因此数值终止条件覆盖所求的一维径向重建场，但不将有限网格计算表述为真实三维材料的全域达标证明。

$$C_{\max}(t) = \max_{0 \leq r \leq R(t)} C(r, t), \quad \bar{C}(t) = \frac{2}{R(t)^2} \int_0^{R(t)} r C(r, t) dr, \quad C_s(t) = C(R(t), t). \quad (14)$$

先定位 $C_{\max} - 0.15$ 从正到负的事件，再输出严格小于阈值的下一整秒，规则见式(15)。如果事件恰好落在整数秒，仍应取下一秒，因为要求是严格不等式而不是小于等于。程序在获得候选整秒后继续积分并检查未舍入的全部节点，只有最大值确实小于 0.15 才写入最终结果。

$$C_{\max}(t_*) = 0.15 \text{ 且向下穿越}, \quad t_{\text{int}} = \lfloor t_* \rfloor + 1, \quad C_{\max}(t_{\text{int}}) < 0.15. \quad (15)$$

图7清楚区分三种指标。按 60 秒保存轨迹作线性插值，表面含水率约在 12.5521 小时达到 0.15，均值约在 36.0251 小时达到，而全域最大要等到 57.4728 小时。这两个前期时间仅是轨迹插值近似，不是另外进行高精度事件求根得到的结果。若把均值或表面值误用为完成条件，就会提前结束本题所要求的全域干燥过程。

7.2 终点数值及每六小时结果

最终网格下，连续事件时间为 206901.960465 秒，即 57.472766796 小时；严格整秒终点为 206902 秒。此时未舍入最大含水率为 0.149999988484，表面为 0.052659483665，均值约 0.1243082239。表9列出每 6 小时及严格终点的指定位置结果。最后一行的中心显示 0.1500 源于四位小数舍入，不能据此否定严格终止检查。

表9还展示了后期减速：中心从 6 小时的 1.0170 降到 12 小时的 0.4561，之后在 48 小时仍为 0.1618，在 54 小时仍为 0.1539。接近阈值后，虽然绝对剩余差值很小，所需时间仍不可忽视。这与低水分区 D 的快速衰减相符；简单地把早期每小时的平均失水幅度线性延长，会低估后期时间。

图8给出严格终点剖面。中心附近接近阈值，表面则接近但仍高于环境有效平衡值 0.05。两者并不矛盾：表面通过有限的传质系数排出内部仍在迁移的水分，需要保留一个正的传质驱动力。若认为“全部达到 0.15”意味着所有位置均匀等于 0.15，就混淆了最大值约束与均匀终态。

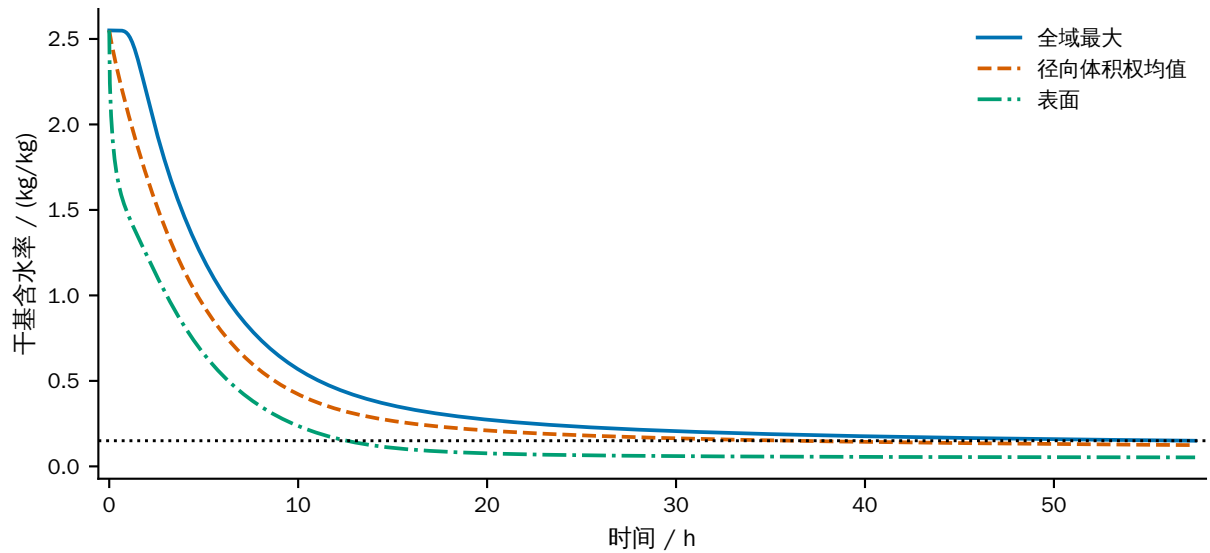


图 7: 问题三的三种含水指标: 60 秒轨迹并补终点, 点线为阈值 0.15; 全域最大越阈晚于表面与均值, 只有前者用于终止判定。

表 9: 问题三每 6 小时及严格终点的干基含水率 (对应题目表 5)

时间/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4561	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2915	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0549
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
57.4728	0.1500	0.1477	0.1402	0.1249	0.0527

数值单位为 kg/kg。未行使用严格整秒终点, 时间转为小时后显示四位小数; 中心显示 0.1500 是舍入结果, 严格不等式按未舍入值检查。

工作簿 3 共有 3449 个输出时刻, 每分钟一次并补最终整秒, 空间为 0 至 2 厘米每 0.1 厘米。题目要求的论文六小时表只是其中较稀疏的摘录, 不承担事件定位。终点依赖未舍入场及求解器事件函数, 而不是从六小时表中人工找最接近 0.15 的一行。该区分使表格的展示分辨率不会限制事件时间的求解分辨率, 但并不消除空间离散和模型假设误差。

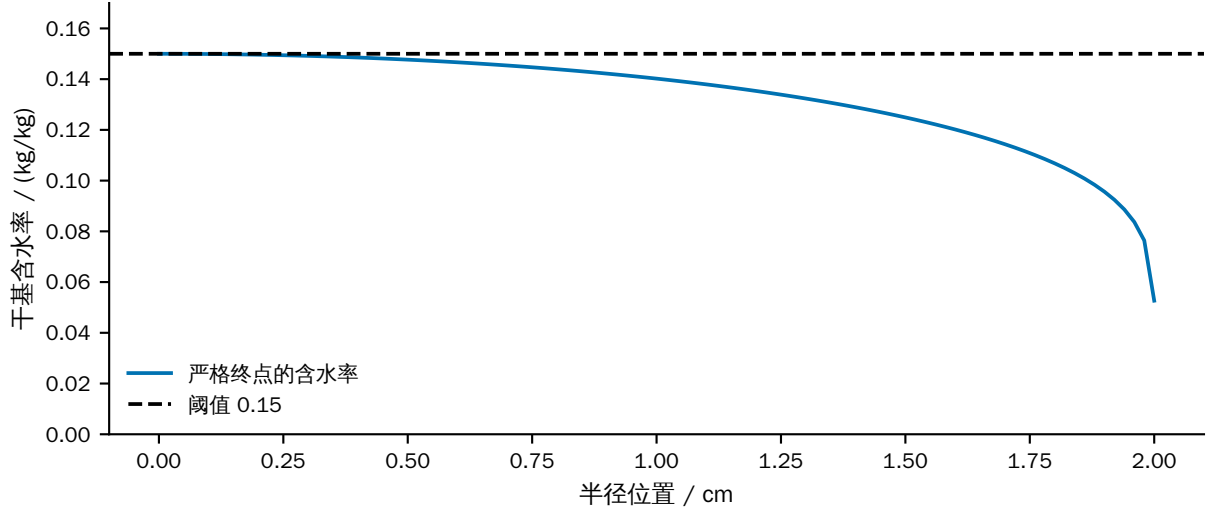


图 8: 问题三 206902 秒的径向含水率：中心接近阈值、表面更低，全域达标并不意味着各处均匀等于 0.15。

8 问题四：实测收缩下的移动边界

8.1 随体坐标与运动项

仅知道外半径 $R(t)$ ，不能唯一确定内部材料速度。本文采用同调径向收缩假设，材料速度为 $v(r, t) = r\dot{R}/R$ ，并令 $x = r/R(t)$ 。对任意随材料输运的标量 u ，定义 $U(x, t) = u(R(t)x, t)$ ，链式关系见式(16)。固定空间时间导数中出现的网格项与材料平流项抵消，因而随体描述中不再有相对对流。

$$\left. \frac{\partial u}{\partial t} \right|_r = \left. \frac{\partial U}{\partial t} \right|_x - \frac{x\dot{R}}{R} U_x, \quad v u_r = \frac{x\dot{R}}{R} U_x, \quad \frac{Du}{Dt} = U_t. \quad (16)$$

移动边界干燥的 ALE 方法可处理网格与材料运动 [4]，但这里采用的是其同调、无相对运动特例，不是对一般非均匀收缩的证明。若仅对固定空间的扩散方程换坐标而不考虑材料运动，会保留一个网格对流项；若已经采用物质导数，却再额外加入该项，则会重复计算运动。式(16)明确了本文选择的物理变量与运动假设。

在该假设下，问题四方程及表面条件变为式(17)。相同的有限体积通量可直接复用，只需在当前时刻更新 R 与问题四物性。半径以 R^{-2} 调节内部扩散尺度，同时以 R^{-1} 影响归一化表面交换强度；这两个作用共同改变动力学，不能只在扩散系数前乘一个经验加速因子。

$$\begin{aligned}
A(C)\theta_t &= \frac{1}{R(t)^2 x} \partial_x (x k(C) \theta_x), \\
C_t &= \frac{1}{R(t)^2 x} \partial_x (x D(\theta, C) C_x), \\
-k_s \theta_x(1, t) &= R(t) h(\theta_s - \theta_\infty), \\
-D_s C_x(1, t) &= R(t) h_m(C_s - C_e).
\end{aligned} \tag{17}$$

8.2 干基变量为何不直接增加压缩项

干基含水率是水质量与干物质质量的比值，而不是单位当前体积中的水质量。均匀压缩会同时改变单位体积中的水量和干固体量，但在没有水分交换时不会凭空改变二者之比。为说明式(17)的条件，设干固体参考密度 $\rho_d(t)$ 在截面上均匀，长度不变且干固体质量守恒，则 $\rho_d R^2$ 为常数。水分相对于干固体的扩散通量若为 $-\rho_d D \nabla C$ ，由水和干固体两个连续性方程相减可得到式(18)。

$$\begin{aligned}
\frac{D\rho_d}{Dt} + \rho_d \nabla \cdot v &= 0, \\
\rho_d \frac{DC}{Dt} &= \nabla \cdot (\rho_d D \nabla C), \\
\rho_d(t) = \rho_{d0} \left(\frac{R_0}{R(t)} \right)^2 &\implies \frac{DC}{Dt} = \nabla \cdot (D \nabla C).
\end{aligned} \tag{18}$$

因此，在初始干固体密度均匀、干质量不流失的同调收缩定义下， C 方程不额外含有 $-C \nabla \cdot v$ 。式中 $\rho_{d0} > 0$ 是初始干固体密度，它在归一化方程中约去，不需要赋予额外数值。若状态是当前体积水浓度，则压缩项通常需要保留；非等温移动边界研究中使用过这种体积变量 [5]，不能把其变量定义省略后直接套入本题。本模型始终采用空间均匀的 $\rho_d(t)$ ，不在求解过程中另改这一条件。

题给经验参数 $\rho_h = 760 + 90C$ 只参与有效热容量，不能同时把它当成上述守恒用的 ρ_d ，也不附加 $\rho_h = (1 + C)\rho_d$ 这样的关系。热容量函数与干质量权重承担不同的模型作用，因此不会因另加一组湿密度约束形成超定问题。本文检验相对初始干质量基准的归一化水分平衡，不报告实际失水克数，也不把体积缩小造成的单位体积变化误算成额外失水。

8.3 含水率场、域外位置与严格终点

图9使用物理半径作纵轴，黑线为实测插值表面，黑线以上不着色，因为该区域已经没有药材。内部色场显示随时间推进的脱水过程，全部规则采样时刻参与绘制，没有为得到平整边界而删除早期半径变化。由于物理坐标点不随材料运动，不能把图中的某一水平线直接解释为同一块材料的历史；同一材料点应沿 $r = xR(t)$ 轨迹读取。

采用与问题三相同的全域最大判据，连续事件时间为 183926.994548 秒，即 51.090831819 小时；严格整秒为 183927 秒。此时最大含水率为 0.149999997200，表面

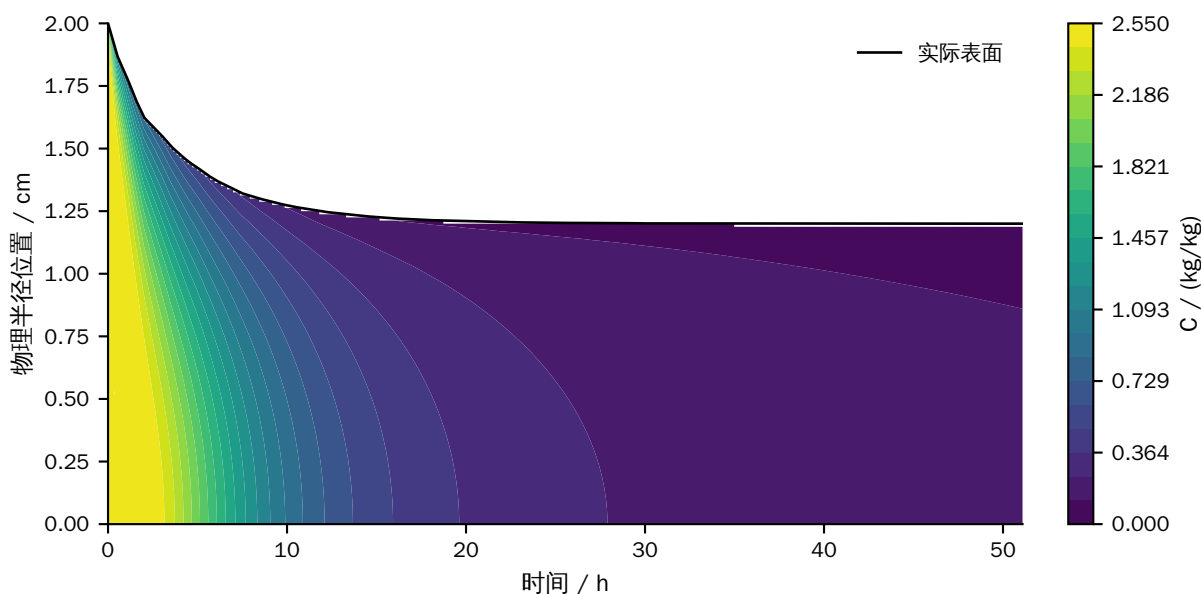


图 9: 问题四的移动域数值场: 以 60 秒轨迹及严格终点绘制, 黑线为当前表面, 域外留白; 固定物理半径不代表固定材料点。

为 0.052620019078, 体积权均值约 0.1184052326, 半径为 1.2000 厘米。终点位于半径记录的 72 小时范围内, 计算没有外推实测半径。后续对照情景延伸到更长时间时使用固定半径, 并未外推该测量曲线。

表10列出每 6 小时及严格终点的指定位置含水率。0、0.5 和 1.0 厘米处始终在这些时刻的药材内; 1.5 厘米在这些表行全部位于药材外, 应显示破折号。表面列是实际 $R(t)$ 处的状态, 不能替换为 1.2 厘米固定列, 因为前期真实半径仍大于 1.2 厘米。工作簿 4 进一步保留全部逐分钟时刻, 共 3066 行, 累计 20099 个域外位置按定义留空。

表 10: 问题四每 6 小时及严格终点的干基含水率 (对应题目表 6)

时间/h	0 cm	0.5 cm	1.0 cm	1.5 cm	移动表面
6	1.7196	1.5377	1.0226	—	0.4207
12	0.7377	0.6546	0.4083	—	0.1670
18	0.4087	0.3686	0.2397	—	0.0892
24	0.2851	0.2611	0.1790	—	0.0673
30	0.2263	0.2094	0.1493	—	0.0595
36	0.1928	0.1797	0.1317	—	0.0560
42	0.1712	0.1604	0.1200	—	0.0541
48	0.1561	0.1468	0.1116	—	0.0530
51.0908	0.1500	0.1413	0.1081	—	0.0526

“—”表示该时刻位置已在药材外, 结果工作簿对应单元格为空; 数值单位为 kg/kg。末行使用严格整秒终点, 时间转为小时后显示四位小数; 中心显示 0.1500 是舍入结果, 严格不等式按未舍入值检查。

问题四在 6 小时的中心含水率为 1.7196，高于问题三的 1.0170；在 42 小时则为 0.1712，略低于问题三的 0.1721。按 60 秒最大含水率轨迹插值，两曲线约在 41.2 小时交叉。因此“问题四最终更快达标”不等于“问题四在每个时刻都更干”。早期物性变化与晚期收缩带来的扩散距离改变相互竞争，只有考察完整过程才能解释这一先后关系。

9 数值可靠性与误差边界

数值可靠性包含“方程是否实现正确”和“本题结果是否达到所选分辨率”两类问题。表11先说明各项检验的职责，再分别给出参数、误差与结果。水量守恒并不意味着空间分布正确；解析例准确也不意味着本题变物性工况具有同样的绝对误差。

表 11: 验证项目、参照对象与可识别的错误

检验	参照对象	主要识别问题
固定域解析模态	Robin 边界 Bessel 解与精确越阈时间	柱坐标算子、中心和表面符号、事件定位
移动域非均匀模态	变半径零通量解析解	几何因子是否真正随时间更新
错误算子对照	只把 $R(t)^{-2}$ 冻结为 R_0^{-2}	测试是否具有拒绝错误的能力
水分收支	离散均值与累计边界损失	面通量、积分权重和干基归一化
雅可比方向差分	实际右端的中心差分	状态及交叉导数是否一致
网格与容差复验	同一工况下更细离散结果	本题场与终点对数值设置的敏感性

9.1 解析极限、封闭场与导数检查

数值验证先检查离散方法能否求对已知问题，再检查本题的网格和时间精度。对常扩散系数 $d_0 = 10^{-5}$ 平方米每秒、半径 0.02 米、 $C_e = 0.05$ 、 $h_m = d_0/R$ 的独立测试，取式(19)的单一 Bessel 模态初值。这里 β 是 $\beta J_1(\beta) = J_0(\beta)$ 的第一正根，故解析解同时满足中心对称与 Robin 表面条件。该测试参数专用于验证，不参与药材名义计算。

$$C(r, t) = 0.05 + 0.20 J_0 \left(\beta \frac{r}{R} \right) \exp \left(-\frac{d_0 \beta^2}{R^2} t \right), \quad t_{\text{exact}} = \frac{R^2 \log 2}{d_0 \beta^2}. \quad (19)$$

采用 160 个区间，精确越阈时间为 17.581493358 秒，数值事件时间为 17.581482896 秒，差约 1.05×10^{-5} 秒。这同时检查了柱坐标扩散算子、中心处理、表面 Robin 符号和事件函数。它是常系数已知解的数值验证，不能转述为真实药材干燥时间也具有同样误差。另令表面传质系数为零、初始含水率均匀，离散右端严格为零，说明封闭均匀场不会因中心或边界处理而自行产生水分变化。

解析雅可比通过方向差分检查，比较 $J(y)p$ 与 $[F(y + \varepsilon p) - F(y - \varepsilon p)]/(2\varepsilon)$ 。基础测试覆盖四问，并固定随机方向种子；独立扩展检查还覆盖常湿、低湿、均匀与加密网

格的 16 组块导数, 最大相对差约 3.78×10^{-10} 。这说明所提供导数与实际右端函数一致, 不意味着右端函数的物理假设已被实验确认。问题三、四的实际长时积分也单独运行, 未出现运行期数值警告。

9.2 非均匀初值的移动域解析基准

固定半径解析例不能单独检验问题四的几何因子是否随时间更新; 无通量均匀场则因梯度恒为零, 也不能识别错误的扩散速度。为检验这一点, 另取 $R(t) = R_0(1 - \gamma t)$, 其中 $R_0 = 0.02$ 米、 $\gamma = 1/7200$ 秒⁻¹、 $0 \leq t \leq 3600$ 秒, 半径实际从 0.02 米缩至 0.01 米。水分扩散系数固定为 $D_0 = 10^{-8}$ 平方米每秒, 中心和表面均取零通量。令 $\lambda = 3.8317059702$ 为 $J_1(\lambda) = 0$ 的第一正根, 采用式(20)给出空间非均匀初值及解析解。

$$C(x, t) = 1 + 0.3J_0(\lambda x) \exp[-\lambda^2 D_0 I(t)],$$

$$I(t) = \int_0^t \frac{ds}{R(s)^2} = \frac{t}{R_0^2(1 - \gamma t)}. \quad (20)$$

由 $J'_0(z) = -J_1(z)$, 该解在 $x = 0, 1$ 处均满足零梯度; 柱坐标 Bessel 特征方程又保证其满足式(17)的常系数水分方程。非零模态随 $I(t)$ 衰减, 因此即使总水量不变, 径向分布仍随移动几何演化。本例与名义药材计算使用相同的生产水分通量算子, 测试只替换其常物性、几何轨迹和无通量边界, 不另外实现一个仅供验证的空间差分求解器。

空间测试保持相对容差 10^{-11} 及最大时间步 14.0625 秒, 在 0 至 3600 秒的 129 个等间隔时刻和每组网格全部节点上比较数值解与解析解。表12中的误差为这些点上的最大绝对差。加密后误差逐级下降, 640 区间的 8.11×10^{-7} 小于预设接受界 10^{-5} 。

表 12: 移动域解析基准的空间加密与错误算子对照

算子	径向区间数	最大解析对比误差/(kg/kg)
随时间更新 $R(t)^{-2}$	40	2.05124×10^{-4}
随时间更新 $R(t)^{-2}$	80	5.17059×10^{-5}
随时间更新 $R(t)^{-2}$	160	1.29592×10^{-5}
随时间更新 $R(t)^{-2}$	320	3.24227×10^{-6}
随时间更新 $R(t)^{-2}$	640	8.10753×10^{-7}
故意冻结为 R_0^{-2}	640	5.86815×10^{-2}

时间测试固定为 160 区间均匀网格, 相对容差依次取 10^{-3} 、 10^{-5} 、 10^{-7} , 最大时间步依次 450、225、112.5 秒, 实际接受步数为 14、25、47。为避免空间截断误差掩盖时间收敛, 参照同一半离散常矩阵 L_N 的指数解 $\exp[D_0 I(t) L_N] C(0)$, 时间误差依次为 1.60038×10^{-3} 、 2.56224×10^{-5} 、 2.65945×10^{-7} , 最后小于 5×10^{-6} 。同时保留与连续 Bessel 解的比较: 最细时间设置的总误差为 6.37797×10^{-6} , 其中仍包含固定空间网格误差, 二者不混称为纯时间误差。

负对照保持相同移动半径、非均匀初值和无通量条件，仅故意将生产水分算子中的 $R(t)^{-2}$ 替换为 R_0^{-2} 。所得误差约 5.87×10^{-2} ，远大于 10^{-5} 接受界，单独执行错误算子检验时程序按预期返回非零退出码。该错误算子仍能保持离散水量，说明守恒测试本身无法排除移动几何更新错误；解析分布对照确实补上了这一识别能力。

正确移动域算子的离散平均含水率漂移不超过 6.66×10^{-16} 。这里漂移相对于数值初值的离散加权均值计算，不与非均匀初值的积分求积误差混淆；连续解析均值为 1，而有限节点求积均值未必恰好等于 1。原有问题一、三、四的非零表面通量质量回归与无通量均匀场测试也继续通过。上述检验验证的是约定移动域方程的实现，不另引入实际药材机制。

9.3 归一化水分平衡

在式(17)的均匀干固体参考权重下，截面平均为 $\bar{C} = 2 \int_0^1 xC dx$ 。将水分方程积分并代入表面条件，得到式(21)。累计损失 Λ 与温湿状态同步积分，初值为零。因为内部面通量两两相消， $\bar{C} + \Lambda$ 应保持初值 2.55；这项测试对固定域和同调收缩域均适用。

$$\frac{d\bar{C}}{dt} = -\frac{2h_m}{R(t)}(C_s - C_e), \quad \dot{\Lambda} = \frac{2h_m}{R(t)}(C_s - C_e), \quad \varepsilon_M = \bar{C} + \Lambda - 2.55. \quad (21)$$

在明确的输出诊断采样上，问题一、三按 1 秒间隔，问题四按 60 秒间隔，均包含 $t = 0$ 和严格终点并检查全部求解节点；最大平衡残差分别约为 4.44×10^{-15} 、 4.88×10^{-15} 和 7.11×10^{-15} 。这些小残差表明离散通量、平均权重和累计损失在同一模型内高度自洽。累计损失本来就是根据相同边界通量构造，因此这是一项离散一致性检查，不将其解释为真实质量通量的独立测量证据。

特别是在收缩域中，不能直接把 $R(t)^2\bar{C}$ 当作干基总水量来判断守恒。参考干固体密度随体积变化，归一化到不变干质量后应使用 $\bar{C}$ 。若要换算实际克数，须另外知道干物质总质量或一致的干固体密度。该归一化残差的数值量级不代表实际失水量的测量精度。

9.4 空间网格独立性

采用 160、320 和 640 个区间逐级加密。比较时固定题目请求的物理位置，问题一遍历前 1800 秒的全部逐秒点，问题二遍历从 1 秒至两次计算共同终端的全过程逐秒点，问题三、四遍历全部逐分钟点并加入共同终端。问题四只比较实际域内位置，同时单独比较实际表面值，避免域外空值被错误替换为零或参与误差统计。误差定义见式(22)。

$$E_\theta = \max_S |\theta_N - \theta_{2N}|, \quad E_C = \max_S |C_N - C_{2N}|, \quad E_t = \frac{|t_{*,N} - t_{*,2N}|}{t_{*,2N}}, \quad (22)$$

其中 S 是上述共同输出点集合。预设接受阈值为 $E_\theta < 0.005$ 摄氏度、 $E_C < 5 \times 10^{-5}$ 、有事件的情景还需 $E_t < 10^{-3}$ 。这些阈值是数值分辨率标准，不是依据实验精度拟合得

到的容差。表13列出两次加密的含水率差和最终加密的其余指标，图10展示空间场差异的下降。

表 13: 三网格比较；末三列为 320 与 640 区间之差

问题	$E_C(160, 320)$	$E_C(320, 640)$	$E_\theta/\text{摄氏度}$	E_t
一	5.864×10^{-5}	1.468×10^{-5}	8.309×10^{-6}	—
二	5.130×10^{-5}	1.284×10^{-5}	1.444×10^{-5}	—
三	4.423×10^{-5}	1.106×10^{-5}	1.444×10^{-5}	2.434×10^{-5}
四	1.896×10^{-4}	4.742×10^{-5}	1.687×10^{-5}	1.511×10^{-5}

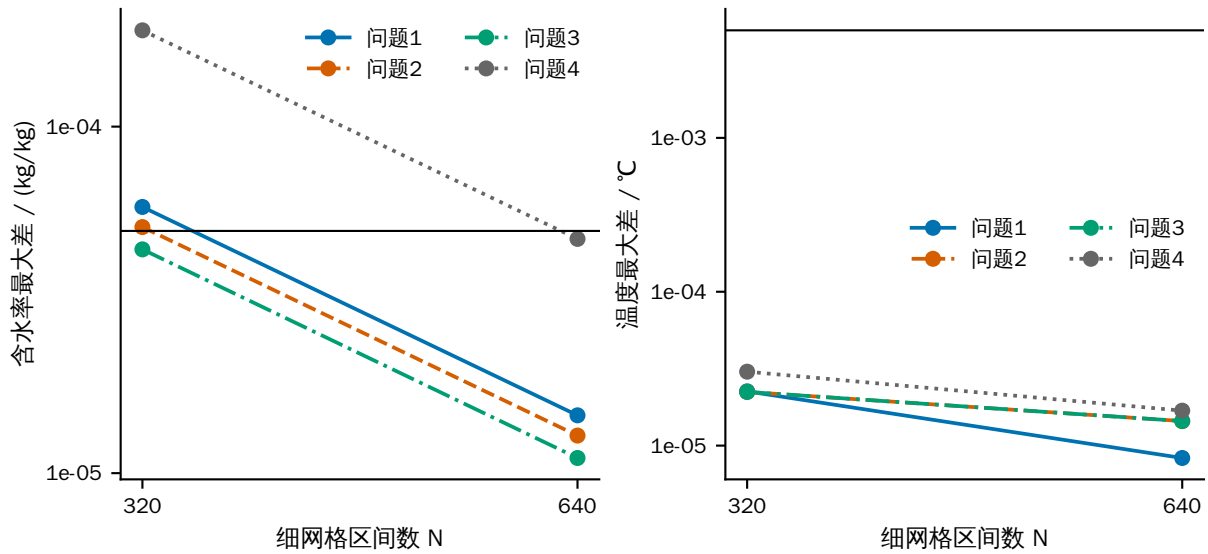


图 10: 本题相邻网格的最大场差: N 为细网格区间数, 横线为 $E_C < 5 \times 10^{-5}$ 和 $E_\theta < 0.005$ 的接受界; 事件和时间容差另见表13、14。

160 到 320 区间时, 问题一、二和四的含水率差超过接受阈值, 因此不能在 320 处停止。320 到 640 时全部指标通过, 最终统一选用 640 区间。含水率误差大体随加密减少至四分之一, 与空间离散接近二阶的表现相容; 这里只观察了有限网格序列, 不声称已证明任意退化扩散情景的全局二阶收敛定理。温度差较小, 但不能因此替水分差作决定, 因为两变量具有不同尺度和阈值。

问题三的 320 与 640 网格事件时间仍相差约 5.04 秒, 问题四相差约 2.78 秒。相对误差虽然均远低于 10^{-3} , 绝对差并非小于一秒。因此 206902 秒和 183927 秒应理解为当前已通过数值精度标准的模型所给严格整秒, 不应作为真实过程亚秒级保证。若任务进一步要求终点绝对误差低于一秒, 应另设绝对事件误差标准并继续加密, 不能靠多打印小数位实现。

9.5 时间容差与输出一致性

在最终 640 区间网格上，将相对容差从 10^{-8} 收紧到 10^{-9} ，相应绝对容差同步收紧。表14列出时间精度复验。四问温度最大差均不超过 1.843×10^{-5} 摄氏度，含水率最大差不超过 1.589×10^{-7} ；问题三、四事件相对差分别为 1.749×10^{-9} 和 3.510×10^{-9} 。在本次设置下，时间容差误差明显小于已观察到的空间场差。

表 14: 最终网格上两组时间容差的差异

问题	最大温度差/摄氏度	最大含水率差/(kg/kg)	事件相对差
一	8.635×10^{-6}	1.045×10^{-7}	—
二	1.476×10^{-5}	1.295×10^{-7}	—
三	1.476×10^{-5}	1.230×10^{-7}	1.749×10^{-9}
四	1.842×10^{-5}	1.589×10^{-7}	3.510×10^{-9}

图10仅展示空间场差，不包含事件和时间容差全部信息；相应结论须结合表13、表14理解。空间收敛、事件定位与时间精度是不同检验目标，需分别报告。这里检查的是已定义模型的离散误差；环境延拓改变的后果则由后文已执行的情景对照说明，两者不相互替代。

全部结果文件从同一最终解生成，按对应时间和位置核对工作簿的四位小数显示与保留较高精度的 CSV 及场数组。独立完整复现得到相同的数值数组及六张指定表；问题一两表各 1800 行、问题二两表各 206902 行、问题三 3449 行和问题四 3066 行均按实际数据内容核对，工作簿数字格式保持一致。问题二还核对了两张全量工作表的 XML 及样式，而不是仅检查前三小时。这里的一致性说明输出与代码具有可重复性，不代表存在第二份独立实验；重复运行的耗时和压缩封装信息不同，也不直接视为数值错误。

诊断统计区分采样极值与连续时空极值。默认摘要在包含首末点的 250 个等间隔时刻、全部 641 个求解节点上统计；正式输出诊断另按上述逐秒或逐分钟网格分块检查，字段均明确标为采样统计并记录点数及步长。例如问题三默认抽样最高温度为 50.0346733 摄氏度，而 0 至 206902 秒逐秒、全部节点的最高采样温度为 50.0398567 摄氏度。二者差异由采样集合不同造成，温度场和事件时间并未改变。工作簿从 1 秒开始且只有 21 个径向位置，诊断还包含初始时刻和全部求解节点，不能把这几种范围混写为未经限定的连续全时空最大值。

计算还检查温度、水分是否有限、含水率是否为正、初始上界是否被非物理超越，以及问题四采样位置与半径是否一致。初值附近出现的机器舍入量级偏差按浮点容许范围处理，不通过截断负值或裁剪曲线“修复”求解。若出现实质性的非正含水率、半径数据范围外请求或严格终点失败，程序会停止并报告，而不会继续输出外观完整的表格。

10 几何对照与工况灵敏度

10.1 分离物性改变与收缩影响

问题三和问题四同时改变物性与几何，直接比较只能得到总差异。为隔离几何作用，再运行一个对照：完全保持问题四物性、初值、传递系数和环境延拓，但令半径始终为 2 厘米。该对照的事件时间为 129.844890 小时，明显长于实测收缩情景的 51.090832 小时。它是模型反事实，不是新增实验，也不用于重新标定半径数据。

记 $\tau(\mathcal{P}, R)$ 为物性组 $\mathcal{P}$ 与几何轨迹 R 下的越阈时间， $\mathcal{P}_3, \mathcal{P}_4$ 分别表示问题三、四的整组物性。保持共同初值、边界、环境延拓和终点判据，现有计算构成 $\tau(\mathcal{P}_3, R_0) \rightarrow \tau(\mathcal{P}_4, R_0) \rightarrow \tau(\mathcal{P}_4, R(t))$ 这一条件比较路径。沿此路径，先改变物性使时间增加 72.3721 小时，再引入收缩使时间减少 78.7541 小时；净变化见式(23)。

$$\begin{aligned} \Delta\tau_{\text{total}} &= \underbrace{\tau(\mathcal{P}_4, R_0) - \tau(\mathcal{P}_3, R_0)}_{\text{固定几何下的整组物性差异}} + \underbrace{\tau(\mathcal{P}_4, R(t)) - \tau(\mathcal{P}_4, R_0)}_{\text{问题四物性下的几何差异}} \\ &= 72.3721 - 78.7541 \approx -6.3819 \text{ h.} \end{aligned} \quad (23)$$

上述数值由未舍入事件时间作差后展示，末行近似号反映四位小数舍入。它说明沿该比较路径，几何缩短效应超过物性改变带来的延长，因而问题四最终更快；并不表示各物性参数可以逐个分配独立份额。本文没有计算“先收缩、再换物性”的另一条路径，故不把这一代数分解称为唯一路径无关的因果分解。

图11同时给出三种情景，式(24)明确两个降幅的不同分母。第一项是问题四相对问题三的总体差异；第二项在相同问题四物性下比较移动与固定几何。式(23)的绝对时间差可以相加，但下面两个百分比基准不同，不能相加，也不表示能耗节约比例。

$$\begin{aligned} \eta_{\text{total}} &= 1 - \frac{51.090831819}{57.472766796} = 11.1043\%, \\ \eta_{\text{shrink}} &= 1 - \frac{51.090831819}{129.844890064} = 60.6524\%. \end{aligned} \quad (24)$$

问题四终点半径约为初始的 0.6 倍，长度平方尺度相应约为 0.36 倍；侧面积与体积之比则约增加至初始的 1.667 倍。这些几何变化解释了收缩为什么可能显著改变扩散和表面交换，但不能把固定半径时间简单乘 0.36 作为最终答案。实际半径在早期持续变化，温度和水分物性也沿轨迹变化，且有限表面阻力使整体时间并不严格按单一长度平方缩放。

同物性固定半径对照超过 72 小时并不违反实测半径范围约束，因为该情景不调用收缩曲线；其恒定几何是对照定义的一部分。对照仍使用四小时后的名义恒定环境，因而对长期环境假设的依赖更长。本文只将它用于结构归因，不将 129.8449 小时与未经测量的真实无收缩药材直接对应。

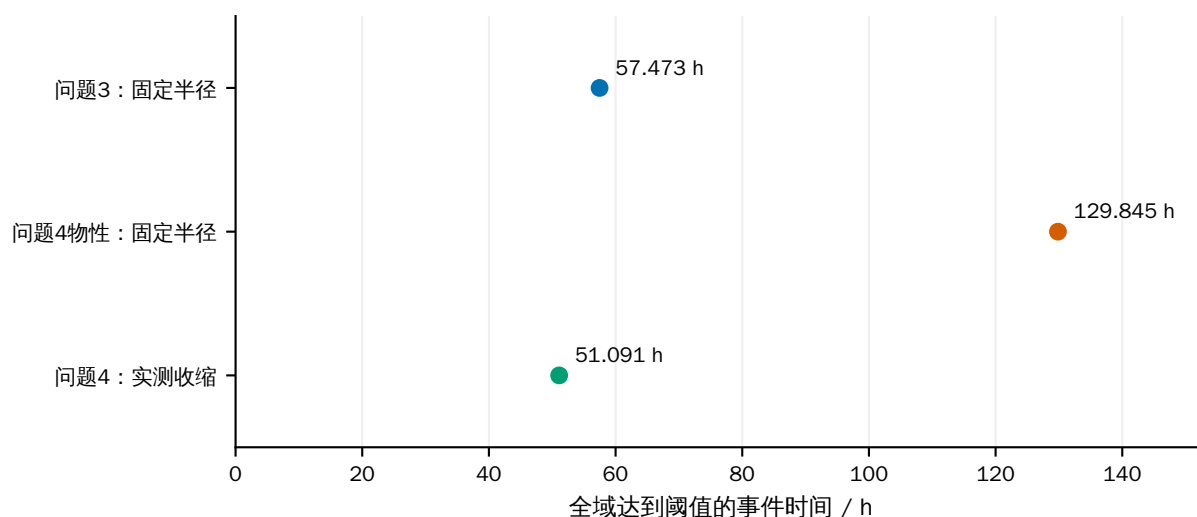


图 11: 三情景条件对照: 上、中两组几何相同而物性不同; 中、下两组物性相同而几何不同。各点为确定性模型事件时间, 不是重复实验均值。

10.2 环境延拓和传质系数情景

围绕问题三、四各自名义情景, 分别考察末值延拓、末小时均值延拓、尾温加减 0.5 摄氏度、尾平衡含水率加减 0.001, 以及传质系数加减 10%。前两类延拓和温湿扰动仅在四小时以后作用, 传质系数则在全过程改变。每次只改变一个情景设置, 保持其他模型条件不变, 从而解释结果差异的来源。表15列出实际事件时间。

表 15: 各单因素情景下的全域越阈时间, 单位: 小时

情景	问题三	问题四
名义环境及传质系数	57.4728	51.0908
四小时后采用末值	57.1698	50.8245
四小时后采用末小时均值	57.4745	51.0923
尾段温度降低 0.5 摄氏度	58.3959	51.9004
尾段温度升高 0.5 摄氏度	56.5700	50.2989
尾段平衡含水率降低 0.001	57.4532	51.0691
尾段平衡含水率升高 0.001	57.4931	51.1137
全过程传质系数降低 10%	58.2086	51.3820
全过程传质系数提高 10%	56.8954	50.8700

图12将变化量归一化到各自名义时间。尾温降低 0.5 摄氏度时, 问题三时间增加 1.6062%, 问题四增加 1.5846%; 升温时分别减少 1.5707% 和 1.5500%。这种轻微不对称与非线性扩散函数相容。末小时均值延拓与名义情景的差异均约 0.003%, 末值延拓则约缩短 0.52%, 提示将一条末端噪声记录永久延拓可能放大其工况意义。

传质系数降低 10% 使问题三时间增加 1.2804%, 问题四增加 0.5699%; 提高 10%

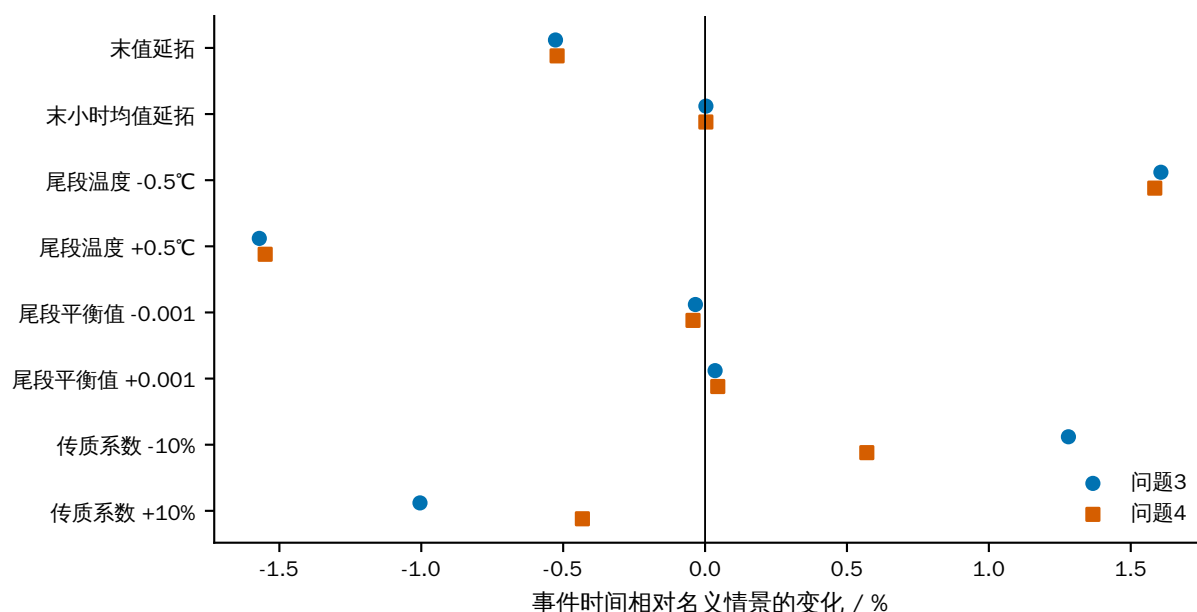


图 12: 各单因素情景相对本问名义时间的变化；圆点、方块区分两问，各扰动幅度不等价，点位不是置信区间或全局重要性排名。

分别使其减少 1.0046% 和 0.4323%。这表明在所考察工况附近，外部传质阻力的变化仍会影响终点，但其幅度小于同物性几何对照造成的差异。不能由此断言传质系数在所有干燥阶段都不重要，因为早期表面响应、初始尺度和不同材料函数可能改变阻力分配。

这些扰动幅度并不等价：0.5 摄氏度、0.001 含水率和 10% 传质系数分别具有不同单位、相对尺度和作用时间。图中百分比适合说明所列情景的后果，不构成统一的全局参数重要性排名，也不是由测量误差概率分布推出的置信区间。所有情景保持已经定义的一维、有效参数、忽略潜热及同调收缩条件，不在单因素比较中另行改变模型族。名义时间应连同这些约定一起报告。

在所有成对的所列工况下，问题四终点仍早于问题三，说明该排序对这些局部变化没有翻转。本文只在已执行的情景范围内作此结论，不扩展到任意湿度、任意气流或任意药材尺寸。若工艺允许调节温度，还需同时约束药效成分损失和能耗；当前题目没有提供这些数据，所以本文不把缩短时间直接等同于综合工艺最优。

11 模型评价与适用范围

11.1 方法价值及可推广的处理思路

本文没有为四问分别构造互不相干的经验曲线，而是保留相同的状态定义、径向通量与终点判据。固定域到移动域的变化落实在物性组、几何因子和输出位置上，问题二与问题三则直接共享同一初边值解。这使短时剖面、长期阈值及收缩场能够放在同一计算逻辑下解释。

第一，离散方程保留柱坐标体积权重及变系数散度形式，中心控制体避免点值奇异，表面节点保留有限热质交换阻力。相邻控制体共用面通量，内部通量在全域求和中相消；这同时支持场计算与归一化水分收支，而不依赖事后调整输出数据。

第二，移动域从变量的质量基准出发推导。干基含水率不同于单位当前体积的水浓度；在表4的同调收缩定义下，材料坐标固定、干固体密度随 R^{-2} 变化，压缩不会凭空改变水与干物质的质量比。因此几何进入扩散尺度与表面交换尺度，却不另加干基水分压缩源项。非均匀解析模态及错误算子对照检验了这一几何变化是否确实进入程序。

第三，问题三、四采用全域最大含水率而非均值或表面值定位终点，并将整秒输出与连续事件分开。收缩后实际表面和固定物理半径也分开处理，域外单元格留空。由此，输出文件中的时间、位置和阈值分别具有明确含义，表格的展示分辨率不会反过来定义计算的终止时刻。

第四，同物性固定半径对照使“总体时间更短”和“收缩作用有多大”成为两个可分别回答的问题。沿已计算路径，物性差异与几何差异方向相反；这一结果也解释了问题四早期中心更湿、最终却更早达标的现象。方法价值在于比较条件明确，而不是将多因素结果归结为一个不加限定的加速比例。

11.2 精度、工况与结论边界

模型的解释范围由表4一次确定：轴向与周向均匀，有效边界 $C_e = W_a$ ，忽略蒸发潜热， ρ_h 与 ρ_d 分工，问题四同调径向收缩且长度不变。本文不在结果讨论中重新加入潜热、端面或其他机制，也不把这些约定宣称为特定药材的完整真实描述。

数值验证应分层理解。解析基准检查已知方程的实现，水分收支检查离散一致性，网格与容差复验衡量本题输出对数值设置的敏感性。它们共同支持当前离散结果，但不能互相替代。尤其是错误的冻结几何算子仍可能守恒，只有非均匀解析分布对照才能在本测试中识别其错误扩散速度。

报告精度也须与证据相称。最终网格采用 640 个区间，320 至 640 区间的事件时间差仍约为 5.04 秒和 2.78 秒；因此 206902 秒与 183927 秒是该离散模型按严格不等式生成的整秒结果，不是亚秒准确性的保证。四位小数的含水率显示可能为 0.1500，终点检查仍使用未舍入值。诊断极值一律限定到明确采样集合，不由逐秒输出推断连续时空极值已精确取得。

四小时后的恒定环境属于长期工况约定，而不是新增测量。所列末值、末小时均值及单因素扰动情景中，问题四早于问题三的排序均未翻转；这说明排序在已计算局部情景下保持，不是对任意环境的全局鲁棒性证明。不同扰动的单位和幅度不同，不能用其百分比变化直接构造统一参数重要性排名，更不能视为统计置信区间。

最后，题给 0.15 仅定义含水率目标，没有给出能耗、药效或综合品质目标。本文完成的是明确理想模型下的温湿场和达标时间计算，而不是多目标工艺优化。内部观测缺失意味着不能报告实验预测准确率，但不影响本题在既定输入和约定条件下完成求解。

12 结论

在题给物性与明确的有效参数约定下，采用统一守恒有限体积及隐式耦合积分，得到四问的温湿场和终点结果。

问题一 1800 秒时，中心、表面温度分别为 33.5753、36.7856 摄氏度，含水率为 2.5500、1.5102；热量已传入内部，而水分下降主要集中于外层。问题二三小时的径向温差仅约 0.1169 摄氏度，但中心、表面含水率仍为 1.7662、1.0081。温度接近均匀与含水率全域达标是不同阶段，不能相互替代。

问题三越阈时间为 57.4728 小时，严格整秒终点为 206902 秒；问题四为 51.0908 小时和 183927 秒，终点半径为 1.2000 厘米。两问均按全部节点的未舍入最大含水率判断，问题四域外固定位置不赋值。问题二两工作表覆盖 1 至 206902 秒的全过程逐秒结果，问题三、四按 60 秒间隔输出并补入最终整秒，论文六张题表保留指定时刻。

问题四相对问题三缩短 6.3819 小时、约 11.1043%，这是物性与几何共同改变的总体差异。保持问题四物性而固定半径的对照为 129.8449 小时，同物性收缩降幅为 60.6524%。沿先改变物性、再引入收缩的路径，时间分别增加 72.3721 小时、减少 78.7541 小时；该条件对照解释了最终排序，不等同于唯一因果份额或全过程处处更干。

最终 640 区间网格满足所选空间与时间精度标准。移动域非均匀解析基准的最大采样误差为 8.11×10^{-7} ，冻结几何因子的错误算子误差为 5.87×10^{-2} ，被检验拒绝，水分平衡检查同时通过。因网格细化仍带来数秒事件差，整秒输出不作亚秒误差保证。上述结论共同回答了约定一维径向模型中的场变化、严格达标及收缩条件对照问题。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于题意分析、文献线索整理、程序设计与核验、图表制作和文稿编辑，详细使用情况见支撑材料。

使用工具为 OpenAI Codex，工具版本记录、关键提示、采纳范围与校验说明见支撑材料《AI 工具使用详情》。本次文稿完善用于摘要与论证组织、假设表述、已有结果的条件对照解释和图注审查，保持已声明模型与实际计算结果不变。计算数值来自所附程序，不以生成文本填造内部温湿数据。参赛队仍须按真实使用记录核对披露、独立理解和审阅方程与结果，并对提交内容负责；本声明不表示最终人工审阅已经完成。

参考文献

- [1] da Silva W. P., e Silva C. M. D. P. S., Gomes J. P. Comparison of boundary conditions to describe drying of turmeric (*Curcuma longa*) rhizomes using diffusion models[J]. Journal of Food Science and Technology, 2014, 51(11): 3181–3189. DOI: 10.1007/s13197-012-0813-x.

- [2] da Silva W. P., e Silva C. M. D. P. S., Gama F. J. A. Estimation of thermo-physical properties of products with cylindrical shape during drying: The coupling between mass and heat[J]. *Journal of Food Engineering*, 2014, 141: 65–73. DOI: 10.1016/j.jfoodeng.2014.05.010.
- [3] Tzempelikos D. A., Mitrakos D., Vouros A. P., Bardakas A. V., Filios A. E., Margaritis D. P. Numerical modeling of heat and mass transfer during convective drying of cylindrical quince slices[J]. *Journal of Food Engineering*, 2015, 156: 10–21. DOI: 10.1016/j.jfoodeng.2015.01.017.
- [4] Seyedabadi E., Khojastehpour M., Abbaspour-Fard M. H. Convective drying simulation of banana slabs considering non-isotropic shrinkage using FEM with the Arbitrary Lagrangian–Eulerian method[J]. *International Journal of Food Properties*, 2017, 20(sup1): S36–S49. DOI: 10.1080/10942912.2017.1288134.
- [5] Adrover A., Venditti C., Brasiello A. A Non-Isothermal Moving-Boundary Model for Continuous and Intermittent Drying of Pears[J]. *Foods*, 2020, 9(11): 1577. DOI: 10.3390/foods9111577.

A 支撑材料目录与复现说明

表16列出支撑材料。以下代码为计算、验证和导出本文结果的完整源文件，包括七个脚本和五个辅助模块；它们与电子支撑材料中的对应文件一致。输入附件和结果模板需保持相对目录不变，不能仅复制主求解器而遗漏读取和导出模块。数值运行不依赖论文编译环境，也不需要保留开发时的缓存、临时目录或代理配置。

表 16: 电子支撑材料目录

文件或目录	内容
原题及附件目录	题目 PDF、环境、半径及四个原始结果模板
<code>solve.py</code>	物性、网格、通量、雅可比、积分、事件与小例检验
<code>run_all.py</code>	网格和时间复验、结果表、场数组及情景分析
<code>make_figures.py</code>	从真实结果生成 14 组矢量和位图
<code>verify_moving_domain.py</code>	移动域解析模态、时空加密、错误算子与守恒测试
<code>export_result2_full.py</code>	全过程逐秒工作簿的分块导出和完整结构检查
<code>reproduce.py</code>	唯一完整入口，创建隔离目录并记录真实运行
<code>test_sync.py</code>	采样统计与 600 秒导出的小规模回归
<code>utils/</code>	全量读表、绘图风格、图表自检、导出与环境记录
<code>requirements.txt</code>	本次已验证的 Python 包版本
<code>results/</code>	四个 XLSX、六张题表、过程数组、诊断与参数摘要
<code>figures/</code>	原始输入、运行过程和结果图的 PDF、SVG、PNG
完整论文 LaTeX 目录	主文稿、章节、表格、绑定图及完整代码副本
AI 工具使用详情 PDF	关键提示、采纳范围、工具说明和人工复核提示

本次使用 Python 3.10.12。先按依赖清单安装所需包，再准备文泉驿正黑 TrueType 字体：Linux 可安装发行版的 `fonts-wqy-zenhei`，也可把原版 `wqy-zenhei.ttc` 置于项目的 `assets/` 目录。字体不是计算过程中自动联网下载的隐含资源。Python 依赖、原始输入和字体准备完成后，从项目根目录执行：

```
python3 -m pip install -r requirements.txt
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 python3 -B reproduce.py
```

该命令在项目内新建 `reproduced_*` 目录，从原始附件副本重新执行移动域解析检验、四问完整计算、网格及时间精度复验、全过程工作簿、六张题表、场数组、情景对照和 14 组图，不覆盖现有结果。某一步失败即返回非零退出码。快速回归和独立基准可分别执行：

```
python3 -B reproduce.py --smoke
python3 -B verify_moving_domain.py
python3 -B verify_moving_domain.py --mutant-only
```

最后一条命令故意使用错误几何算子，预期以非零退出码拒绝；不能将该预期失败与正式完整入口失败混同。快速回归仅覆盖 600 秒，不替代完整复现。问题二正式范围始终为全干燥过程逐秒输出，开发用前三小时选项不是本文交付范围。工作簿体积较大，因此采用分块采样和流式压缩写入，避免同时在内存中保存全部单元格对象。

论文在 LaTeX 工程内以 XeLaTeX 编译两遍，无需 shell escape 或外部 BibTeX。需准备 ctex、fvextra、常用数学和表格宏包，以及 Noto Serif/Sans CJK SC 字体。论文中图使用已嵌入 TrueType 字体的矢量 PDF，代码副本仅用于阅读和来源核对；实际数值命令应从包含原始附件的项目根目录运行。公开支撑压缩包保留全部数值、图和源码，但不重复打包字体二进制、主论文 PDF 及本机临时文件；字体应按上述方法准备，主论文单独交付。

B 完整计算与输出代码

B.1 核心求解与诊断

```
"""Nonlinear radial heat/moisture diffusion, SI units throughout.

Run from the project directory: python3 solve.py --smoke
All assumptions and boundary extrapolations are defined in the model report.
"""

from __future__ import annotations
from dataclasses import dataclass
from pathlib import Path
import argparse
import hashlib
import importlib.util
import json
import math
import sys
import time

ROOT = Path(__file__).resolve().parent
if (ROOT / '.work/deps').exists():
    sys.path.insert(0, str(ROOT / '.work/deps'))
import numpy as np
from scipy.integrate import solve_ivp
from scipy.interpolate import PchipInterpolator
from scipy.optimize import brentq
from scipy.sparse import bmat, csc_matrix, coo_matrix, diags
from scipy.special import j0, j1

SEED = 20260910

def read_inputs():
    from utils.read_rows import read_excel_rows
    output, snapshots = [], []
    for name, rows, first, last in (
        ('附件 1.xlsx', 241, [0, 28, .01963], [14400, 50.165, .04986]),
        ('附件 2.xlsx', 145, [0, 2], [259200, 1.198])):
        path = ROOT / '附件' / name
```

```

        data = np.array([list(r.values()) for r in read_excel_rows(path, header=True,
        expected_rows=rows)], float)
        assert np.allclose(data[0], first) and np.allclose(data[-1], last)
        assert np.all(np.isfinite(data)) and np.all(np.diff(data[:,0]) > 0)
        output.append(data)
        snapshots.append(dict(path=str(path.relative_to(ROOT)),
        sha256=hashlib.sha256(path.read_bytes()).hexdigest(),
        rows=rows, first=data[0].tolist(), last=data[-1].tolist()))
    assert np.all(np.diff(output[1][:,1]) <= 0)
    return output, snapshots

def properties(q, temperature, moisture):
    if np.any(moisture <= 0) or np.any(temperature <= -273.15):
        raise ValueError('Nonpositive moisture or absolute temperature')
    if q == 1:
        return np.full_like(moisture, 820*2600), np.full_like(moisture,.36), 7e-9*np.exp(-.89/moisture)
    f = moisture/(1+moisture)
    if q in (2,3):
        capacity = (650+128*moisture)*(1450+2736*f)
        return capacity, .21+.38*f, .0024*np.exp(-.45/moisture-3850/(temperature+273.15))
    if q == 4:
        capacity = (760+90*moisture)*(1850+2150*f)
        return capacity, .12+.20*f, .00042*np.exp(-.30/moisture-3850/(temperature+273.15))
    raise ValueError(q)

class Grid:
    def __init__(self, n, graded=True):
        self.n = n
        s = np.linspace(0.,1.,n+1)
        self.x = 1-(1-s)**2 if graded else s
        self.dx = np.diff(self.x)
        self.faces = np.r_[0., (self.x[:-1]+self.x[1:])/2, 1.]
        self.w = np.diff(self.faces**2)/2
        block = diags([np.ones(n),np.ones(n+1),np.ones(n)],[-1,0,1],format='csc')
        self.scalar_pattern = block
        base = bmat([[block,block],[block,block]],format='csc')
        last = csc_matrix([1.],[0],[2*n+1]), shape=(1,2*(n+1))
        self.pattern = bmat([[base,csc_matrix((2*(n+1),1))],[last,csc_matrix((1,1))]],format='csc')

    def rate(self, u, a, radius, transfer, ambient, capacity=1.):
        a = np.broadcast_to(a, u.shape)
        af = 2*a[:-1]*a[1:]/(a[:-1]+a[1:])
        flux = np.r_[0.,self.faces[1:-1]*af*np.diff(u)/self.dx,radius*transfer*(ambient-u[-1])]
        return np.diff(flux)/(radius**2*self.w*capacity)

    def jac_block(self, u, a, da, radius, transfer, capacity=1., same=False):
        """Derivative of conservative fluxes with respect to one state field."""
        m = self.n+1
        a = np.broadcast_to(a, u.shape)
        da = np.broadcast_to(da, u.shape)
        den = radius**2*self.w*capacity
        left, right = a[:-1], a[1:]
        harmonic = 2*left*right/(left+right)
        fac = self.faces[1:-1]/self.dx
        delta = np.diff(u)
        dl = fac*(2*right**2/(left+right)**2*da[:-1]*delta

```

```

        - (harmonic if same else 0.))
dr = fac*(2*left**2/(left+right)**2*da[1:]*delta
        + (harmonic if same else 0.))
i = np.arange(m-1)
row = np.r_[i,i,i+1,i+1]
col = np.r_[i,i+1,i,i+1]
val = np.r_[dl/den[:-1],dr/den[:-1],-dl/den[1:],-dr/den[1:]]
if same:
    row = np.r_[row,m-1]; col = np.r_[col,m-1]
    val = np.r_[val,-radius*transfer/den[-1]]
return coo_matrix((val,(row,col)),shape=(m,m)).tocsc()

@dataclass
class Scenario:
    tail: str = 'nominal'
    temperature_shift: float = 0.
    moisture_shift: float = 0.
    mass_factor: float = 1.
    fixed_radius: bool = False

class Model:
    def __init__(self, q, n, arrays, scenario=None, graded=True):
        self.q, self.grid = q, Grid(n,graded)
        self.boundary, self.shrink = arrays
        self.scenario = scenario or Scenario()

    def radius(self,t):
        if self.q != 4 or self.scenario.fixed_radius:
            return np.asarray(t)*0+.02
        if np.max(t) > self.shrink[-1,0]+1e-7:
            raise ValueError('Radius extrapolation beyond measured 72 h is not authorised')
        return np.interp(t,self.shrink[:,0],self.shrink[:,1])/100

    def ambient(self,t):
        b, s = self.boundary,self.scenario
        if t <= b[-1,0]:
            return np.interp(t,b[:,0],b[:,1]),np.interp(t,b[:,0],b[:,2])
        tail = b[-1,1:] if s.tail=='last' else b[b[:,0]>=10800,1:].mean(axis=0) if s.tail=='mean' else
        (50.,.05)
        return tail[0]+s.temperature_shift,tail[1]+s.moisture_shift

    def rhs(self,t,y):
        m = self.grid.n+1
        temp, moisture = y[:m],y[m:2*m]
        cap,k,d = properties(self.q,temp,moisture)
        radius = float(self.radius(t))
        ti,ce = self.ambient(t)
        hm = 8e-7*self.scenario.mass_factor
        tr = self.grid.rate(temp,k,radius,25.,ti,cap)
        cr = self.grid.rate(moisture,d,radius,hm,ce)
        loss_rate = 2*hm*(moisture[-1]-ce)/radius
        return np.r_[tr,cr,loss_rate]

    def jac(self,t,y):
        m = self.grid.n+1
        temp, c = y[:m], y[m:2*m]

```

```

cap,k,d = properties(self.q,temp,c)
if self.q == 1:
    dc = np.zeros(m); kc = np.zeros(m); dt = np.zeros(m)
    dmoist = d*.89/c**2
elif self.q in (2,3):
    rho=650+128*c; cp=1450+2736*c/(1+c)
    dc=128*cp+rho*2736/(1+c)**2
    kc=.38/(1+c)**2; dt=d*3850/(temp+273.15)**2
    dmoist=d*.45/c**2
else:
    rho=760+90*c; cp=1850+2150*c/(1+c)
    dc=90*cp+rho*2150/(1+c)**2
    kc=.20/(1+c)**2; dt=d*3850/(temp+273.15)**2
    dmoist=d*.30/c**2
radius=float(self.radius(t)); ti,ce=self.ambient(t)
hm=8e-7*self.scenario.mass_factor; g=self.grid
tt=g.jac_block(temp,k,0.,radius,25.,cap,same=True)
tc=g.jac_block(temp,k,kc,radius,25.,cap)
tr=g.rate(temp,k,radius,25.,ti,cap)
tc=tc-diags(tr*dc/cap,format='csc')
cc=g.jac_block(c,d,dmoist,radius,hm,same=True)
ct=g.jac_block(c,d,dt,radius,hm)
loss=csc_matrix(( [2*hm/radius], ([0], [m-1]) ), shape=(1,m))
return bmat([ [tt,tc,csc_matrix((m,1))],
               [ct,cc,csc_matrix((m,1))],
               [csc_matrix((1,m)),loss,csc_matrix((1,1))]], format='csc')

def event(self,t,y):
    m = self.grid.n+1
    return np.max(y[m:2*m])-.15

class Run:
    def __init__(self,model,segments,crossing,strict,runtime):
        self.model,self.segments = model,segments
        self.crossing,self.strict,self.runtime = crossing,strict,runtime
        self.end = float(segments[-1].t[-1])

    def values(self,times):
        times = np.atleast_1d(times).astype(float)
        if times.min() < 0 or times.max() > self.end+1e-6:
            raise ValueError('Requested sampling outside integrated interval')
        y = np.empty((2*(self.model.grid.n+1)+1,len(times)))
        assigned = np.zeros(len(times),bool)
        for segment in self.segments:
            take=(times>=segment.t[0]-1e-8)&(times<=segment.t[-1]+1e-8)
            if not np.any(take):
                continue
            y[:,take]=segment.sol(times[take])
            assigned[take]=True
        assert assigned.all()
        return y

    def sample(self,times,radii_cm=None,normalized=False):
        times = np.atleast_1d(times).astype(float)
        y=self.values(times); m=self.model.grid.n+1
        if radii_cm is None:

```

```

        radii_cm=np.linspace(0,2,21)
coords=np.asarray(radii_cm,dtype=float)
temperatures,moistures=[],[]
for j,t in enumerate(times):
    x=coords if normalized else coords/100/float(self.model.radius(t))
    temperatures.append(PchipInterpolator(self.model.grid.x,y[:m,j],extrapolate=False)(x))
    moistures.append(PchipInterpolator(self.model.grid.x,y[m:2*m,j],extrapolate=False)(x))
return np.array(temperatures),np.array(moistures)

def summary(self, sample_step_s=None, chunk_size=1024):
    """Diagnostics on an explicit sample grid, never continuous extrema.

    Default: 250 uniformly spaced times including both endpoints.
    A positive sample_step_s uses 0, step, ..., plus the exact endpoint.
    All radial solver nodes participate; chunking bounds memory use.
    """
    if chunk_size < 1 or int(chunk_size) != chunk_size:
        raise ValueError('chunk_size must be a positive integer')
    if sample_step_s is None:
        times=np.unique(np.r_[0.,np.linspace(0,self.end,250),self.end])
    else:
        if not np.isfinite(sample_step_s) or sample_step_s <= 0:
            raise ValueError('sample_step_s must be finite and positive')
        times=np.unique(np.r_[np.arange(0,self.end,sample_step_s),self.end])
    m=self.model.grid.n+1
    tmin=cmin=float('inf'); tmax=cmax=-float('inf')
    balance=above=0.
    for start in range(0,len(times),int(chunk_size)):
        y=self.values(times[start:start+int(chunk_size)])
        t,c=y[:m],y[m:2*m]
        if not np.isfinite(y).all():
            raise ValueError('Nonfinite state on diagnostic sample grid')
        means=2*self.model.grid.w@c
        balance=max(balance,float(np.abs(means+y[-1]-2.55).max()))
        above=max(above,float((c.max(axis=0)-c[0]).max()))
        tmin=min(tmin,float(t.min())); tmax=max(tmax,float(t.max()))
        cmin=min(cmin,float(c.min())); cmax=max(cmax,float(c.max()))
    yend=self.values([self.end]); t,c=yend[:m],yend[m:2*m]
    return dict(question=self.model.q,n=self.model.grid.n,end_s=self.end,
        crossing_s=self.crossing,strict_s=self.strict,
        crossing_h=None if self.crossing is None else self.crossing/3600,
        strict_h=None if self.strict is None else self.strict/3600,
        end_radius_cm=float(self.model.radius(self.end))*100,
        T_center_end=float(t[0,-1]),T_surface_end=float(t[-1,-1]),
        C_center_end=float(c[0,-1]),C_surface_end=float(c[-1,-1]),Cmax_end=float(c[:,-1].max()),
        ),
        diagnostic_sampling=dict(
            time_grid='uniform_250_including_endpoints' if sample_step_s is None else
            'fixed_step_plus_endpoint',
            time_count=len(times),first_time_s=float(times[0]),last_time_s=float(times[-1]),
            step_s=float(times[1]-times[0]) if sample_step_s is None and len(times)>1 else
            sample_step_s,
            radial_grid='all_solver_nodes',radial_node_count=m,
            continuous_extrema_claimed=False),
        moisture_balance_max_sampled=balance,
        max_above_center_sampled=above,

```

```

        T_min_sampled=tmin,T_max_sampled=tmax,C_min_sampled=cmin,C_max_sampled=cmax,
        nfev=sum(s.nfev for s in self.segments),runtime_s=self.runtime)

def integrate(q,n,arrays,horizon=None,stop_at_threshold=None,scenario=None,rtol=1e-8,graded=True):
    start=time.perf_counter()
    model=Model(q,n,arrays,scenario,graded)
    if horizon is None:
        horizon={1:1800.,2:10800.,3:604800.,4:259200.}[q]
    stop_at_threshold=q in (3,4) if stop_at_threshold is None else stop_at_threshold
    def event(t,y): return model.event(t,y)
    event.terminal=True;event.direction=-1
    m=n+1; y=np.r_[np.full(m,28.),np.full(m,2.55),0.]
    atol=np.r_[np.full(m,rtol),np.full(m,rtol*.01),rtol*.01]
    segments=[];crossing=None;strict=None;begin=0.
    for end in sorted(set([min(horizon,14400.),float(horizon)])):
        if end<=begin: continue
        sol=solve_ivp(model.rhs,(begin,end),y,method='BDF',rtol=rtol,atol=atol,
                      jac=model.jac,dense_output=True,
                      events=event if stop_at_threshold else None,max_step=15. if begin<14400 else 600.)
        if not sol.success: raise RuntimeError(sol.message)
        segments.append(sol);y=sol.y[:,-1];begin=float(sol.t[-1])
        if sol.status==1:
            crossing=float(sol.t_events[0][0]);strict=math.floor(crossing)+1
            extra=solve_ivp(model.rhs,(crossing,strict),y,method='BDF',rtol=rtol,atol=atol,
                           jac=model.jac,dense_output=True)
            if not extra.success or model.event(strict,extra.y[:,-1])>=0:
                raise RuntimeError('Strict completion endpoint failed')
            segments.append(extra)
            break
    run=Run(model,segments,crossing,strict,time.perf_counter()-start)
    info=run.summary()
    assert info['C_min_sampled']>0 and info['C_max_sampled']<=2.5500001
    assert info['T_min_sampled']>=27.99999 and info['T_max_sampled']<=max(arrays[0][:,1].max(),50.5)+1e-5
    assert info['moisture_balance_max_sampled']<1e-7
    return run

def smoke():
    arrays,snapshots=read_inputs()
    checks=[]
    for q in (1,2,4):
        run=integrate(q,80,arrays,horizon=600,stop_at_threshold=False)
        temp,moist=run.sample([1,60,600])
        checks.append(dict(**run.summary(),sample_T=np.where(np.isfinite(temp),temp,None).tolist(),sample_
            _C=np.where(np.isfinite(moist),moist,None).tolist()))

    jac_errors=[]
    rng=np.random.default_rng(SEED)
    for q in (1,2,3,4):
        model=Model(q,24,arrays)
        m=25; x=model.grid.x
        y=np.r_[32+3*x**2,2.3-.4*x**2,0.]
        direction=np.r_[rng.normal(size=m),.1*rng.normal(size=m),0.]
        eps=1e-5
        fd=(model.rhs(3600.,y+eps*direction)-model.rhs(3600.,y-eps*direction))/(2*eps)
        exact_jv=model.jac(3600.,y)@direction
        error=float(np.linalg.norm(fd-exact_jv,np.inf)/max(np.linalg.norm(exact_jv,np.inf),1e-30))

```

```

        assert error<1e-6
        jac_errors.append(dict(question=q,relative_directional_error=error))
    grid=Grid(160)
    beta=brentq(lambda b:b*j1(b)-j0(b),1,2)
    exact=math.log(2)*.02**2/(1e-5*beta**2)
    rhs=lambda t,c:grid.rate(c,1e-5,.02,1e-5/.02,.05)
    def event(t,c):return c.max()-.15
    event.terminal=True;event.direction=-1
    result=solve_ivp(rhs,(0,80),.05+.2*j0(beta*grid.x),method='BDF',rtol=1e-10,atol=1e-12,
                    jac_sparsity=grid.scalar_pattern,events=event)
    assert result.success and result.status==1 and abs(result.t[-1]-exact)<1e-3
    fixed=grid.rate(np.full(161,2.55),1e-9,.015,0.,0.)
    assert np.max(np.abs(fixed))==0
    output=dict(inputs=snapshots,runs=checks,jacobian_checks=jac_errors,analytic_event_s=float(result.t[-1]),exact_event_s=exact,
                sealed_uniform_error=float(np.max(np.abs(fixed))))
    (ROOT/'.work').mkdir(exist_ok=True)
    (ROOT/'.work/smoke.json').write_text(json.dumps(output,ensure_ascii=False,indent=2),encoding='utf-8')
    print(json.dumps({k:v for k,v in output.items() if k!='runs'},ensure_ascii=False))
    for row in checks: print(json.dumps({k:v for k,v in row.items() if not
    k.startswith('sample')},ensure_ascii=False))

if __name__=='__main__':
    parser=argparse.ArgumentParser()
    parser.add_argument('--smoke',action='store_true')
    args=parser.parse_args()
    if args.smoke: smoke()
    else: parser.error('Use --smoke, or run the complete pipeline with run_all.py')

```

B.2 全流程计算与结果导出

```

"""Reproduce numerical tables, convergence, sensitivity and figures."""
from pathlib import Path
import argparse
import copy
import json
import subprocess
import sys
import time
import warnings
from solve import ROOT, SEED, Scenario, integrate, read_inputs
import numpy as np
import pandas as pd
from openpyxl import load_workbook
from export_result2_full import export_run, sample as sample_q3

RESULTS=ROOT/'results'

def emit(value):
    print(json.dumps(value,ensure_ascii=False),flush=True)

def sampling_times(run,step,initial=False):
    return np.unique(np.r_[0. if initial else [],np.arange(step,run.end,step),run.end])

def profile(run,times,radii=None,with_surface=False):

```

```

t,c=run.sample(times,radii)
if with_surface:
    y=run.values(times);m=run.model.grid.n+1
    t=np.c_[t,y[m-1]];c=np.c_[c,y[2*m-1]]
return t,c

def compare(coarse,fine,q):
    end=min(coarse.end,fine.end)
    times=np.arange(1,1801) if q==1 else np.arange(1,int(end)+1) if q==2 else
    np.r_[np.arange(60,end,60),end]
    radii=np.arange(20)/10 if q==4 else np.arange(21)/10
    dt=dc=0.
    for start in range(0,len(times),1024):
        block=times[start:start+1024]
        at,ac=sample_q3(coarse,block) if q==2 else profile(coarse,block,radii,q==4)
        bt,bc=sample_q3(fine,block) if q==2 else profile(fine,block,radii,q==4)
        dt=max(dt,float(np.nanmax(abs(at-bt))))
        dc=max(dc,float(np.nanmax(abs(ac-bc))))
    event_diff=None if q in (1,2) else abs(coarse.crossing-fine.crossing)/fine.crossing
    return dict(question=q,coarse_n=coarse.model.grid.n,fine_n=fine.model.grid.n,
        temperature_max_difference=dt,moisture_max_difference=dc,
        event_relative_difference=event_diff,sampled_times=len(times),
        passed=dt<.005 and dc<5e-5 and (event_diff is None or event_diff<.001))

def write_book(number,times,arrays,headers):
    template=ROOT/'附件/附件 3'/f'result{number}.xlsx'
    wb=load_workbook(template)
    for ws,values in zip(wb.worksheets,arrays):
        style=copy.copy(ws['B2'].style)
        ws.cell(1,1,'时间\到药材中心的距离')
        for col,label in enumerate(headers,2):
            ws.cell(1,col,label)
        for row,(instant,values_row) in enumerate(zip(times,values),2):
            ws.cell(row,1,int(round(instant)))
            for col,value in enumerate(values_row,2):
                cell=ws.cell(row,col)
                cell.style=copy.copy(style)
                cell.value=None if np.isnan(value) else float(f'{value:.4f}')
                cell.number_format='0.0000'
        ws.freeze_panes='B2'
    target=RESULTS/f'result{number}.xlsx'
    wb.save(target)
    check=load_workbook(target,read_only=True,data_only=True)
    for ws in check.worksheets:
        assert ws.max_row==len(times)+1 and ws.max_column==len(headers)+1
    return dict(path=target.name,rows=len(times),columns=len(headers)+1,bytes=target.stat().st_size)

def save_tables(runs,q2_horizon):
    outputs=[]
    for q in (1,2,3,4):
        run=runs[3] if q==2 else runs[q]
        end=10800 if q==2 and q2_horizon=='3h' else run.end
        times=np.arange(1,int(end)+1) if q in (1,2) else sampling_times(run,60)
        radii=np.arange(20 if q==4 else 21)/10
        headers=radii.tolist()+(['药材表面'] if q==4 else [])
        if q==2:

```

```

        outputs.append(export_run(run, RESULTS, rows=int(end)))
    else:
        t, c = profile(run, times, radii, q==4)
        outputs.append(write_book(q, times, [t, c] if q==1 else [c], headers))
    if q==4:
        pd.DataFrame({'time_s': times, 'radius_cm': run.model.radius(times)*100}).to_csv(RESULTS/'q4_radius.csv', index=False)
    if q==1: paper_times = np.array([100, 300, 600, 900, 1200, 1500, 1800])
    elif q==2: paper_times = np.arange(1800, 10801, 1800)
    else: paper_times = sampling_times(run, 21600)
    rad = np.array([0, .5, 1, 1.5]) if q==4 else np.arange(5)/2
    pt, pc = profile(run, paper_times, rad, q==4)
    cols = [f'r_{v:g}_cm' for v in rad] + (['surface'] if q==4 else [])
    for tab, values in ((1, pt), (2, pc)) if q==1 else ((3, pt), (4, pc)) if q==2 else ((5 if q==3 else 6, pc)):
        frame = pd.DataFrame(values, columns=cols)
        frame.insert(0, 'time_s' if q==1 else 'time_h', paper_times if q==1 else paper_times/3600)
        frame.to_csv(RESULTS/f'table{tab}.csv', index=False, float_format='%.10g')
    return outputs

def save_fields(runs):
    for q, run in runs.items():
        times = sampling_times(run, 10 if q==1 else 60, initial=True)
        xi = np.linspace(0, 1, 101)
        t, c = run.sample(times, xi, normalized=True)
        y = run.values(times); m = run.model.grid.n+1
        cmax = y[m:2*m].max(axis=0); cmean = 2*run.model.grid.w@m[m:2*m]
        np.savez_compressed(RESULTS/f'q{q}_fields.npz', time_s=times, x=xi,
                           radius_cm=run.model.radius(times)*100, temperature=t, moisture=c,
                           Cmax=cmax, Cmean=cmean, Csurface=y[2*m-1], Tcenter=y[0], Tsurface=y[m-1],
                           balance_error=cmean+y[-1]-2.55)
        pd.DataFrame(dict(time_s=times, Cmax=cmax, Cmean=cmean, Csurface=y[2*m-1],
                           Tcenter=y[0], Tsurface=y[m-1])).to_csv(RESULTS/f'q{q}_history.csv', index=False)

def compute(q2_horizon='full'):
    started = time.perf_counter(); RESULTS.mkdir(exist_ok=True)
    arrays, snapshot = read_inputs()
    pd.DataFrame(arrays[0], columns=['time_s', 'temperature_C', 'moisture_kg_kg']).to_csv(RESULTS/'input_environment.csv', index=False)
    pd.DataFrame(arrays[1], columns=['time_s', 'radius_cm']).to_csv(RESULTS/'input_radius.csv', index=False)
    runs_by_grid = {}; summaries = []; comparisons = []; last = None
    for n in (160, 320, 640, 1280, 2560):
        current = {}
        for q in (1, 3, 4):
            current[q] = integrate(q, n, arrays)
            if q in (3, 4) and current[q].crossing is None:
                raise RuntimeError(f'q{q} threshold not reached within prescribed search range')
            row = current[q].summary(); summaries.append(row); emit(row)
        if last is not None:
            batch = [compare(last[1] if q==1 else last[4] if q==4 else last[3],
                            current[1] if q==1 else current[4] if q==4 else current[3], q) for q in (1, 2, 3, 4)]
            comparisons.extend(batch); emit({'comparisons': batch})
            if n >= 640 and all(r['passed'] for r in batch): break
        last = current
    else: raise RuntimeError('Spatial convergence targets not met up to N2560')

```

```

pd.DataFrame(summaries).to_csv(RESULTS/'grid_runs.csv',index=False)
pd.DataFrame(comparisons).to_csv(RESULTS/'grid_comparison.csv',index=False)
final=current;n=final[1].model.grid.n
time_checks=[]
for q in (1,3,4):
    tight=integrate(q,n,arrays,rtol=1e-9)
    row=compare(final[q],tight,q)
    row['check']='time_tolerance_1e-8_vs_1e-9'
    time_checks.append(row)
    if q==3:
        second=compare(final[q],tight,2);second['check']=row['check'];time_checks.append(second)
        if not second['passed']:raise RuntimeError('Time tolerance convergence failed q2')
    if not row['passed']:raise RuntimeError(f'Time tolerance convergence failed q{q}')
pd.DataFrame(time_checks).to_csv(RESULTS/'time_comparison.csv',index=False)
books=save_tables(final,q2_horizon);save_fields(final)
scenarios=[('last',Scenario(tail='last')),('mean',Scenario(tail='mean')),
            ('T_minus_0.5',Scenario(temperature_shift=-.5)),('T_plus_0.5',Scenario(temperature_shift=.5)),
            ('Ce_minus_0.001',Scenario(moisture_shift=-.001)),('Ce_plus_0.001',Scenario(moisture_shift=.001)),
            ('hm_minus_10pct',Scenario(mass_factor=.9)),('hm_plus_10pct',Scenario(mass_factor=1.1))]
sensitivity=[]
for q in (3,4):
    baseline=final[q].crossing
    sensitivity.append(dict(question=q,scenario='nominal',crossing_h=baseline/3600,change_pct=0.))
    for name,settings in scenarios:
        run=integrate(q,n,arrays,scenario=settings)
        if run.crossing is None: raise RuntimeError(f'Sensitivity {q}/{name} exceeds data range')
        row=dict(question=q,scenario=name,crossing_h=run.crossing/3600,
                change_pct=100*(run.crossing/baseline-1))
        sensitivity.append(row);emit(row)
    counter=integrate(4,n,arrays,horizon=604800,scenario=Scenario(fixed_radius=True))
    counter_info=counter.summary();emit({'counterfactual':counter_info})
    sensitivity.append(dict(question=4,scenario='fixed_R_appendix4',crossing_h=counter.crossing/3600 if
    counter.crossing else None,
                        change_pct=100*(counter.crossing/final[4].crossing-1) if counter.crossing
                        else None))
pd.DataFrame(sensitivity).to_csv(RESULTS/'sensitivity.csv',index=False)
report=dict(seed=SEED,grid_n=n,rtol=1e-8,q2_horizon=q2_horizon,
            runs={str(q):run.summary() for q,run in final.items()},
            output_grid_diagnostics={str(q):run.summary(sample_step_s=1 if q in (1,3) else 60)
                                    for q,run in final.items()},
            counterfactual=counter_info,grid_comparisons=comparisons,time_comparisons=time_checks,
            workbooks=books,compute_runtime_s=time.perf_counter()-started)
(RESULTS/'summary.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
emit({'compute_complete':report})

def main():
    warnings.filterwarnings('error',category=RuntimeWarning)
    parser=argparse.ArgumentParser()
    parser.add_argument('--compute-only',action='store_true')
    parser.add_argument('--figures-only',action='store_true')
    parser.add_argument('--q2-horizon',choices=['3h','full'],default='full')
    args=parser.parse_args()
    if not args.figures_only:
        if (RESULTS/'result1.xlsx').exists() or (RESULTS/'result2.xlsx').exists():

```

```

        raise FileExistsError('Results already exist; use reproduce.py for a fresh isolated run')
    subprocess.run([sys.executable, '-B', str(ROOT/'verify_moving_domain.py'),
                    '--output-dir', str(RESULTS)], check=True)
    compute(args.q2_horizon)
if not args.compute_only:
    from make_figures import make_figures
    make_figures()
    script=ROOT/'utils/repro_manifest.py'
    command=[sys.executable, str(script), '--project-root', str(ROOT), '--seed', str(SEED),
              '--parameters', json.dumps({'grid_n': json.loads((RESULTS/'summary.json').read_text())['grid_n'],
                                          'rtol': 1e-8, 'q2_horizon': args.q2_horizon, 'mesh': '1-(1-s)^2'}),
              '--command', f'OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 python3 run_all.py --q2-horizon {args.q2_horizon}',
              '--overwrite']
    for path in [ROOT/'A题.pdf', *sorted((ROOT/'附件').rglob('*.xlsx'))]:
        command.extend(['--input', str(path)])
    for package in ['numpy', 'scipy', 'pandas', 'openpyxl', 'matplotlib']:
        command.extend(['--package', package])
    env=dict(__import__('os').environ)
    env['PYTHONPATH']=str(ROOT/'.work/deps')+__import__('os').pathsep+env.get('PYTHONPATH', '')
    subprocess.run(command, check=True, env=env)

if __name__=='__main__':main()

```

B.3 真实结果可视化

```

"""Generate evidence-led figures from completed, unrounded result files."""
from pathlib import Path
import json
import logging
import os
import sys
from solve import ROOT, properties
os.environ.setdefault('MPLCONFIGDIR', str(ROOT/'.work/mplconfig'))
os.environ.setdefault('SOURCE_DATE_EPOCH', '1788998400')
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.colors import LogNorm
from matplotlib.ticker import FuncFormatter
from utils.setup_style import setup_style, CJK_FONT_PRIORITY
from utils.plot_style import export_figure as audited_export
from utils.export_figure import export_figure as pdf_export

OUT=ROOT/'figures'
DATA=ROOT/'results'
BLUE='#0072B2'; ORANGE='#D55E00'; GREEN='#009E73'; GRAY='#666666'
COLORS=[BLUE, ORANGE, GREEN]
LINES=['-', '--', '-.']
WIDTH=6.3

class GlyphFailureHandler(logging.Handler):

```

```

def emit(self,record):
    message=record.getMessage()
    if 'does not have a glyph' in message or 'missing from font' in message:
        raise RuntimeError(message)

def configure():
    logging.getLogger('matplotlib').addHandler(GlyphFailureHandler())
    sources=[ROOT/'assets/wqy-zenhei.ttc',
             ROOT/'./work/tex_runtime/usr/share/fonts/truetype/wqy/wqy-zenhei.ttc',
             Path('/usr/share/fonts/truetype/wqy/wqy-zenhei.ttc')]
    source=next((p for p in sources if p.exists()),None)
    if source is None:
        raise RuntimeError('Install fonts-wqy-zenhei or supply assets/wqy-zenhei.ttc')
    font_manager.fontManager.addfont(str(source))
    CJK_FONT_PRIORITY.insert(0,'WenQuanYi Zen Hei')
    setup_style(journal='general',lang='zh',use_sciplots=False)
    plt.rcParams.update({'font.size':8.5,'axes.labelsize':9,'axes.titlesize':9,
                        'legend.fontsize':8,'xtick.labelsize':8,'ytick.labelsize':8,
                        'axes.titlelocation':'left','axes.spines.top':False,
                        'axes.spines.right':False,'lines.linewidth':1.4,
                        'figure.facecolor':'white','axes.facecolor':'white',
                        'legend.frameon':False,'savefig.dpi':300,'svg.hashsalt':'20260910'})
    OUT.mkdir(exist_ok=True)

def panels(count=1,height=2.9):
    return plt.subplots(1,count,figsize=(WIDTH,height),layout='constrained',squeeze=False)[0:2]

def save(fig,stem):
    for ax in fig.axes:
        ax.minorticks_off()
        if ax.get_yscale()=='log' and not hasattr(ax,'_colorbar'):
            ax.yaxis.set_major_formatter(FuncFormatter(lambda x,pos:f'{x:.0e}'))
        if hasattr(ax,'_colorbar') and ax._colorbar.solids is not None:
            ax._colorbar.solids.set_rasterized(False)
    audited_export(fig,OUT/stem,dpi=300,strict_layout=True,strict_design=True)
    pdf_export(fig,str(OUT/stem),formats=['pdf'],dpi=300,
               size_inches=tuple(fig.get_size_inches()),grayscale_preview=False,tight=False)
    plt.close(fig)

def series(ax,x,ys,labels):
    for y,label,color,ls in zip(ys,labels,COLORS,LINES):
        ax.plot(x,y,label=label,color=color,ls=ls)
    ax.legend()
    ax.margins(x=0.01)

def make_figures():
    configure()
    env=pd.read_csv(DATA/'input_environment.csv')
    rad=pd.read_csv(DATA/'input_radius.csv')
    summary=json.loads((DATA/'summary.json').read_text())
    q1=np.load(DATA/'q1_fields.npz');q3=np.load(DATA/'q3_fields.npz');q4=np.load(DATA/'q4_fields.npz')

    fig,axes=panels(2);a,b=axes[0]
    a.plot(env.time_s/3600,env.temperature_C,color=BLUE)
    b.plot(env.time_s/3600,env.moisture_kg_kg,color=ORANGE)
    for ax in (a,b):ax.set_xlabel('时间 / h');ax.set_xlim(0,4)

```

```

a.set_ylabel('环境温度 /  $^{\circ}\text{C}$ ');b.set_ylabel('有效平衡含水率 / (kg/kg)')
a.set_title('(a) 温度边界');b.set_title('(b) 水分边界')
save(fig, 'raw_q1_environment')

tail=env[env.time_s>=10800]
fig,axes=panels(2);a,b=axes[0]
a.hist(tail.temperature_C,bins=8,color=BLUE,alpha=.75,edgecolor='white')
b.hist(tail.moisture_kg_kg,bins=8,color=ORANGE,alpha=.75,edgecolor='white')
a.axvline(50,color='black',ls='--',lw=1);b.axvline(.05,color='black',ls='--',lw=1)
a.set_xlabel('环境温度 /  $^{\circ}\text{C}$ ');b.set_xlabel('有效平衡含水率 / (kg/kg)')
for ax in (a,b):ax.set_ylabel('频数');ax.set_title('末小时: 61 个记录')
b.ticklabel_format(axis='x',style='plain',useOffset=False)
b.set_xticks([.0498,.05,.0502])
save(fig, 'raw_q2_tail')

fig,axes=panels(height=2.7);a=axes[0,0];c=np.linspace(.05,2.55,501)
for t,ls,col in ((28,'--',BLUE),(50,'-',ORANGE)):
    a.semilogy(c,properties(3,np.full_like(c,t),c)[2],color=col,ls=ls,label=f'{t} $^{\circ}\text{C}$ ')
a.set_xlabel('干基含水率 C / (kg/kg)');a.set_ylabel('扩散系数 D / ( $\text{m}^2/\text{s}$ )')
a.legend();a.set_title('题设物性函数 (非实测)')
save(fig, 'raw_q3_diffusivity')

fig,axes=panels(height=2.7);a=axes[0,0]
a.plot(rad.time_s/3600,rad.radius_cm,color=BLUE,lw=1)
a.scatter(rad.time_s/3600,rad.radius_cm,s=7,color=BLUE)
a.set_xlabel('时间 / h');a.set_ylabel('药材半径 / cm');a.set_xlim(0,72)
a.set_title('实测半径: 145 个记录')
save(fig, 'raw_q4_radius')

fig,axes=panels(height=2.7);a=axes[0,0]
times=q1['time_s']
series(a,times/60,[q1['Tcenter'],q1['Tsurface']],np.interp(times,env.time_s,env.temperature_C),
      ['中心','表面','环境'])
a.set_xlabel('时间 / min');a.set_ylabel('温度 /  $^{\circ}\text{C}$ ')
save(fig, 'process_q1_heating')

fig,axes=panels(2);a,b=axes[0]
for t,color,ls in zip([100,600,1800],COLORS,LINES):
    i=np.flatnonzero(times==t)[0]
    a.plot(q1['x']*2,q1['temperature'][i],color=color,ls=ls,label=f'{t} s')
    b.plot(q1['x']*2,q1['moisture'][i],color=color,ls=ls,label=f'{t} s')
a.set_ylabel('温度 /  $^{\circ}\text{C}$ ');b.set_ylabel('干基含水率 / (kg/kg)')
for ax in (a,b):ax.set_xlabel('半径位置 / cm');ax.legend()
a.set_title('(a) 温度');b.set_title('(b) 水分')
save(fig, 'result_q1_profiles')

take=q3['time_s']<=10800;tt=q3['time_s'][take]/3600;xx=q3['x']*2
temps=q3['temperature'][take];moist=q3['moisture'][take]
diff=properties(3,temps,moist)[2]
fig,axes=panels(height=2.9);a=axes[0,0]
mesh=a.pcolormesh(tt,xx,diff.T,norm=LogNorm(),cmap='viridis',shading='auto')
cb=fig.colorbar(mesh,ax=a,ticks=np.geomspace(diff.min(),diff.max(),4),
               format=FuncFormatter(lambda x,pos:f'{x:.1e}'))
cb.minorticks_off();cb.set_label('D / ( $\text{m}^2/\text{s}$ ), 对数色标')
a.set_xlabel('时间 / h');a.set_ylabel('半径位置 / cm')
save(fig, 'process_q2_diffusion')

```

```

fig, axes = panels(2, height=3.1); a, b = axes[0]
for ax, field, label in [(a, temps, '温度 / °C'), (b, moist, 'C / (kg/kg)')]:
    mesh = ax.pcolormesh(tt, xx, field.T, cmap='viridis', shading='auto')
    cb = fig.colorbar(mesh, ax=ax, pad=.02); cb.set_label(label)
    ax.set_xlabel('时间 / h'); ax.set_ylabel('半径位置 / cm')
a.set_title('(a) 温度场'); b.set_title('(b) 含水率场')
save(fig, 'result_q2_fields')

fig, axes = panels(height=2.9); a = axes[0, 0]
series(a, q3['time_s']/3600, [q3['Cmax'], q3['Cmean'], q3['Csurface']], ['全域最大', '径向体积权均值', '表面'])
a.axhline(.15, color='black', ls=':', lw=1.1)
a.set_xlabel('时间 / h'); a.set_ylabel('干基含水率 / (kg/kg)')
save(fig, 'process_q3_threshold')

fig, axes = panels(height=2.7); a = axes[0, 0]
a.plot(xx, q3['moisture'][-1], color=BLUE, label='严格终点的含水率')
a.axhline(.15, color='black', ls='--', label='阈值 0.15')
a.set_xlabel('半径位置 / cm'); a.set_ylabel('干基含水率 / (kg/kg)')
a.legend(); a.set_ylim(0, .17)
save(fig, 'result_q3_terminal')

fig, axes = panels(height=3.1); a = axes[0, 0]
ts = q4['time_s']/3600; rs = q4['radius_cm']; rr = np.linspace(0, 2, 161)
physical = np.full((len(ts), len(rr)), np.nan)
for i, radius in enumerate(rs):
    inside = rr <= radius
    physical[i, inside] = np.interp(rr[inside]/radius, q4['x'], q4['moisture'][i])
mesh = a.contourf(ts, rr, np.ma.masked_invalid(physical.T), cmap='viridis',
                  levels=np.linspace(0, 2.55, 22), corner_mask=False)
fig.colorbar(mesh, ax=a).set_label('C / (kg/kg)')
a.plot(ts, rs, color='black', lw=1, label='实际表面')
a.set_xlabel('时间 / h'); a.set_ylabel('物理半径位置 / cm'); a.legend(loc='upper right')
save(fig, 'process_q4_moving')

fig, axes = panels(height=2.7); a = axes[0, 0]
labels = ['问题 3: 固定半径', '问题 4 物性: 固定半径', '问题 4: 实测收缩']
hours = [summary['runs']['3']['crossing_h'], summary['counterfactual']['crossing_h'],
         summary['runs']['4']['crossing_h']]
for i, (value, color) in enumerate(zip(hours, COLORS)):
    a.scatter(value, i, color=color, s=32, zorder=3)
    a.annotate(f'{value:.3f} h', (value, i), xytext=(6, 5), textcoords='offset points', fontsize=8)
a.set_yticks(range(3), labels); a.set_xlabel('全域达到阈值的事件时间 / h')
a.set_xlim(0, max(hours)*1.18); a.set_ylim(2.5, -.5); a.grid(axis='x', alpha=.2)
save(fig, 'result_q4_counterfactual')

grid = pd.read_csv(DATA/'grid_comparison.csv')
fig, axes = panels(2); a, b = axes[0]
for q, color, ls in zip([1, 2, 3, 4], [BLUE, ORANGE, GREEN, GRAY], ['-', '--', '-.', ':']):
    frame = grid[grid.question == q]
    a.loglog(frame.fine_n, frame.moisture_max_difference, color=color, ls=ls, marker='o', label=f'问题 {q}')
    b.loglog(frame.fine_n, frame.temperature_max_difference, color=color, ls=ls, marker='o', label=f'问题 {q}')
a.axhline(5e-5, color='black', lw=.8); b.axhline(.005, color='black', lw=.8)
for ax in (a, b):
    ax.set_xlabel('细网格区间数 N'); ax.legend(ncol=2)

```

```

        ticks=sorted(grid.fine_n.unique());ax.set_xticks(ticks,[str(t) for t in ticks])
a.set_ylabel('含水率最大差 / (kg/kg)');b.set_ylabel('温度最大差 / °C')
save(fig,'process_q3_convergence')

sens=pd.read_csv(DATA/'sensitivity.csv')
names=['last','mean','T_minus_0.5','T_plus_0.5','Ce_minus_0.001','Ce_plus_0.001','hm_minus_10pct','hm_
_plus_10pct']
labels=['末值延拓','末小时均值延拓','尾段温度 -0.5°C','尾段温度 +0.5°C','尾段平衡值 -0.001','尾段平衡值
+0.001','传质系数 -10%','传质系数 +10%']
fig,axes=panels(height=3.2);a=axes[0,0]
for q,col,offset,marker in [(3,BLUE,-.12,'o'),(4,ORANGE,.12,'s')]:
    frame=sens[sens.question==q].set_index('scenario')
    vals=frame.loc[names,'change_pct'].values
    a.scatter(vals,np.arange(8)+offset,color=col,marker=marker,s=22,label=f'问题 {q}')
a.set_yticks(range(8),labels);a.invert_yaxis();a.axvline(0,color='black',lw=.7)
a.set_xlabel('事件时间相对名义情景的变化 / %');a.legend(loc='lower right')
save(fig,'result_q4_sensitivity')
print(json.dumps({'figures':len(list(OUT.glob('*.svg'))),'format':['svg','png','pdf'],'dpi':300}))

if __name__=='__main__':make_figures()

```

B.4 移动域解析基准与错误算子测试

"""Moving Neumann/Bessel benchmark of the unchanged production radial operator.

No output files unless --output-dir is supplied. --mutant-only MUST exit 1.

Run from the synchronized project: python3 -B verify_moving_domain.py

"""

from __future__ import annotations

import argparse

import csv

import hashlib

import json

from pathlib import Path

import sys

import time

import warnings

HERE = Path(__file__).resolve().parent

PROJECT = HERE

sys.path.insert(0, str(PROJECT))

from solve import Grid, SEED, integrate, read_inputs

import numpy as np

from scipy.integrate import solve_ivp

from scipy.linalg import eigh_tridiagonal

from scipy.special import j0, j1, jn_zeros

RO, TF, D, CO, AMP = 0.02, 3600.0, 1e-8, 1.0, 0.3

```

BETA = 1.0 / (2 * TF)
LAM = float(jn_zeros(1, 1)[0])
TIMES = np.linspace(0, TF, 129)
SPACE_LIMIT, TIME_LIMIT, MASS_LIMIT = 1e-5, 5e-6, 1e-10

def radius(t):
    return R0 * (1 - BETA * np.asarray(t))

def clock_integral(t):
    t = np.asarray(t)
    return t / (R0**2 * (1 - BETA * t))

def exact(x, times=TIMES):
    return C0 + AMP * j0(LAM * np.asarray(x)[:, None]) * np.exp(
        -LAM**2 * D * clock_integral(times)[None, :])
    )

def semidiscrete_reference(grid, initial, times=TIMES):
    """Matrix exponential via the weighted-symmetric tridiagonal operator.

    Used on a fixed UNIFORM spatial grid to isolate temporal error. The actual
    PDE/Bessel comparison is reported independently; no fitted reference data.
    """
    lap = grid.jac_block(initial, 1.0, 0.0, 1.0, 0.0, same=True)
    sw = np.sqrt(grid.w)
    upper = lap.diagonal(1) * sw[:-1] / sw[1:]
    lower = lap.diagonal(-1) * sw[1:] / sw[:-1]
    assert np.allclose(upper, lower, rtol=1e-13, atol=1e-10)
    vals, vecs = eigh_tridiagonal(lap.diagonal(), upper)
    assert abs(vals[-1]) < 1e-7 and vals[-2] < 0
    # The zero eigenvalue is known exactly from the conservative Neumann matrix.
    vals[-1] = 0.0
    coeff = vecs.T @ (sw * initial)
    answer = (vecs @ (coeff[:, None] * np.exp(
        vals[:, None] * D * clock_integral(times)[None, :])
    ))) / sw[:, None]
    assert np.max(abs(answer[:, 0] - initial)) < 1e-11
    return answer

def solve_case(n, *, graded=True, rtol=1e-11, max_step=TF/256,
               mutant=False, temporal_reference=False):
    grid = Grid(n, graded=graded)
    initial = exact(grid.x, np.array([0.0]))[:, 0]
    # jac_block and rate both come from solve.py, never copied/replaced locally.
    lap = grid.jac_block(initial, D, 0.0, 1.0, 0.0, same=True)

    def r_used(t):
        return R0 if mutant else float(radius(t))

    def rhs(t, c):
        return grid.rate(c, D, r_used(t), 0.0, 0.0)

```

```

result = solve_ivp(rhs, (0, TF), initial, method="BDF", rtol=rtol,
                  atol=rtol*0.01, max_step=max_step,
                  jac=lambda t, c: lap/r_used(t)**2, dense_output=True)
if not result.success:
    raise RuntimeError(result.message)
values = result.sol(TIMES)
reference = exact(grid.x)
mass0 = float(2*grid.w @ initial)
# Check every accepted step as well as all shared comparison instants.
mass_drift = max(float(np.max(abs(2*grid.w @ values-mass0))),
                 float(np.max(abs(2*grid.w @ result.y-mass0))))
continuum_error = float(np.max(abs(values-reference)))
temporal_error = None
if temporal_reference:
    temporal_error = float(np.max(abs(values-semidiscrete_reference(grid, initial))))
row = dict(n=n, graded=graded, rtol=rtol, atol=rtol*0.01,
          max_step_s=max_step, accepted_steps=len(result.t)-1,
          actual_max_step_s=float(np.diff(result.t).max()),
          continuum_max_error=continuum_error, temporal_max_error=temporal_error,
          mass_drift_max=mass_drift, initial_mean_quadrature_error=mass0-C0,
          numerical_center_final=float(values[0,-1]),
          exact_center_final=float(reference[0,-1]),
          mutant=mutant, nfev=result.nfev)
return row

def regression_checks():
    arrays, snapshots = read_inputs()
    rows = []
    for q in (1, 3, 4):
        run = integrate(q, 80, arrays, horizon=600, stop_at_threshold=False)
        times = np.unique(np.concatenate([np.arange(0, 601)]
                                         + [s.t for s in run.segments]))
        values = run.values(times)
        m = run.model.grid.n+1
        residual = float(np.max(abs(2*run.model.grid.w@values[m:2*m]
                                   +values[-1]-2.55)))
        closed = run.model.grid.rate(np.full(m, 2.55), 1e-9, 0.015, 0, 0)
        rows.append(dict(question=q, mass_balance_max=residual,
                        closed_uniform_rate_max=float(np.max(abs(closed))),
                        radius_start_m=float(run.model.radius(0)),
                        radius_end_m=float(run.model.radius(600)),
                        passed=bool(residual < 1e-7 and np.max(abs(closed)) == 0)))
    return rows, snapshots

def run_suite():
    start = time.perf_counter()
    space = [solve_case(n) for n in (40, 80, 160, 320, 640)]
    temporal = [solve_case(160, graded=False, rtol=tol, max_step=step,
                          temporal_reference=True)
               for tol, step in ((1e-3, TF/8), (1e-5, TF/16), (1e-7, TF/32))]
    mutant = solve_case(640, mutant=True)
    regressions, snapshots = regression_checks()
    se = [r["continuum_max_error"] for r in space]

```

```

te = [r["temporal_max_error"] for r in temporal]
checks = dict(
    radius_changes=radius(TF) != radius(0),
    nonuniform_initial=np.ptp(exact(np.linspace(0,1,101),np.array([0]))) > 0.1,
    neumann_eigenvalue=abs(j1(LAM)) < 1e-12,
    correct_operator=se[-1] <= SPACE_LIMIT,
    space_errors_decrease=all(b < a for a,b in zip(se,se[1:])),
    time_errors_decrease=all(b < a for a,b in zip(te,te[1:])),
    time_grid_refines=all(b["accepted_steps"] > a["accepted_steps"]
        for a,b in zip(temporal,temporal[1:])),
    finest_temporal_error=te[-1] <= TIME_LIMIT,
    mutant_rejected=mutant["continuum_max_error"] >= 100*SPACE_LIMIT,
    closed_mass_preserved=all(r["mass_drift_max"] <= MASS_LIMIT
        for r in space+temporal+[mutant]),
    original_mass_regression=all(r["passed"] for r in regressions),
)
checks = {k:bool(v) for k,v in checks.items()}
return dict(status="PASS" if all(checks.values()) else "FAIL", checks=checks,
    parameters=dict(R0_m=R0, final_radius_m=float(radius(TF)), tf_s=TF,
        D_m2_s=D, C0=C0, amplitude=AMP, beta_per_s=BETA,
        eigenvalue=LAM, comparison_times_s=TIMES.tolist(),
        spatial_error_limit=SPACE_LIMIT, time_error_limit=TIME_LIMIT,
        closed_mass_limit=MASS_LIMIT),
    spatial=space, temporal=temporal, mutant=mutant,
    mass_regressions=regressions, inputs=snapshots, seed=SEED,
    production_solver_sha256=hashlib.sha256((PROJECT/"solve.py").read_bytes()).hexdigest(),
    benchmark_sha256=hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
    runtime_s=time.perf_counter()-start)

def main():
    warnings.filterwarnings("error", category=RuntimeWarning)
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--output-dir", type=Path)
    parser.add_argument("--mutant-only", action="store_true")
    args = parser.parse_args()
    if args.mutant_only:
        row = solve_case(640, mutant=True)
        print(json.dumps(row, ensure_ascii=False, indent=2))
        return 1 if row["continuum_max_error"] > SPACE_LIMIT else 0
    report = run_suite()
    if args.output_dir:
        out = args.output_dir.resolve()
        if PROJECT not in out.parents:
            raise ValueError("Outputs must stay below A 题")
        out.mkdir(parents=True, exist_ok=True)
        (out/"moving_domain_benchmark.json").write_text(
            json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
        with (out/"moving_domain_convergence.csv").open("w", newline="", encoding="utf-8") as stream:
            rows = [dict(kind=k, **r) for k, group in
                ((("space",report["spatial"]),("time",report["temporal"])),
                 ("mutant",[report["mutant"]])) for r in group]
            writer = csv.DictWriter(stream, fieldnames=list(rows[0]))
            writer.writeheader(); writer.writerows(rows)
    print(json.dumps(report, ensure_ascii=False, indent=2))
    return 0 if report["status"] == "PASS" else 1

```

```
if __name__ == "__main__":
    sys.exit(main())
```

B.5 全过程逐秒工作簿导出与核对

```
"""Template-preserving, bounded-memory export of q2 from the q3 dense solution.
```

```
Imported by run_all.py; the complete isolated entry point is reproduce.py.
--verify-only audits an existing workbook without writing it.
"""
```

```
from __future__ import annotations
```

```
import argparse
from copy import copy
import csv
import hashlib
from io import BytesIO
import json
from pathlib import Path
import re
import sys
import time
import warnings
from xml.etree import ElementTree as ET
from zipfile import ZipFile, ZIP_DEFLATED

HERE = Path(__file__).resolve().parent
PROJECT = HERE
sys.path.insert(0, str(PROJECT))
from solve import SEED, integrate, read_inputs
from utils.repro_manifest import build_manifest
import numpy as np
from scipy.interpolate import PchipInterpolator
from openpyxl import load_workbook
from openpyxl.utils import get_column_letter
```

```
NS = "{http://schemas.openxmlformats.org/spreadsheetml/2006/main}"
COLS = [get_column_letter(i) for i in range(1,23)]
RADII = np.arange(21)/10
```

```
def sha(path):
    digest = hashlib.sha256()
    with Path(path).open("rb") as stream:
        for block in iter(lambda:stream.read(1024*1024),b""):
            digest.update(block)
    return digest.hexdigest()
```

```
def sample(run, times):
    """Same PCHIP as Run.sample, vectorized over a bounded time block."""
    times = np.asarray(times)
    assert run.model.q == 3 and np.all(run.model.radius(times) == .02)
    state = run.values(times)
```

```

m = run.model.grid.n+1
coords = RADII/100/.02
t = PchipInterpolator(run.model.grid.x,state[:m],axis=0)(coords).T
c = PchipInterpolator(run.model.grid.x,state[m:2*m],axis=0)(coords).T
assert np.isfinite(t).all() and np.isfinite(c).all()
return t,c

def template_skeleton():
    """Load and edit the actual template; preserve all non-data ZIP members.

    Only worksheet row data are streamed into this openpyxl-produced skeleton.
    No sheet/name/style/theme relationships are reconstructed by hand.
    """
    wb = load_workbook(PROJECT/" 附件/附件 3/result2.xlsx")
    assert wb.sheetnames == [" 温度", " 水分浓度"]
    assert not wb.defined_names and not wb._external_links
    styles=[]
    for ws in wb:
        assert not ws.merged_cells and not ws._images and not ws._charts
        assert not ws.tables and not ws.data_validations.dataValidation
        assert not ws.conditional_formatting and not ws.auto_filter.ref
        assert all(c.data_type != "f" for row in ws for c in row)
        header_style=copy(ws["B1"]._style)
        value_style=copy(ws["B2"]._style)
        time_style=copy(ws["A2"]._style)
        ws.delete_rows(2,ws.max_row-1) # Only placeholders in the in-memory copy.
        for col,r in enumerate(RADII,2):
            cell=ws.cell(1,col,float(r));cell._style=copy(header_style)
        ws.cell(2,1,1)._style=time_style
        for col in range(2,23):
            cell=ws.cell(2,col,0.0);cell._style=copy(value_style)
            cell.number_format="0.0000"
        ws.freeze_panes="B2"
        styles.append((ws["A2"]._style_id,ws["B2"]._style_id))
    buffer=BytesIO();wb.save(buffer)
    return buffer.getvalue(),styles

def write_book(run,path,rows,chunk):
    if path.exists():
        raise FileExistsError(f"Refusing to overwrite {path}")
    skeleton,styles=template_skeleton()
    with ZipFile(BytesIO(skeleton)) as source, ZipFile(path,"w",compression=ZIP_DEFLATED,
        compresslevel=9) as target:
        for item in source.infolist():
            match=re.fullmatch(r"xl/worksheets/sheet([12])\.xml",item.filename)
            if not match:
                target.writestr(item,source.read(item.filename))
                continue
            index=int(match[1])-1
            original=source.read(item.filename).decode("utf-8")
            prefix,rest=original.split("<sheetData>",1)
            data,suffix=rest.split("</sheetData>",1)
            header=data[:data.index("</row>")+len("</row>")]
            prefix=re.sub(r'<dimension ref="[~]+"'

```

```

        f'<dimension ref="A1:V{rows+1}"',prefix)
style_time,style_value=styles[index]
with target.open(item.filename,"w",force_zip64=True) as stream:
    stream.write((prefix+"<sheetData>" + header).encode("utf-8"))
    for start in range(1,rows+1,chunk):
        times=np.arange(start,min(rows+1,start+chunk))
        values=sample(run,times)[index]
        # Independent scalar production sampler checks vectorization.
        scalar=run.sample([times[0],times[-1]],RADII)[index]
        assert np.max(abs(values[[0,-1]]-scalar)) < 1e-12
        lines=[]
        for instant,row in zip(times,values):
            number=int(instant)+1
            cells=[f'<c r="A{number}" s="{style_time}" t="n"><v>{int(instant)}</v></c>']
            cells.extend(f'<c r="{col}{number}" s="{style_value}"
                t="n"><v>{float(v):.4f}</v></c>')
            for col,v in zip(COLS[1:],row)):
                lines.append(f'<row r="{number}">'+"".join(cells)+"</row>")
        stream.write("".join(lines).encode("utf-8"))
        if start == 1 or (start-1)//chunk % 16 == 0:
            print(json.dumps({"sheet":index+1,"written_through_s":int(times[-1])}),flush=True)
        stream.write(("</sheetData>" + suffix).encode("utf-8"))

def audit_book(path,rows,expected=None,old_prefix=False):
    """Full XML scan: all cells, row/time order, ranges, plus numerical samples."""
    book=load_workbook(path,read_only=True,data_only=True)
    assert book.sheetnames == [" 温度"," 水分浓度"]
    for ws in book:
        assert ws.max_row == rows+1 and ws.max_column == 22
        first=next(ws.iter_rows(min_row=1,max_row=1,values_only=True))
        assert np.allclose(first[1:],RADII)
    book.close()
    prefixbook=load_workbook(PROJECT/"results/result2.xlsx",read_only=True,data_only=True) if old_prefix
    else None
    output=[]
    with ZipFile(path) as archive:
        for index in (0,1):
            olditer=iter(prefixbook.worksheets[index].iter_rows(min_row=2,values_only=True)) if
            prefixbook else None
            count=0;minimum=float("inf");maximum=-float("inf");sample_count=0;prefix_count=0
            with archive.open(f"xl/worksheets/sheet{index+1}.xml") as stream:
                context=ET.iterparse(stream,events=("start","end"))
                _,root=next(context)
                for event,row in context:
                    if event != "end" or row.tag != NS+"row":
                        continue
                    number=int(row.attrib["r"])
                    if number == 1:
                        row.clear();root.clear();continue
                    count+=1
                    assert number == count+1
                    cells=list(row)
                    assert len(cells)==22
                    assert all(c.attrib["r"]==f"{col}{number}" and c.attrib.get("t","n")==="n"
                        for c,col in zip(cells,COLS))

```

```

        vals=[float(c.find(NS+"v").text) for c in cells]
        assert vals[0] == count
        assert np.isfinite(vals).all()
        low,high=min(vals[1:]),max(vals[1:])
        assert (27.99 <= low <= high <= 50.6) if index==0 else (0 < low <= high <= 2.5501)
        minimum=min(minimum,low);maximum=max(maximum,high)
        if expected is not None and count in expected:
            assert np.array_equal(np.asarray(vals[1:]),np.asarray(expected[count][index]))
            sample_count+=1
        if olditer is not None and count <= min(rows,10800):
            assert tuple(vals)==tuple(next(olditer))
            prefix_count+=1
        row.clear();root.clear()

    assert count==rows
    assert expected is None or sample_count==len(expected)
    output.append(dict(sheet=[" 温度", " 水分浓度"][index],data_rows=count,
                        columns=22,minimum=minimum,maximum=maximum,
                        sampled_rows_verified=sample_count,
                        old_3h_rows_identical=prefix_count))

if prefixbook:prefixbook.close()
return output

def export_run(run, out, *, rows=None, chunk=4096):
    """Export the supplied, unrounded q3 solution; no dependency on old outputs."""
    start=time.perf_counter(); out=Path(out).resolve()
    if PROJECT not in out.parents or chunk < 1:
        raise ValueError('Output must be below project root; positive chunk required')
    name='result2.xlsx'
    if rows is None:
        rows=int(run.end)
    if not 1 <= rows <= int(run.end):
        raise ValueError('Invalid export horizon')
    stamps=np.unique(np.r_[np.linspace(1,rows,101).astype(int),
                           [t for t in (1,60,600,1800,10800,14400,14401,rows) if t<=rows]])
    st,sc=sample(run,stamps)
    expected={int(t):[[float(f"{v:.4f}") for v in tt],[float(f"{v:.4f}") for v in cc]]
              for t,tt,cc in zip(stamps,st,sc)}
    out.mkdir(parents=True,exist_ok=True)
    write_book(run,out/name,rows,chunk)
    audit=audit_book(out/name,rows,expected)
    report=dict(status="PASS",mode="full" if rows==int(run.end) else "prefix",rows=rows,
               first_time_s=1,last_time_s=rows,radial_positions_cm=RADII.tolist(),
               production_summary=run.summary(sample_step_s=1),samples=expected,
               audit=audit,output_bytes=(out/name).stat().st_size,output_sha256=sha(out/name),
               exporter_sha256=sha(Path(__file__)),runtime_s=time.perf_counter()-start,
               diagnostic_note='Statistics include t=0 and all solver nodes; Excel starts at 1 s and
                               uses 21 output radii.')
    (out/"result2_export.json").write_text(
        json.dumps(report,ensure_ascii=False,indent=2),encoding="utf-8")
    return dict(path=name,rows=rows,columns=22,bytes=report['output_bytes'])

def main():
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--verify-only',action='store_true',required=True)

```

```

parser.add_argument('--output-dir',type=Path,default=HERE/'results')
args=parser.parse_args()
report=json.loads((args.output_dir/'result2_export.json').read_text())
assert sha(args.output_dir/'result2.xlsx')==report['output_sha256']
audit=audit_book(args.output_dir/'result2.xlsx',report['rows'],
                  expected={int(k):v for k,v in report['samples'].items()})
print(json.dumps(dict(status='PASS',audit=audit),ensure_ascii=False,indent=2))
return 0

if __name__=="__main__":
    sys.exit(main())

```

B.6 隔离完整复现入口

"""Reproduce all four questions in a fresh local directory; never overwrite.

Run from this project: python3 -B reproduce.py

Use --smoke only for the 600-second integration/export regression tests.

```

"""
from pathlib import Path
import argparse
import hashlib
import json
import os
import shutil
import subprocess
import sys
import tempfile
import time

ROOT=Path(__file__).resolve().parent
SOURCES=('solve.py','run_all.py','make_figures.py','verify_moving_domain.py',
        'export_result2_full.py','test_sync.py','reproduce.py','requirements.txt',
        'A 题.pdf','题目分析报告.md','术语表格.md')

def main():
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--smoke',action='store_true')
    parser.add_argument('--compute-only',action='store_true')
    args=parser.parse_args()
    start=time.perf_counter()
    out=Path(tempfile.mkdtemp(prefix='reproduced_',dir=ROOT))
    for name in SOURCES:
        shutil.copy2(ROOT/name,out/name)
    for name in ('附件','utils','assets'):
        shutil.copytree(ROOT/name,out/name,ignore=shutil.ignore_patterns('__pycache__', '*.pyc'))
    env=dict(os.environ,PYTHONDONTWRITEBYTECODE='1',OPENBLAS_NUM_THREADS='1',OMP_NUM_THREADS='1')
    if env.get('PYTHONPATH'):
        env['PYTHONPATH']=os.pathsep.join(str(Path(p or '.').resolve())
                                           for p in env['PYTHONPATH'].split(os.pathsep))
    print(f'Reproduction directory: {out}',flush=True)
    command=[sys.executable,'-B','test_sync.py'] if args.smoke else
    [sys.executable,'-B','run_all.py','--q2-horizon','full']
    if args.compute_only and not args.smoke:

```

```

        command.append('--compute-only')
subprocess.run(command,cwd=out,env=env,check=True)
from utils.repro_manifest import build_manifest
inputs=([ROOT/n for n in SOURCES]+sorted((ROOT/'附件').rglob('*.xlsx'))
        +sorted((ROOT/'utils').glob('*.py'))+[p for p in sorted((ROOT/'assets').rglob('*')) if
        p.is_file()])
parameters=(dict(n=80,rtol=1e-8,horizon_s=600,smoke=True)
            if args.smoke else dict(n=json.loads((out/'results/summary.json').read_text()))['grid_n'],
            rtol=1e-8,q2_horizon='full',smoke=False,figures=not
            args.compute_only))
suffix=' --smoke' if args.smoke else ' --compute-only' if args.compute_only else ''
manifest=build_manifest(inputs,20260910,parameters,
                        'OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 python3 -B reproduce.py'+suffix,
                        ['numpy','scipy','pandas','openpyxl','matplotlib'])
manifest['runtime_s']=time.perf_counter()-start
manifest['outputs']=[dict(path=str(p.relative_to(out)),bytes=p.stat().st_size,
                        sha256=hashlib.sha256(p.read_bytes()).hexdigest())
                    for folder in ('results','figures') if (out/folder).exists()
                    for p in sorted((out/folder).rglob('*'))
                    if p.is_file() and p.name != '复现清单.json')]
(out/'results').mkdir(exist_ok=True)
(out/'results/复现清单.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps(dict(status='PASS',output_directory=str(out),runtime_s=manifest['runtime_s']),ensure
_ascii=False),flush=True)

if __name__=='__main__':
    main()

```

B.7 诊断采样和导出回归

```

"""Small real-input regression of diagnostic scope and streamed output."""
from pathlib import Path
import json
import tempfile
import numpy as np
from solve import integrate, read_inputs
from export_result2_full import export_run, audit_book

def main():
    arrays,_=read_inputs()
    rows=[]
    for q in (1,3,4):
        run=integrate(q,80,arrays,horizon=600,stop_at_threshold=False)
        sparse=run.summary()
        dense=run.summary(sample_step_s=1,chunk_size=37)
        assert sparse['diagnostic_sampling']['time_count']==250
        assert dense['diagnostic_sampling']['time_count']==601
        assert not dense['diagnostic_sampling']['continuous_extrema_claimed']
        assert not {'T_max','T_min','C_max','C_min','moisture_balance_max','max_above_center'} &
        dense.keys()
        y=run.values(np.arange(601));m=81
        expected={'T_max_sampled':y[:m].max(),'T_min_sampled':y[:m].min(),
                  'C_max_sampled':y[m:2*m].max(),'C_min_sampled':y[m:2*m].min(),

```

```

        'moisture_balance_max_sampled': np.max(np.abs(2*run.model.grid.w@y[m:2*m]+y[-1]-2.55)))
    for key,value in expected.items():
        assert abs(dense[key]-value)<1e-12,(q,key,dense[key],value)
    assert dense['moisture_balance_max_sampled']<1e-7
    for bad in (0,-1,float('nan'),float('inf')):
        try:run.summary(sample_step_s=bad)
        except ValueError:pass
        else:raise AssertionError('Invalid step accepted')
    rows.append(dict(question=q,diagnostics=dense))
    if q==3:
        out=Path(tempfile.mkdtemp(prefix='smoke_output_',dir=Path(__file__).resolve().parent))
        export_run(run,out,chunk=137)
        audit=audit_book(out/'result2.xlsx',600)
        assert len(audit)==2 and all(a['data_rows']==600 for a in audit)
    print(json.dumps(dict(status='PASS',runs=rows),ensure_ascii=False,indent=2))

if __name__=='__main__':
    main()

```

B.8 完整附件读取

```

#!/usr/bin/env python3
""" 无隐式表头推断地读取 XLSX，并可断言数据行数。"""

from pathlib import Path

from openpyxl import load_workbook

def read_excel_rows(path, sheet=0, *, header=False, expected_rows=None):
    workbook = load_workbook(Path(path), read_only=True, data_only=True)
    worksheet = workbook.worksheets[sheet] if isinstance(sheet, int) else workbook[sheet]
    rows = [tuple(row) for row in worksheet.iter_rows(values_only=True)]
    while rows and not any(value is not None for value in rows[-1]):
        rows.pop()
    workbook.close()

    if header:
        if not rows:
            return []
        names = rows.pop(0)
        rows = [dict(zip(names, row)) for row in rows]
    if expected_rows is not None and len(rows) != expected_rows:
        raise ValueError(f" 工作表实际读取 {len(rows)} 行，期望 {expected_rows} 行")
    return rows

```

B.9 图表结构与布局检查

```

#!/usr/bin/env python3
""" 数学建模论文图的一致样式工具（向后兼容层）。

本文件保留原有 API 以兼容 ``from utils.plot_style import ...`` 的外部代码。
实际常量和工具函数已迁移到 tools/figure/scripts/style_constants.py。
布局审计和导出功能由 tools/figure/scripts/visual_qa.py 和

```

```

tools/figure/scripts/export_figure.py 替代。
"""

from __future__ import annotations

import logging
import unicodedata
import warnings
from pathlib import Path
from typing import Sequence

# -----
# SKILL_ROOT 定位
# -----

def _is_math_modeling_skill_root(path: Path) -> bool:
    return (
        (path / "SKILL.md").is_file()
        and (path / "VERSION").is_file()
        and (
            path
            / "references"
            / "roles"
            / " 编程手"
            / "scripts"
            / "plot_style.py"
        ).is_file()
    )

def _find_skill_root(path: Path) -> Path | None:
    directory = path.resolve().parent
    return next(
        (candidate for candidate in (directory, *directory.parents)
         if _is_math_modeling_skill_root(candidate)),
        None,
    )

SKILL_ROOT = _find_skill_root(Path(__file__))

# -----
# 色盲安全配色
# -----

PALETTE = {
    "primary": "#0072B2",
    "secondary": "#E69F00",
    "positive": "#009E73",
    "contrast": "#D55E00",
    "accent": "#CC79A7",
    "sky": "#56B4E9",
    "neutral": "#6B7280",
    "dark": "#222222",
}

```

```

COLOR_SEQUENCE = tuple(PALETTE[name] for name in (
    "primary",
    "secondary",
    "positive",
    "contrast",
    "accent",
    "sky",
    "neutral",
))

# -----
# 论文栏宽 (英寸)
# -----

WIDTHS_IN = {
    "single": 3.5,
    "double": 7.2,
    "report": 6.3,
}

# -----
# 字体选择
# -----

_CJK_SANS = (
    "Noto Sans CJK SC",
    "Source Han Sans SC",
    "Microsoft YaHei",
    "SimHei",
    "PingFang SC",
)

def _available_fonts() -> set[str]:
    from matplotlib import font_manager
    return {item.name for item in font_manager.fontManager.ttflist}

def choose_font(language: str = "zh") -> str:
    """ 选择可用字体; 中文字体缺失时给出警告并安全回退。"""
    if language not in {"zh", "en"}:
        raise ValueError("language 只能是 'zh' 或 'en'")
    available = _available_fonts()
    if language == "zh":
        for name in _CJK_SANS:
            if name in available:
                return name
        warnings.warn(
            " 未检测到常用中文字体, 已回退到 DejaVu Sans; 导出前必须检查中文和特殊符号是否缺字。",
            RuntimeWarning,
            stacklevel=2,
        )
    for name in ("Arial", "Helvetica", "DejaVu Sans"):
        if name in available:
            return name

```

```

        return "DejaVu Sans"

# -----
# 尺寸计算
# -----

def figure_size(width: str = "report", aspect: float = 0.62) -> tuple[float, float]:
    """ 按最终使用宽度返回英寸尺寸，避免在论文中二次大幅缩放。 """
    if width not in WIDTHS_IN:
        raise ValueError(f" 未知宽度方案: {width}")
    if aspect <= 0:
        raise ValueError("aspect 必须大于 0")
    width_in = WIDTHS_IN[width]
    return width_in, width_in * aspect

# -----
# 子图创建
# -----

def publication_subplots(
    nrows: int = 1,
    ncols: int = 1,
    *,
    width: str = "report",
    aspect: float = 0.62,
    width_ratios: Sequence[float] | None = None,
    height_ratios: Sequence[float] | None = None,
    squeeze: bool = True,
):
    """ 按最终尺寸创建子图，并允许显式声明主次面板比例。 """
    import matplotlib.pyplot as plt

    if nrows < 1 or ncols < 1:
        raise ValueError("nrows 和 ncols 必须大于 0")
    if width_ratios is not None and len(width_ratios) != ncols:
        raise ValueError("width_ratios 数量必须与 ncols 一致")
    if height_ratios is not None and len(height_ratios) != nrows:
        raise ValueError("height_ratios 数量必须与 nrows 一致")
    gridspec_kw = {}
    if width_ratios is not None:
        gridspec_kw["width_ratios"] = list(width_ratios)
    if height_ratios is not None:
        gridspec_kw["height_ratios"] = list(height_ratios)
    return plt.subplots(
        nrows,
        ncols,
        figsize=figure_size(width, aspect),
        layout="constrained",
        gridspec_kw=gridspec_kw or None,
        squeeze=squeeze,
    )

# -----

```

```

# 面板标签
# -----

from collections.abc import Iterable as _Iterable

def add_panel_labels(
    axes: _Iterable,
    labels: Sequence[str] | None = None,
    *,
    x_offset_pt: float = -8.0,
    y_offset_pt: float = 1.0,
) -> None:
    """ 在各面板左上外侧添加小写粗体编号。 """
    axes_list = list(axes)
    panel_labels = list(labels) if labels is not None else [chr(97 + i) for i in range(len(axes_list))]
    if len(panel_labels) != len(axes_list):
        raise ValueError("labels 数量必须与 axes 数量一致")
    for axis, label in zip(axes_list, panel_labels):
        axis.annotate(
            label,
            xy=(0, 1),
            xycoords="axes fraction",
            xytext=(x_offset_pt, y_offset_pt),
            textcoords="offset points",
            ha="right",
            va="bottom",
            fontsize=8,
            fontweight="bold",
            annotation_clip=False,
        )

# -----
# 路径安全
# -----

def resolve_output_stem(output_stem: str | Path) -> Path:
    """ 解析导出路径，并禁止把任务产物写回 Skill 目录。 """
    stem = Path(output_stem).expanduser().resolve()
    if stem.suffix.lower() in {".svg", ".png", ".pdf"}:
        stem = stem.with_suffix("")
    if any(
        _is_math_modeling_skill_root(candidate)
        for candidate in (stem.parent, *stem.parent.parents)
    ):
        raise ValueError(" 图形产物必须写入 PROJECT_ROOT，不能写入 SKILL_ROOT")
    return stem

# -----
# 向后兼容：已被 tools/figure/ 新工具替代的函数
# 以下函数保留以兼容现有测试和外部代码。新代码请使用
# tools/figure/scripts/ 下的 setup_style、visual_qa、export_figure。
# -----

```

```

def apply_publication_style(language: str = "zh", width: str = "report") -> dict:
    """ 应用出版样式基线（向后兼容）。新代码请用 setup_style()。 """
    import matplotlib as mpl
    from cycler import cycler

    font_name = choose_font(language)
    size = figure_size(width)
    mpl.rcParams.update({
        "figure.figsize": size,
        "figure.constrained_layout.use": True,
        "figure.facecolor": "white",
        "axes.facecolor": "white",
        "font.family": "sans-serif",
        "font.sans-serif": [font_name, "Arial", "Helvetica", "DejaVu Sans"],
        "font.size": 7.5,
        "axes.titlesize": 8.0,
        "axes.titleweight": "normal",
        "axes.titlelocation": "left",
        "axes.titlepad": 4.0,
        "axes.labelsiz": 8.0,
        "xtick.labelsize": 7.0,
        "ytick.labelsize": 7.0,
        "legend.fontsize": 7.0,
        "axes.linewidth": 0.7,
        "lines.linewidth": 1.1,
        "lines.markersize": 3.5,
        "patch.linewidth": 0.6,
        "hatch.linewidth": 0.4,
        "xtick.major.size": 3.0,
        "ytick.major.size": 3.0,
        "xtick.major.width": 0.6,
        "ytick.major.width": 0.6,
        "axes.spines.top": False,
        "axes.spines.right": False,
        "axes.grid": False,
        "legend.frameon": False,
        "legend.handlelength": 1.6,
        "legend.borderaxespad": 0.4,
        "axes.unicode_minus": False,
        "axes.prop_cycle": cycler(color=COLOR_SEQUENCE),
        "svg.fonttype": "none",
        "pdf.fonttype": 42,
        "savefig.facecolor": "white",
        "savefig.dpi": 300,
    })
    return {"font": font_name, "size_inches": size, "colors": COLOR_SEQUENCE}

class _GlyphHandler(logging.Handler):
    def __init__(self) -> None:
        super().__init__()
        self.messages: list[str] = []

    def emit(self, record) -> None:
        message = record.getMessage()
        if "Glyph" in message and "missing from font" in message:

```

```

        self.messages.append(message)

def _labels_overlap(labels, renderer, axis: str) -> bool:
    boxes = [
        label.get_window_extent(renderer)
        for label in labels
        if label.get_visible() and label.get_text().strip()
    ]
    boxes.sort(key=(lambda box: box.x0) if axis == "x" else (lambda box: box.y0))
    if axis == "x":
        return any(left.x1 > right.x0 + 1 for left, right in zip(boxes, boxes[1:]))
    return any(lower.y1 > upper.y0 + 1 for lower, upper in zip(boxes, boxes[1:]))

def audit_layout(fig) -> list[str]:
    """ 在导出前检查缺字、画布外文字和相邻刻度重叠（向后兼容）。 """
    import matplotlib.text as mtext

    handler = _GlyphHandler()
    logger = logging.getLogger("matplotlib")
    logger.addHandler(handler)
    try:
        with warnings.catch_warnings(record=True) as caught:
            warnings.simplefilter("always")
            fig.canvas.draw()
    finally:
        logger.removeHandler(handler)

    messages = handler.messages + [
        str(item.message)
        for item in caught
        if "Glyph" in str(item.message) and "missing from font" in str(item.message)
    ]
    issues = [f" 缺字: {message}" for message in dict.fromkeys(messages)]
    renderer = fig.canvas.get_renderer()
    width, height = fig.bbox.width, fig.bbox.height
    tick_ids = {
        id(label)
        for axis in fig.axes
        for label in (*axis.get_xticklabels(), *axis.get_yticklabels())
    }
    clipped = []
    for text in fig.findobj(mtext.Text):
        if id(text) in tick_ids or not text.get_visible() or not text.get_text().strip():
            continue
        box = text.get_window_extent(renderer)
        if box.x0 < -1 or box.y0 < -1 or box.x1 > width + 1 or box.y1 > height + 1:
            clipped.append(text.get_text().replace("\n", " ")[:24])
    if clipped:
        issues.append(" 文字可能超出画布: " + "、".join(dict.fromkeys(clipped)))
    for index, axis in enumerate(fig.axes, start=1):
        if _labels_overlap(axis.get_xticklabels(), renderer, "x"):
            issues.append(f" 第 {index} 个坐标轴的 x 刻度标签重叠")
        if _labels_overlap(axis.get_yticklabels(), renderer, "y"):
            issues.append(f" 第 {index} 个坐标轴的 y 刻度标签重叠")

```

```

return issues

def _display_width(text: str) -> int:
    return sum(
        2 if unicodedata.east_asian_width(character) in {"W", "F"} else 1
        for character in text
    )

def _is_colorbar_axis(axis) -> bool:
    return axis.get_label() == "<colorbar>" or hasattr(axis, "_colorbar")

def audit_design(fig) -> list[str]:
    """ 检查可由对象结构确定的高风险设计（向后兼容）。"""
    from matplotlib.container import BarContainer

    issues: list[str] = []
    data_axes = [axis for axis in fig.axes if not _is_colorbar_axis(axis)]
    colorbar_axes = [axis for axis in fig.axes if _is_colorbar_axis(axis)]
    colorbar_mappables = {
        id(axis._colorbar.mappable)
        for axis in colorbar_axes
        if hasattr(axis, "_colorbar") and getattr(axis._colorbar, "mappable", None) is not None
    }
    for index, axis in enumerate(data_axes, start=1):
        for location in ("left", "center", "right"):
            title = axis.get_title(loc=location).strip()
            title_lines = title.splitlines()
            if title and (
                len(title_lines) > 1
                or max(_display_width(line) for line in title_lines) > 28
            ):
                issues.append(
                    f" 第 {index} 个坐标轴标题过长；完整论述应移到图注，面板内只保留短标题"
                )
                break

        legend = axis.get_legend()
        if legend is not None and len(legend.get_texts()) > 5:
            issues.append(
                f" 第 {index} 个坐标轴图例超过 5 项；应直接标注、改为共享图例或拆分证据"
            )

        for line in axis.lines:
            marker = line.get_marker()
            if marker in {None, "", " ", "None", "none"}:
                continue
            point_count = len(line.get_xdata(orig=False))
            if point_count > 25 and line.get_markevery() is None:
                issues.append(
                    f" 第 {index} 个坐标轴对 {point_count} 个点逐点绘制标记；"
                    " 应取消标记或设置 markevery"
                )
            break

```

```

for container in axis.containers:
    if not isinstance(container, BarContainer) or not container.patches:
        continue
    if container.orientation == "vertical":
        lower, upper = axis.get_ylim()
    else:
        lower, upper = axis.get_xlim()
    tolerance = max(abs(upper - lower), 1.0) * 1e-9
    if lower > tolerance or upper < -tolerance:
        issues.append(
            f" 第 {index} 个坐标轴的柱状图未从零开始; "
            " 若必须截断, 应改用点图/区间图并显式说明"
        )
        break

for image in axis.images:
    values = image.get_array()
    if (
        getattr(values, "size", 0) <= 4
        and len(axis.texts) >= getattr(values, "size", 0)
        and id(image) in colorbar_mappables
    ):
        issues.append(
            f" 第 {index} 个坐标轴是已标数值的 2x2 小矩阵, "
            " 不应再使用冗余 colorbar"
        )
        break

for legend in fig.legends:
    if len(legend.get_texts()) > 5:
        issues.append(" 整图共享图例超过 5 项; 应直接标注、拆分证据或突出核心系列")
        break

return issues


def _save_grayscale_preview(png_path: Path, dpi: int) -> Path:
    import matplotlib.image as image_io
    import numpy as np

    pixels = image_io.imread(png_path)
    rgb = pixels[..., :3]
    if pixels.shape[-1] == 4:
        alpha = pixels[..., 3:4]
        rgb = rgb * alpha + (1 - alpha)
    grayscale = np.dot(rgb, (0.2126, 0.7152, 0.0722))
    output = png_path.parent / "_qa" / f"{png_path.stem}_grayscale.png"
    output.parent.mkdir(parents=True, exist_ok=True)
    image_io.imsave(output, grayscale, cmap="gray", vmin=0, vmax=1, dpi=dpi)
    return output


def export_figure(
    fig,
    output_stem: str | Path,
    *,
    dpi: int = 300,

```

```

    grayscale_preview: bool = True,
    strict_layout: bool = True,
    strict_design: bool = True,
) -> dict[str, str]:
    """ 按固定物理尺寸导出（向后兼容）。新代码请用 tools/figure/scripts/export_figure.py。 """
    if dpi < 300:
        raise ValueError(" 论文图 PNG 的 dpi 不能低于 300")
    layout_issues = audit_layout(fig)
    if strict_layout and layout_issues:
        raise ValueError(" 版面预检未通过: " + "; ".join(layout_issues))
    design_issues = audit_design(fig)
    if strict_design and design_issues:
        raise ValueError(" 出版设计预检未通过: " + "; ".join(design_issues))
    stem = resolve_output_stem(output_stem)
    stem.parent.mkdir(parents=True, exist_ok=True)
    svg_path = stem.with_suffix(".svg")
    png_path = stem.with_suffix(".png")
    fig.savefig(svg_path)
    fig.savefig(png_path, dpi=dpi)
    outputs = {"svg": str(svg_path), "png": str(png_path)}
    if grayscale_preview:
        outputs["grayscale"] = str(_save_grayscale_preview(png_path, dpi))
    return outputs

__all__ = [
    "COLOR_SEQUENCE",
    "PALETTE",
    "WIDTHS_IN",
    "SKILL_ROOT",
    "add_panel_labels",
    "apply_publication_style",
    "audit_design",
    "audit_layout",
    "choose_font",
    "export_figure",
    "figure_size",
    "publication_subplots",
    "resolve_output_stem",
]

```

B.10 统一科研绘图风格

```

"""
scipilot-figure-skill :: setup_style.py
=====
Publication-grade matplotlib / seaborn style configuration.

```

应用出版级样式预设。支持 nature / ieee / science / general 四种期刊预设，支持中英文（lang='zh'/'en'），中文模式按优先级自动查找 Noto Sans CJK SC > Source Han Sans SC > SimHei > Microsoft YaHei 并修正负号渲染。SciencePlots 可选——装了就用，没装回退到内置等效预设。

Usage

```
from setup_style import setup_style
```

```

# Nature 单栏英文图
setup_style(journal='nature', lang='en')

# 中文期刊通用
setup_style(journal='general', lang='zh')

# 关闭 SciencePlots 强制用内置预设
setup_style(journal='ieee', use_sciplots=False)

CLI: ``python setup_style.py --list-fonts`` 列出可用 CJK 字体。
"""

from __future__ import annotations

import argparse
import sys
import warnings

import matplotlib
import matplotlib.font_manager as fm
import matplotlib.pyplot as plt

# 期刊预设: figsize 单位为英寸, 对应该期刊单栏标称宽度
# Nature: 89mm = 3.5 in 双栏: 183mm = 7.2 in
# Science: 与 Nature 接近
# IEEE: 单栏 3.5 in, 双栏 7.16 in
JOURNAL_PRESETS = {
    "nature": {
        "figure.figsize": (3.5, 2.625), # 4:3 ratio @ 89mm
        "figure.dpi": 150,
        "savefig.dpi": 300,
        "font.family": "sans-serif",
        "font.sans-serif": ["Helvetica", "Arial", "DejaVu Sans"],
        "font.size": 7,
        "axes.labelsize": 8,
        "axes.titlesize": 8,
        "xtick.labelsize": 7,
        "ytick.labelsize": 7,
        "legend.fontsize": 7,
        "lines.linewidth": 1.0,
        "lines.markersize": 4,
        "axes.linewidth": 0.6,
        "xtick.major.width": 0.6,
        "ytick.major.width": 0.6,
        "xtick.minor.width": 0.4,
        "ytick.minor.width": 0.4,
        "axes.spines.top": False,
        "axes.spines.right": False,
        "pdf.fonttype": 42,
        "ps.fonttype": 42,
        "svg.fonttype": "none",
    },
    "science": {
        "figure.figsize": (3.5, 2.625),
        "figure.dpi": 150,

```

```

    "savefig.dpi": 300,
    "font.family": "sans-serif",
    "font.sans-serif": ["Helvetica", "Arial", "DejaVu Sans"],
    "font.size": 7,
    "axes.labelsize": 7,
    "axes.titlesize": 8,
    "xtick.labelsize": 7,
    "ytick.labelsize": 7,
    "legend.fontsize": 6,
    "lines.linewidth": 1.0,
    "lines.markersize": 4,
    "axes.linewidth": 0.6,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "pdf.fonttype": 42,
    "ps.fonttype": 42,
    "svg.fonttype": "none",
},
"ieee": {
    "figure.figsize": (3.5, 2.5),
    "figure.dpi": 150,
    "savefig.dpi": 600,
    "font.family": "serif",
    "font.serif": ["Times New Roman", "Times", "DejaVu Serif"],
    "font.size": 8,
    "axes.labelsize": 8,
    "axes.titlesize": 9,
    "xtick.labelsize": 7,
    "ytick.labelsize": 7,
    "legend.fontsize": 7,
    "lines.linewidth": 1.0,
    "lines.markersize": 4,
    "axes.linewidth": 0.7,
    "axes.grid": False,
    "axes.spines.top": True,
    "axes.spines.right": True,
    "pdf.fonttype": 42,
    "ps.fonttype": 42,
    "svg.fonttype": "none",
},
"general": {
    "figure.figsize": (5.0, 3.5),
    "figure.dpi": 150,
    "savefig.dpi": 300,
    "font.family": "sans-serif",
    "font.sans-serif": ["Arial", "Helvetica", "DejaVu Sans"],
    "font.size": 9,
    "axes.labelsize": 10,
    "axes.titlesize": 11,
    "xtick.labelsize": 9,
    "ytick.labelsize": 9,
    "legend.fontsize": 9,
    "lines.linewidth": 1.2,
    "lines.markersize": 5,
    "axes.linewidth": 0.8,
    "axes.spines.top": False,

```

```

        "axes.spines.right": False,
        "pdf.fonttype": 42,
        "ps.fonttype": 42,
        "svg.fonttype": "none",
    },
}

```

中文字体优先级列表（按可用性 + 期刊接受度排序）

```

CJK_FONT_PRIORITY = [
    "Noto Sans CJK SC",
    "Noto Sans SC",
    "Source Han Sans SC",
    "Source Han Sans CN",
    "SimHei",
    "Microsoft YaHei",
    "PingFang SC",
    "Heiti SC",
    "WenQuanYi Zen Hei",
    "Arial Unicode MS",
]

```

```

CJK_SERIF_PRIORITY = [
    "Noto Serif CJK SC",
    "Noto Serif SC",
    "Source Han Serif SC",
    "Source Han Serif CN",
    "SimSun",
    "STSong",
    "Songti SC",
]

```

CJK_INSTALL_HINT = ""\n

找不到任何中文字体。请安装 Noto CJK 字体之一：

```

Linux:    sudo apt install fonts-noto-cjk    # Debian/Ubuntu
          sudo dnf install google-noto-sans-cjk-fonts # Fedora/RHEL
macOS:    brew install --cask font-noto-sans-cjk-sc
          或下载: https://github.com/notofonts/noto-cjk/releases
Windows:  下载 https://github.com/notofonts/noto-cjk/releases
          解压后右键 .ttf/.otf 文件 -> " 为所有用户安装"

```

或者列出当前已安装的中文字体：

```

python setup_style.py --list-fonts
"""

```

```

def _available_fonts() -> set[str]:
    """ 返回 matplotlib 已索引的全部字体名集合。"""
    return {f.name for f in fm.fontManager.ttflist}

```

```

def list_cjk_fonts() -> list[str]:
    """ 返回系统上可用的 CJK 字体（按优先级排序）。"""
    available = _available_fonts()
    hits = []
    for f in CJK_FONT_PRIORITY + CJK_SERIF_PRIORITY:

```

```

        if f in available and f not in hits:
            hits.append(f)
# 额外做一次包含中文关键词的扫描, 捕获非标准命名的中文字体
for f in available:
    lower = f.lower()
    if any(k in lower for k in ("cjk", "han", "songti", "yahei", "simhei", "simsun")):
        if f not in hits:
            hits.append(f)
return hits

def configure_chinese_fonts(serif_for_zh: bool = False) -> str:
    """
    自动检测并配置中文字体; 同时修正负号渲染。

    Args:
        serif_for_zh: True 时优先选用衬线中文字体 (宋体类), 用于中文期刊
            " 宋体正文 + Times New Roman 数字" 的混排约定。
    Returns:
        实际选用的中文字体名。
    Raises:
        RuntimeError: 系统未安装任何识别到的中文字体。
    """
    available = _available_fonts()
    priority = CJK_SERIF_PRIORITY + CJK_FONT_PRIORITY if serif_for_zh else CJK_FONT_PRIORITY

    chosen = None
    for f in priority:
        if f in available:
            chosen = f
            break

    if chosen is None:
        # 兜底: 扫描包含 cjk/han 等关键字的字体
        for f in available:
            lower = f.lower()
            if any(k in lower for k in ("cjk", "han", "song", "hei", "yahei", "kaiti")):
                chosen = f
                break

    if chosen is None:
        raise RuntimeError(CJK_INSTALL_HINT)

    # 中文期刊常要求中文 + Times New Roman 混排: 中文走中文字体, 西文走 Times
    plt.rcParams["font.family"] = ["sans-serif"] if not serif_for_zh else ["serif"]
    if serif_for_zh:
        plt.rcParams["font.serif"] = [chosen, "Times New Roman", "Times", "DejaVu Serif"]
    else:
        plt.rcParams["font.sans-serif"] = [chosen, "Arial", "Helvetica", "DejaVu Sans"]
    # 修正负号 unicode minus 在某些中文字体里渲染成方框的问题
    plt.rcParams["axes.unicode_minus"] = False
    return chosen

def _try_sciencplots(journal: str) -> bool:
    """If SciencePlots is installed, apply its style stack; otherwise return False."""

```

```

try:
    import scienceplots # noqa: F401
except ImportError:
    return False

# SciencePlots 风格栈: 基础 + 期刊变体
stack = ["science"]
if journal == "nature":
    stack.append("nature")
elif journal == "ieee":
    stack.append("ieee")
# 关掉 LaTeX 渲染避免环境缺 LaTeX 时崩溃; 中文模式必须关
stack.append("no-latex")
try:
    plt.style.use(stack)
    return True
except OSError as e:
    warnings.warn(f"SciencePlots style stack failed: {e}; fallback to builtin.")
    return False

def setup_style(
    journal: str = "general",
    lang: str = "en",
    use_sciplots: bool = True,
    serif_for_zh: bool = False,
    constrained_layout: bool = True,
) -> dict:
    """
    应用出版级样式预设。

    Args:
        journal: 'nature' | 'science' | 'ieee' | 'general'
        lang: 'en' | 'zh' —中文模式自动配置中文字体并修正负号
        use_sciplots: 优先尝试 SciencePlots; 不可用则回退到内置预设
        serif_for_zh: 中文模式下使用宋体类衬线字体 (中文期刊常约定)
        constrained_layout: 默认 True——全局开启 constrained_layout 自适应排版,
            从源头减少标题/轴标签被裁、图例压数据、子图互相重叠。需要手动
            subplots_adjust 或某些 colorbar 写法时可传 False 关闭。

    Returns:
        dict 包含 keys: journal / lang / sciplots_used / cjk_font / constrained_layout
    """
    if journal not in JOURNAL_PRESETS:
        raise ValueError(f"Unknown journal preset: {journal}. "
                        f"Choose from {sorted(JOURNAL_PRESETS)}")

    sciplots_used = False
    if use_sciplots:
        sciplots_used = _try_sciencplots(journal)

    # 内置预设始终在 SciencePlots 之上覆盖一遍, 确保关键参数 (fonttype、字号) 落实
    plt.rcParams.update(JOURNAL_PRESETS[journal])

    # 默认开启自适应排版: 从源头减少文字遮盖 / 裁切 / 子图重叠
    plt.rcParams["figure.constrained_layout.use"] = constrained_layout

```

```

# 全模式默认修正负号：避免所选字体缺 U+2212 时负号渲染成方框（一种乱码）。
# 用 ASCII hyphen-minus 代替真减号，几乎所有字体都含，最稳妥。
plt.rcParams["axes.unicode_minus"] = False

cjk_font = None
if lang == "zh":
    cjk_font = configure_chinese_fonts(serif_for_zh=serif_for_zh)
elif lang != "en":
    raise ValueError(f"lang must be 'en' or 'zh', got {lang!r}")

return {
    "journal": journal,
    "lang": lang,
    "sciplots_used": sciplots_used,
    "cjk_font": cjk_font,
    "constrained_layout": constrained_layout,
}

def _cli() -> int:
    p = argparse.ArgumentParser(description="scipilot-figure-skill style setup")
    p.add_argument("--list-fonts", action="store_true",
                    help=" 列出当前系统上可用的 CJK 字体")
    p.add_argument("--test", action="store_true",
                    help=" 应用预设并打印 rcParams 关键值")
    p.add_argument("--journal", default="general",
                    choices=list(JOURNAL_PRESETS))
    p.add_argument("--lang", default="en", choices=["en", "zh"])
    p.add_argument("--no-sciplots", action="store_true")
    p.add_argument("--serif-zh", action="store_true")
    args = p.parse_args()

    if args.list_fonts:
        fonts = list_cjk_fonts()
        if fonts:
            print(" 已检测到的 CJK 字体（按优先级排序）：")
            for f in fonts:
                print(f"    - {f}")
        else:
            print(" 未检测到任何 CJK 字体。")
            print(CJK_INSTALL_HINT)
        return 0

    if args.test:
        info = setup_style(journal=args.journal, lang=args.lang,
                           use_sciplots=not args.no_sciplots,
                           serif_for_zh=args.serif_zh)
        print(f"applied: {info}")
        for k in ("figure.figsize", "font.family", "font.size",
                  "axes.labelsize", "pdf.fonttype", "axes.unicode_minus"):
            print(f"    {k} = {plt.rcParams[k]!r}")
        return 0

    p.print_help()
    return 0

```

```
if __name__ == "__main__":
    sys.exit(_cli())
```

B.11 多格式图表导出

```
"""
scipilot-figure-skill :: export_figure.py
=====
Unified figure export to multiple formats at exact final size.

- Vector preferred: PDF / SVG / EPS for line/bar/scatter (lossless, journal-friendly).
- Raster for photos / micrographs: PNG / TIFF at >= 300 DPI; never JPEG for data figures.
- Embeds TrueType fonts (fonttype 42) so journals don't reject Type-3 PDFs.
- Optional grayscale preview to sanity-check colorblind safety.

Usage
-----

from export_figure import export_figure
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.plot([0,1,2],[3,1,4])

paths = export_figure(
    fig,
    basename="figs/fig1_main",
    formats=["pdf", "svg", "png"],
    size_inches=(3.5, 2.625),  # 强制成 Nature 单栏尺寸
    dpi=600,
    grayscale_preview=True,
)
# -> ['figs/fig1_main.pdf', 'figs/fig1_main.svg',
#      'figs/fig1_main.png', 'figs/fig1_main_grayscale.png']

CLI: ``python export_figure.py demo`` 生成一张演示图并多格式导出。
"""

from __future__ import annotations

import argparse
import os
import sys
from typing import Iterable

import matplotlib.pyplot as plt

VECTOR_FORMATS = {"pdf", "svg", "eps"}
RASTER_FORMATS = {"png", "tiff", "tif", "jpg", "jpeg"}
SUPPORTED_FORMATS = VECTOR_FORMATS | RASTER_FORMATS

def _ensure_parent(path: str) -> None:
    parent = os.path.dirname(os.path.abspath(path))
    if parent and not os.path.exists(parent):
```

```

    os.makedirs(parent, exist_ok=True)

def export_figure(
    fig,
    basename: str,
    formats: Iterable[str] | None = None,
    dpi: int = 300,
    size_inches: tuple[float, float] | None = None,
    grayscale_preview: bool = False,
    tight: bool = True,
    pad_inches: float = 0.05,
    transparent: bool = False,
) -> list[str]:
    """
    Export a matplotlib Figure to one or more formats at exact final size.

    Args:
        fig: matplotlib Figure object.
        basename: 输出路径前缀（不含扩展名）；可包含子目录，自动创建。
        formats: list/tuple of extensions, e.g. ['pdf', 'svg', 'png'].
            Default: ['pdf', 'svg', 'png'].
        dpi: raster 格式分辨率；建议 300（普通）/ 600（IEEE 等）。
        size_inches: (width, height) 英寸；指定后会 fig.set_size_inches() 强制
            最终尺寸。强烈建议传入——保证导出后不必在 Word/LaTeX 里二次缩放。
        grayscale_preview: 额外生成一张 _grayscale.png 供色盲安全检查。
        tight: 是否走 bbox_inches='tight'（裁掉留白）。
        pad_inches: tight 模式下保留的边距（英寸）。
        transparent: 透明背景（PPT/海报可能需要）。

    Returns:
        实际写出的文件路径列表。
    """
    if formats is None:
        formats = ("pdf", "svg", "png")
    formats = [f.lower().lstrip(".") for f in formats]
    unknown = [f for f in formats if f not in SUPPORTED_FORMATS]
    if unknown:
        raise ValueError(f"Unsupported formats: {unknown}. "
                        f"Supported: {sorted(SUPPORTED_FORMATS)}")

    if size_inches is not None:
        if len(size_inches) != 2:
            raise ValueError("size_inches must be (width, height)")
        fig.set_size_inches(*size_inches)

    # 强制嵌入 TrueType 字体 (fonttype 42)；多家期刊明确拒绝 Type-3 PDF。
    plt.rcParams["pdf.fonttype"] = 42
    plt.rcParams["ps.fonttype"] = 42
    plt.rcParams["svg.fonttype"] = "none" # 文本仍可编辑，期刊通常更欢迎

    saved: list[str] = []
    for fmt in formats:
        if fmt in {"jpg", "jpeg"}:
            print(f"[scipilot-figure-skill] WARNING: skipping {fmt} — "
                  "JPEG is lossy and unsuitable for line/text figures.",

```

```

        file=sys.stderr)
    continue
    path = f"{basename}.{fmt}"
    _ensure_parent(path)
    kwargs: dict = {
        "bbox_inches": "tight" if tight else None,
        "pad_inches": pad_inches,
        "transparent": transparent,
    }
    if fmt in RASTER_FORMATS:
        kwargs["dpi"] = dpi
    fig.savefig(path, **kwargs)
    saved.append(path)
    print(f"[scipilot-figure-skill] wrote {path}")

if grayscale_preview:
    gray_path = _grayscale_from(fig, basename, dpi=dpi)
    if gray_path:
        saved.append(gray_path)
return saved

def _grayscale_from(fig, basename: str, dpi: int) -> str | None:
    """
    导出灰度预览版用于色盲安全检查。
    优先用 PIL 转灰度；找不到 PIL 时退化为重新画图（关闭颜色）。
    """
    try:
        from PIL import Image
    except ImportError:
        print("[scipilot-figure-skill] Pillow not available; "
            "grayscale preview skipped.", file=sys.stderr)
        return None

    png_path = f"{basename}.png"
    _ensure_parent(png_path)
    fig.savefig(png_path, dpi=dpi, bbox_inches="tight")

    gray_path = f"{basename}_grayscale.png"
    Image.open(png_path).convert("L").save(gray_path)
    print(f"[scipilot-figure-skill] wrote {gray_path} (grayscale preview)")
    return gray_path

def _demo(out_basename: str) -> None:
    """Generate a small demo figure for quick smoke testing."""
    import numpy as np
    rng = np.random.default_rng(7)
    x = np.linspace(0, 10, 50)
    y1 = np.sin(x) + rng.normal(0, 0.1, x.size)
    y2 = np.cos(x) + rng.normal(0, 0.1, x.size)

    fig, ax = plt.subplots(figsize=(3.5, 2.625))
    ax.plot(x, y1, label="sin", marker="o", markersize=3)
    ax.plot(x, y2, label="cos", marker="s", markersize=3, linestyle="--")
    ax.set_xlabel("x")

```

```

ax.set_ylabel("y")
ax.legend(frameon=False)

paths = export_figure(
    fig, out_basename,
    formats=["pdf", "svg", "png"],
    size_inches=(3.5, 2.625),
    dpi=300,
    grayscale_preview=True,
)
print("\nDemo done. Files:")
for p in paths:
    print(f"  {p}")

def _cli() -> int:
    p = argparse.ArgumentParser(description="scipilot-figure-skill figure exporter")
    p.add_argument("cmd", choices=["demo"], help="`demo`: 跑一张演示图导出 4 种格式")
    p.add_argument("--out", default="./scipilot_demo",
                    help="输出 basename (默认 ./scipilot_demo)")
    args = p.parse_args()
    if args.cmd == "demo":
        _demo(args.out)
    return 0

if __name__ == "__main__":
    sys.exit(_cli())

```

B.12 输入与运行环境记录

```

#!/usr/bin/env python3
""" 生成可复现运行清单，并隔离 SKILL_ROOT 与 PROJECT_ROOT。 """

import argparse
import hashlib
import importlib.metadata
import json
import platform
import sys
from datetime import datetime, timezone
from pathlib import Path

SKILL_ROOT = Path(__file__).resolve().parents[2] / '.agents/skills/math-modeling'

def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()

```

```

def build_manifest(
    inputs,
    seed,
    parameters,
    command,
    packages,
    runtime_name="python",
    runtime_version=None,
    dependencies=None,
):
    files = []
    for item in inputs:
        path = Path(item).resolve()
        if not path.is_file():
            raise FileNotFoundError(f" 输入文件不存在: {path}")
        files.append({"path": str(path), "sha256": sha256_file(path), "bytes": path.stat().st_size})

    versions = dict(dependencies or {})
    if runtime_name == "python":
        for package in packages:
            try:
                versions[package] = importlib.metadata.version(package)
            except importlib.metadata.PackageNotFoundError:
                versions[package] = None
        runtime_version = runtime_version or sys.version.split()[0]

    return {
        "schema_version": 1,
        "created_at_utc": datetime.now(timezone.utc).isoformat(),
        "random_seed": seed,
        "input_files": files,
        "runtime": {
            "name": runtime_name,
            "version": runtime_version,
            "platform": platform.platform(),
            "dependencies": versions,
        },
        "key_parameters": parameters,
        "reproduce_command": command,
    }

def _is_within(path: Path, parent: Path) -> bool:
    try:
        path.relative_to(parent)
        return True
    except ValueError:
        return False

def resolve_output(project_root, relative_output):
    project = Path(project_root).resolve()
    if _is_within(project, SKILL_ROOT):
        raise ValueError("PROJECT_ROOT 不能位于 SKILL_ROOT 内部")
    output = (project / relative_output).resolve()
    if not _is_within(output, project):

```

```

        raise ValueError(" 输出路径必须位于 PROJECT_ROOT 内部")
    if _is_within(output, SKILL_ROOT):
        raise ValueError(" 输出路径不能位于 SKILL_ROOT 内部")
    return output

def main() -> int:
    parser = argparse.ArgumentParser(description=" 生成数学建模复现清单")
    parser.add_argument("--project-root", required=True)
    parser.add_argument("--input", action="append", default=[])
    parser.add_argument("--seed", required=True, type=int)
    parser.add_argument("--parameters", default={}, help="JSON 对象")
    parser.add_argument("--command", required=True)
    parser.add_argument("--package", action="append", default=[])
    parser.add_argument("--runtime", choices=["python", "matlab"], default="python")
    parser.add_argument("--runtime-version")
    parser.add_argument("--dependencies", default={}, help=" 依赖或 MATLAB 工具箱版本 JSON 对象")
    parser.add_argument("--output", default="results/复现清单.json")
    parser.add_argument("--overwrite", action="store_true")
    args = parser.parse_args()

    output = resolve_output(args.project_root, args.output)
    if output.exists() and not args.overwrite:
        print(f" 错误: 输出已存在, 未覆盖: {output}", file=sys.stderr)
        return 1
    parameters = json.loads(args.parameters)
    if not isinstance(parameters, dict):
        raise ValueError("--parameters 必须是 JSON 对象")
    dependencies = json.loads(args.dependencies)
    if not isinstance(dependencies, dict):
        raise ValueError("--dependencies 必须是 JSON 对象")
    manifest = build_manifest(
        args.input,
        args.seed,
        parameters,
        args.command,
        args.package,
        runtime_name=args.runtime,
        runtime_version=args.runtime_version,
        dependencies=dependencies,
    )
    output.parent.mkdir(parents=True, exist_ok=True)
    output.write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
    print(output)
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

B.13 Python 依赖版本

```
numpy==2.2.6  
scipy==1.15.3  
pandas==2.3.3  
openpyxl==3.1.5  
matplotlib==3.10.9  
Pillow==12.3.0  
fonttools==4.64.0  
pymupdf==1.28.2
```